

RESEARCH MONOGRAPH AND IMPLEMENTATION DESIGN

Interpreting Regulation with Accountable Ensembles

From SFC circulars to competing interpretations,
bounded resolution and verified Java controls

LegalMath

A technical account for compliance, legal, engineering and model-risk readers. The running case is SFC circular 23EC46 and a retained, deliberately unapproved implementation of its selected gift restriction. The proposed ensemble makes agreement, disagreement, missing evidence and the limits of automatic resolution visible before a control reaches a bank system.

22 September 2026

Complete author draft, version 1.0

Public-source and synthetic examples. Independent compliance, technical and reader acceptance remain pending. Proposed extensions are distinguished from the currently implemented workbench throughout.

The decision this book supports

A private bank cannot treat a software test result as a complete answer to a regulatory question. A program may reproduce a decision table exactly while the table overlooks an exception, applies to the wrong activity, or relies on a term whose meaning has never been established. The practical question is how to make those mistakes harder to introduce, easier to discover, and impossible to hide inside a confident-looking automated answer.

This book develops an interpretation workbench that retains several candidate readings, investigates the reasons for and against them, checks their consequences, and keeps unresolved questions attached to the control. Multiple methods provide different opportunities to discover error. A discrepancy opens a recorded resolution process with finite limits. Reaching those limits produces a report of the remaining uncertainty and prevents release of an affected control; it does not cause the system to choose the most popular answer.

The intended reader need not know a formal legal language, an argumentation semantics or a tree-search algorithm. Those methods are developed through a single circular and deliberately contrasting client scenarios. Engineers should be able to derive data structures, state transitions and acceptance tests from the text. Compliance readers should be able to challenge the source mappings, scope, classifications and review decisions without reading the software.

There are two distinct contributions. The first is an explanation of what the literature and the existing LegalMath evidence establish. The second is a new engineering design that combines those methods. The design is a proposal for implementation and evaluation; published results about another task do not establish its performance on SFC circulars. Finite search also cannot establish that every defensible legal reading has been found. The design instead makes coverage obligations, source dependencies, disagreement and stopping reasons explicit, and assigns responsibility for the decisions automation cannot settle.

The ten chapters follow the order in which a bank would have to justify a control. The initial circular exercise exposes the gap between execution and interpretation. Formal meaning and relevant languages then make the alternatives precise. Legal argumentation and search preserve competing readings. The central ensemble chapter specifies their bounded investigation. Verification, Java integration, service contracts, evaluation and continuing operation complete the route from a source document to an attributable bank decision.

The companion machine-readable contracts describe a proposed extension, not a silent alteration of the accepted runtime. The existing oracle and source snapshots are preserved. A successful document build establishes that this manuscript compiles; it does not supply the independent legal judgment, empirical validation or reader feedback

required to adopt the product.

Contents

The decision this book supports	i
1 A correct program can enforce the wrong rule	1
1.1 The decision before the decision	1
1.2 What the existing experiment actually established	2
1.3 Five places where meaning can be lost	3
1.4 The denominator problem in completeness	4
1.5 Why agreement is useful and insufficient	5
1.6 The object being classified is part of the rule	6
1.7 Time changes the question being answered	7
1.8 From an answer to an accountable decision	8
2 The second circular, from source to an unreleased control	10
2.1 The source packet and the question it supports	10
2.2 Reading the surrounding circular before isolating the rule	11
2.3 Naming the six inputs precisely	11
2.4 The named cases are an argument about the rule	12
2.5 What the 729 combinations cover	16
2.6 Mutations ask whether the tests can notice an error	16
2.7 A new discrepancy: what counts as a discount?	17
2.8 A complete dry run of the proposed workflow	18
2.9 From the JSON oracle to independent adjudication	20
3 What can be proved, and under which assumptions	21
3.1 Giving a correctness claim a precise target	21
3.2 Quantifiers change which entity carries the obligation	22
3.3 Necessary conditions, sufficient conditions and permissions	23
3.4 A proof obligation should expose its assumptions	24
3.5 Expressions, types and meanings	25
3.6 Unknown is an information state	26

3.7	Conflict, scope and errors are not extra Boolean values	27
3.8	A readable specification with a fixed translation	28
3.9	How a preservation argument is constructed	29
3.10	Counterexamples and the domain they live in	30
3.11	Proving a workflow property	31
3.12	Temporal duties require histories	32
3.13	An assurance claim is a conjunction of different evidence	33
4	Languages that make interpretation inspectable	35
4.1	Choose the question before choosing the language	35
4.2	Rules do not all behave like ordinary implications	35
4.3	Controlled English: readable structure with restricted freedom	36
4.4	Catala: defaults, exceptions and source-linked authoring	38
4.5	Metadata is part of the meaning: LegalRuleML	39
4.6	Stipula: a contract as parties, resources and transitions	40
4.7	A small state contract that a reader can execute by hand	41
4.8	What contract verification does and does not buy	43
4.9	Other normative languages clarify different distinctions	45
4.10	Decision tables and business-rule engines	46
4.11	A solver can improve reasoning after a translation	46
4.12	Roundtrip verification and targeted repair	48
4.13	Evidence spans and distant qualifications	48
4.14	Testing paths, dates and inconsistent constraints	49
4.15	A synthesis for the first implementation	50
5	Constructing and comparing competing interpretations	52
5.1	The object of search is a legal choice	52
5.2	Legal theories are more than predicted outcomes	53
5.3	What case-based argument contributes	53
5.4	AGATHA: explicit construction of rival theories	54
5.5	Arguments, attacks and unresolved conclusions	55
5.6	Burdens, assumptions and evidence in Carneades	56
5.7	Breadth before expensive investigation	57
5.8	Tree search and the meaning of a reward	58
5.9	Choose questions that separate the surviving readings	59

5.10	Uncertainty about meaning is not a vote count	60
5.11	What can be reused from the adjacent projects	60
6	An accountable ensemble with bounded resolution	62
6.1	The design objective and its limits	62
6.2	Why diversity has to be designed	63
6.3	The ensemble members and their responsibilities	64
6.4	Build a source perimeter before interpreting it	65
6.5	Freeze blind proposals before reconciliation	66
6.6	Detect discrepancies at the right level	67
6.7	An issue needs an identity that survives retries	67
6.8	Statuses describe evidence, not optimism	68
6.9	Choose a remedy that can answer the question	69
6.10	A resolution matrix prevents the wrong kind of closure	70
6.11	Resolution can reveal that the first question was wrong	72
6.12	The investigation graph should preserve shared causes	73
6.13	Finite budgets are part of the semantics	74
6.14	No progress is a substantive stopping reason	75
6.15	A controller that can be implemented and checked	76
6.16	Termination and the boundary of the proof	77
6.17	Durable actions, leases and ambiguous execution	78
6.18	What counts as a justified repair	79
6.19	Scores that assist investigation without pretending to be probabilities . .	80
6.20	A complete dry run: repairing an explicit omission	81
6.21	A complete dry run: an ambiguity that survives the budget	82
6.22	The uncertainty report is a first-class result	83
6.23	Agreement can still conceal a missed obligation	84
6.24	Interaction between uncertainty and operational action	85
6.25	Acceptance conditions for the central method	86
7	From an accepted interpretation to verified Java behavior	88
7.1	The specification becomes the comparison target	88
7.2	Facts are evidence-bearing values	88
7.3	Exact arithmetic prevents a second kind of ambiguity	89
7.4	Unknown, conflict and scope have an order	90

7.5	Validation failures are part of observable behavior	91
7.6	Independent oracles and their limits	92
7.7	Mutation tests measure what the checks can notice	93
7.8	Symbolic comparison requires a feasible domain	93
7.9	Property tests should follow the semantics	94
7.10	The actual generated Java interface	95
7.11	Traces are part of the delivered meaning	96
7.12	Event histories add obligations that a snapshot cannot express	97
7.13	Build, verification and approval form an acyclic chain	98
7.14	A correct decision can still race with the transaction	99
7.15	A verification dossier that supports a release decision	99
8	An implementation contract for the interpretation workbench	101
8.1	Extend the existing product at a defined boundary	101
8.2	Separate durable records from generated prose	101
8.3	The run and policy records	102
8.4	The candidate record carries the interpretive commitment	103
8.5	The issue and action records make automatic work accountable	104
8.6	Coverage records connect source units to consequences	105
8.7	Storage transactions and event history	106
8.8	An API with reviewable state transitions	106
8.9	The review interface should expose the decisive difference	108
8.10	Provider adapters are deliberately narrow	108
8.11	Security protects the meaning of the evidence	109
8.12	The ordered implementation sequence	110
8.13	Acceptance scenarios that exercise the whole cycle	112
8.14	The first vertical slice and its completion rule	112
9	Measuring error, uncertainty and the value of redundancy	114
9.1	The experiment must ask the right question	114
9.2	Define the unit of evaluation	114
9.3	Build the reference before seeing the model's answer	115
9.4	Construct a corpus that exposes the expected failures	116
9.5	Use a baseline ladder with comparable resources	117
9.6	Measure omissions, alternatives and useful abstention	118

9.7	Severity and direction of error need explicit treatment	119
9.8	Zero observed errors does not mean zero risk	120
9.9	Calibration asks a narrower question than truth	120
9.10	Conformal prediction can support sets under stated assumptions	121
9.11	Search coverage and semantic completeness are different	123
9.12	Evaluate the reviewer, not only the model	123
9.13	Promotion decisions and the next evidence needed	124
10	Operating the product without losing its evidence	126
10.1	The bank remains responsible for the control	126
10.2	Start with a narrow operational perimeter	126
10.3	Keep regulatory meaning distinct from bank policy	127
10.4	Shadow operation is a study, not a hidden release	128
10.5	Source monitoring begins a new interpretation cycle	129
10.6	A worked amendment scenario	129
10.7	Bitemporal replay explains rather than rewrites history	130
10.8	Operational responses to unknown and conflict	131
10.9	Incident analysis should locate the failed boundary	132
10.10	Availability failures must not change the meaning of a result	132
10.11	Model updates are method changes	133
10.12	A release readiness review with concrete evidence	134
10.13	Three operational cases for the implementation team	135
10.14	Maintaining a useful literature and method record	136
10.15	A practical sequence from manuscript to product	137
10.16	What the completed prototype will and will not establish	137

Chapter 1

A correct program can enforce the wrong rule

1.1 The decision before the decision

A distributor proposes a marketing offer involving an investment fund. The offer includes an incentive. A compliance control must determine whether the incentive falls within a restriction, whether an exception applies, and whether the relevant evidence is sufficient to make that determination. A software team can make these questions look simple: gather a few fields, evaluate a condition, and return a result to the bank's Java system. The difficult decisions have already occurred before the condition runs. Someone has decided what counts as an incentive component, what the words in the circular cover, what evidence establishes a promotional link, and which version of a referenced rule applies.

The distinction is consequential. Suppose a gift restriction covers both a named investment product and a type of investment product. An implementation that checks only the first route may work perfectly for a demonstration using one named fund. Its unit tests may be comprehensive with respect to the code that was written. Its developer may correctly explain every branch. The defect is that a branch required by the adopted interpretation never reached the code. Testing generated branches alone will not reveal a missing source obligation.

Now suppose that both promotional routes are represented, but an exception for a fee discount is applied to an entire offer containing a fee waiver and a separate voucher. The formal expression can be well typed, consistent and executable. Its output can be reproduced by Python, Java and a solver. The unresolved question is the object to which the exception attaches. A Boolean variable named `discount` conceals that question unless its definition identifies the assessed component and the evidence supporting the classification.

These are hypothetical failure scenarios derived from the retained 23EC46 exercise, not findings about a deployed bank system. They explain why the user regards interpretation error as a material operational risk. Regulatory action, reputational damage and accountability for an erroneous control make the quality of the decision process important. This book does not estimate the probability or financial severity of an enforcement outcome. It specifies the evidence a bank would need to understand, challenge and authorize its own interpretation.

The system we want to build therefore begins one step earlier than conventional rule

implementation. It asks what the bank has grounds to encode, which alternative readings have been considered, what remains unknown, and who has authority to resolve the uncertainty. Code generation follows that record. It does not substitute for it.

The unit of work is a *regulatory change assessment*. It includes source material, a declared business scope, proposed interpretations, affected controls, data requirements, implementation changes and review decisions. A single circular can create several such assessments because different provisions may concern different actors or activities. Conversely, one assessment may need several circulars, Code provisions, FAQs and earlier decisions. Treating one downloaded page as a self-contained legal specification would lose these relationships.

1.2 What the existing experiment actually established

The present LegalMath workbench provides a useful starting point because it has already crossed the engineering boundary from a formal rule to generated Java. Its second-circular exercise concerns a deliberately narrow interpretation of paragraph 10 of SFC circular 23EC46, dated 24 October 2023 (SFC, 2023). The example assesses one incentive component in a specified distributor and SFC-authorised-fund profile. Whether the component is a gift, whether it is a fee discount, and whether it has a relevant product link are supplied classifications. The program does not establish those classifications by reading an offer document.

The exercise preserved a packet of 22 named expected outcomes before constructing the candidate rule. A separate truth-table implementation enumerated 729 combinations of true, false and unknown values across six inputs. The actual generated Java class and the Python evaluator agreed on the declared results. Four deliberately altered rules were compiled and detected by the named cases. Those are substantial engineering observations: they show that the exercised implementation computes the proposed rule in the checked domain, and that the cases can expose several particular errors.

They do not provide independent evidence that the proposed rule is the correct reading of the circular. The same implementing agent authored the interpretation and the expected outcomes. Writing one before the other makes later alteration detectable; it does not create a second legal judgment. The separate truth-table oracle removes a direct dependency on the candidate's abstract syntax tree. It still expresses the same chosen formula. If that choice omitted a legal condition, all three implementations could agree on the omission.

The candidate also retained five unresolved review issues and two unresolved source dependencies. Its attempt to enter review was rejected. It remained a draft and no release was created. That refusal is part of the useful result. The workbench did not mistake successful compilation for authority to deploy a regulatory control. The monograph preserves this example as a record of an unapproved candidate; resolving the cited dependencies here by assumption would destroy the very distinction the example teaches.

The broader application acceptance contains 154 passing tests, but its drafting provider returns supplied fixtures. It has not interpreted arbitrary circulars in a measured live-model evaluation. The acceptance count is evidence about source intake, data contracts, execution, review workflow and delivery under the tested conditions. It is not an estimate of translation accuracy. A dashboard that presents the count without the target of the tests would invite a stronger conclusion than the evidence supports.

This distinction suggests the question the new product must answer. Given a source packet and a proposed operational scope, can it expose a material misinterpretation before that misinterpretation becomes an approved executable control? Success requires more than generating valid syntax. It requires finding missing alternatives, retrieving the context that changes a reading, and making disagreements concrete enough for a competent reviewer to resolve.

1.3 Five places where meaning can be lost

It is tempting to describe the workflow as one translation from English to Java. That description hides several different kinds of error. The first is source error: the reviewed text is incomplete, extracted incorrectly, superseded, or wrongly treated as authoritative. An omitted footnote can remove a definition. A copied paragraph can lose a heading that limits its scope. A source's publication date can be mistaken for its effective date. None of these failures requires a language model to reason incorrectly; the model may never receive the material needed for the correct decision.

The second is interpretation error. The words are present, but their legal function is misrepresented. A qualification becomes an unconditional rule. An exception is attached to the wrong object. A statement about possible classification becomes a finding of fact. A normative expectation is converted into an operational block without recording the bank's choice. These are decisions about meaning, not merely extraction or programming defects.

The third is specification error. The reviewer may have a defensible interpretation, but the intermediate rule expresses something else. A conjunction is replaced by a disjunction, a threshold loses equality, or an unknown condition is interpreted as false. The specification can also fail through insufficient expressiveness: a time-dependent obligation may be forced into a one-off Boolean test. A software language that cannot represent a required distinction must report the gap rather than approximate it silently.

The fourth is execution error. Java can evaluate a correct intermediate rule incorrectly because of a compiler defect, numeric conversion, time-zone mismatch, or inconsistent evidence handling. A data adapter may supply dollars where the rule expects cents. A build may package an older runtime. An apparent match on a few simple examples can miss a defect in unknown-value or conflict precedence. These failures are often amenable to strong formal or exhaustive checks once the specification and input domain are fixed.

The fifth is operational error. The correct Java result can be used for the wrong client,

product, source version or transaction time. Consent can change between evaluation and order commitment. A system can treat the absence of one prohibition as permission to proceed despite unresolved checks elsewhere. A reviewed rule can remain active after its source is replaced. These are failures of integration and change control, even when the individual rule evaluator is correct.

Figure 1.1 gives each transition its own question. The arrows do not imply that success flows automatically from left to right. Every arrow needs evidence appropriate to the claim it makes.

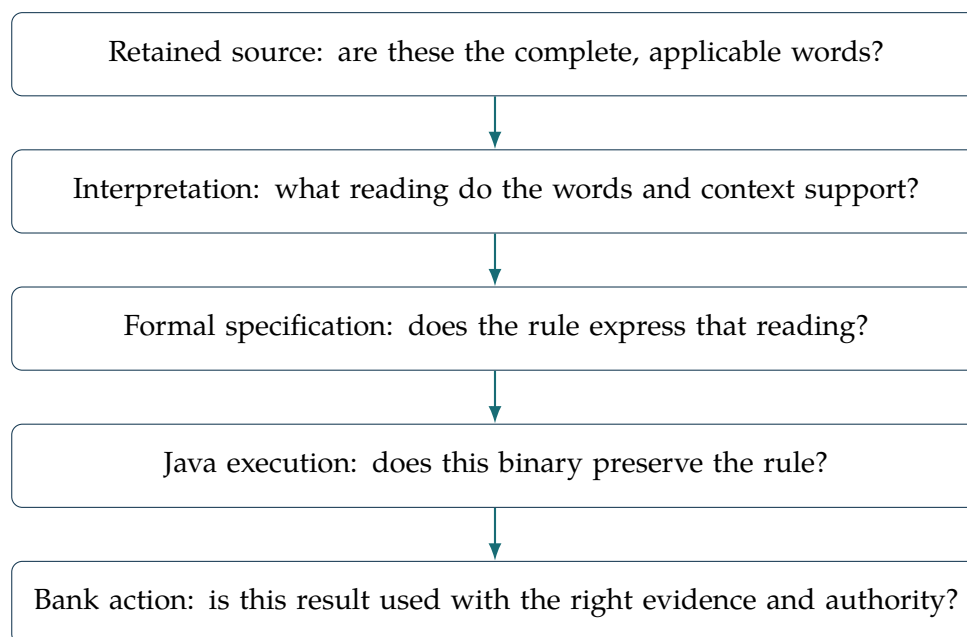


Figure 1.1: A correct downstream step does not repair a mistaken upstream premise. Each transition has a different assurance question.

A useful consequence follows immediately. Two checks count as meaningful redundancy only to the extent that their opportunities for failure differ. A second compiler can challenge code generation. It cannot detect a missing source document if it receives the same incomplete specification. A second language model may propose a different reading, but it cannot authenticate a source merely by describing it confidently. Redundancy must be designed around the failure being investigated.

1.4 The denominator problem in completeness

Suppose a drafting agent extracts eight requirements from a circular and produces eight rules. A coverage report then shows that every extracted requirement has a rule, every rule has a test, and every test passes. The result sounds complete. It is complete only relative to the agent's own extraction. If the circular contains a ninth operative requirement, the report has no row on which that omission can appear. The system has used its answer to define the denominator of its completeness measure.

The source inventory must therefore originate independently of the proposed rule set. At the document level it records paragraphs, headings, annexes, footnotes, tables and incorporated references. At the normative level it records potential duties, prohibitions, permissions, definitions, exceptions and qualifications. These two inventories serve different purposes. A paragraph inventory can establish that a passage was considered, but one paragraph may contain several operative propositions. A rule inventory can establish that an identified proposition has a disposition, but it cannot prove that the propositions were all identified.

For the retained circular, the existing inventory contains 18 numbered paragraphs and four footnotes. Only one paragraph has an executable control in the second-circular example. Other passages concern context, other controls, qualitative assessment or external dependencies. It would be wrong to describe the circular as implemented because its selected gift rule passes 729 input combinations. It would also be wrong to implement every paragraph mechanically: an observation about market practices is not necessarily a new obligation, and a qualitative duty may need a reviewed procedure rather than a fabricated numeric threshold.

A complete disposition process gives every inventoried proposition a reasoned destination. It may be linked to an executable rule, to a reviewed manual procedure, to a justified exclusion from the declared assessment, or to an open question. An open question is visible incompleteness. An exclusion requires a scope rationale and reviewer authority. A manual procedure needs an owner and evidence of performance; attaching the word “manual” to an inconvenient clause does not implement it.

The same denominator problem occurs in search. If five candidate interpretations agree, the correct conclusion is agreement among those five candidates under their recorded assumptions. The system has not measured the interpretations it failed to generate. Breadth constraints and targeted challenges improve the opportunity to find missing readings, but there is no finite list of prompts that proves unrestricted legal interpretation has been exhausted. The product must report which dimensions were explored and which remain outside the search.

This makes completeness a collection of bounded claims. We can check that every retained paragraph has a disposition. We can check that every cited reference has been resolved or explicitly left open. We can check that each represented exception has positive, negative and uncertain cases. We cannot infer from those facts that no relevant circular exists elsewhere or that every legally defensible reading has been found. A bank needs a process for setting and maintaining its regulatory perimeter as well as a translator inside that perimeter.

1.5 Why agreement is useful and insufficient

An ensemble is a group of methods whose results are examined together. In a prediction task it is common to combine answers by majority vote or averaging. For regulatory interpretation, the useful unit of combination is often a reasoned claim: one method identifies a scope restriction, another locates an exception, and another generates a case

that exposes a missing promotional route. Their outputs need not be identical to contribute to the same assessment.

Agreement can provide evidence that a representation is stable across some transformations. A controlled-English sentence and its formal rule may produce the same decisions. A back-translation may preserve the formal structure. A second implementation may agree with the first. Each observation rules out some failure explanations. None rules out every explanation. In particular, methods that share the same omitted premise can agree for the same wrong reason.

Disagreement is therefore not merely a nuisance to suppress. It may be the first observable sign of a missing definition, a conflict in evidence, or a genuine interpretive choice. If one candidate treats a product-type promotion as covered and another does not, the difference points to a specific source phrase. If two candidates disagree only when a classification is unknown, the problem may be their evaluation semantics. If two sources have different effective intervals, the disagreement may be temporal rather than substantive. The controller should classify these possibilities before attempting a repair.

The converse is equally important: disagreement alone does not make every candidate legally respectable. A generated rule that deletes an express exception may be a useful negative control but a poor interpretation. The system should retain it long enough to explain the rejection and test the detectors, while distinguishing it from a reading that remains genuinely unresolved. A shortlist filled with contrived alternatives can increase reviewer workload without improving assurance.

The proposed ensemble consequently has two obligations. It must preserve material dissent until there is a recorded reason to resolve it, and it must identify the evidentiary standing of that dissent. An objection supported by an applicable source is different from an unsupported model suggestion. A solver counterexample proves a difference in formal consequences; it does not by itself give one interpretation greater legal authority. A human reviewer can accept a reading for a declared scope, but that decision must not be mislabelled as a machine proof of English meaning.

Chapter 6 will specify how these distinctions become a finite automatic process. The core idea is that a discrepancy creates work with a defined purpose: retrieve missing authority, correct an encoding, investigate an alternative or clarify a fact. Repetition without a new diagnostic objective is not additional assurance. When the permitted actions have been exhausted, the system must make the remaining issue easier for a reviewer to understand.

1.6 The object being classified is part of the rule

The incentive example reveals a general weakness of compact decision tables: they often define a field without defining the entity to which it belongs. A field called `is_discount` could describe a payment, a fee line, a marketing offer, a product or a client relationship. These objects are not interchangeable. If a fee waiver and voucher are separate

incentive components, a property of the waiver cannot automatically be transferred to the voucher. The rule's subject identity is therefore part of its meaning.

The source-to-rule specification must state the unit of assessment before introducing its predicates. In this example it is one proposed component within a documented offer. The specification must also state how components are identified and when a purported separation is legally meaningful. A data designer cannot settle that second question solely by assigning two database identifiers. Operational decomposition needs to reflect the reviewed legal analysis; otherwise a bank could accidentally change the meaning of an exception by changing how it stores an offer.

The same problem appears in private-bank suitability workflows. A statement about a client's status does not necessarily apply to every account, every product category or every transaction channel. Consent may be linked to an identified service and period. A relationship manager's assessment may have a subject, scope and evidence date that differ from those of a wealth record. The intermediate representation must retain these bindings, not merely a collection of client-level Boolean fields.

A useful specification sentence therefore contains both a predicate and its subject: "component C in offer O is classified as a fee discount under definition D, on the evidence recorded in assessment A." That sentence is longer than "discount = true," but the extra information explains what the Boolean means. The eventual runtime can carry compact identifiers so that ordinary transactions remain efficient. The review record retains the definitions and evidence those identifiers name.

This is also why a language model's ability to extract entities is insufficient. It may correctly identify the words "fee waiver" and "voucher" while failing to determine the legal relationship between them. Extraction can propose candidates for classification. The system must separately record the reasoning that accepts their relationship and the uncertainty that survives it. That separation lets a later source clarification update a definition without pretending that the underlying marketing document changed.

1.7 Time changes the question being answered

An apparently straightforward rule can yield different legitimate assessments at different times. The regulation may change, the facts may change, or the bank may learn something new about an earlier event. These are three distinct sources of variation. A source publication date identifies when a document was issued. An effective interval identifies when a reviewed rule is intended to apply. An evidence recording time identifies when the bank had the relevant information. Substituting one date for another changes the assessment.

Consider a gift classification entered on Tuesday for an offer made on Monday. A historical review can ask what the bank should conclude now about Monday's offer, using Tuesday's evidence. It can also ask what the system could conclude on Monday from the information then recorded. Both questions are useful. They require different knowledge cutoffs. Replaying the earlier decision with all subsequently available facts would

conceal the information gap that existed at the time.

The current runtime makes assessment time and knowledge cutoff explicit through `valid_at` and `known_at`. Known evidence has a validity interval and a recording timestamp. An expired classification becomes unknown for an assessment outside its interval. Evidence recorded after the historical cutoff cannot be used in that historical assessment. A superseding record names the record it corrects; simple arrival order does not determine which assertion is authoritative.

These are engineering conventions selected for a precise replay question. They do not determine the legal effect of every late discovery or correction. A corrected factual record may establish that an obligation was performed on time, while the bank's earlier inability to demonstrate that fact remains part of the audit history. A regulatory amendment may apply prospectively, retrospectively or with transition rules that need interpretation. The translator must represent the relevant temporal question before the runtime can answer it.

The ensemble must therefore challenge time assumptions as well as the visible substantive condition. One candidate can be correct under a historical source version and wrong under the current one. Another can use a current definition to interpret an earlier transaction without justification. A disagreement report that omits the source revision and assessment times can make these distinct questions look like a conflict about the same rule. Immutable context identifiers are necessary to know what is actually being compared.

1.8 From an answer to an accountable decision

A useful product result contains more than a sentence announcing compliance or non-compliance. It identifies the precise condition evaluated, the reviewed scope, the facts and definitions used, the source version, the proposed interpretation, the evidence of correct execution, and any unresolved dependency. These details serve different readers. A compliance reviewer needs to see the interpretation. A data owner needs to see the evidence mapping. An engineer needs reproducible inputs and an artifact identity. A release owner needs to know which approvals bind the exact binary being deployed.

The design should not force every reader to inspect every detail on every case. The review interface can begin with the decision and the material reason, then allow inspection of assumptions and counterexamples. The underlying record must still support that inspection. A concise explanation is valuable when it is a faithful view of a fuller record; it is dangerous when it replaces a missing record with persuasive prose.

Accountability also requires distinguishing an interpretation from a bank policy. A bank may decide to prohibit a class of offers while a regulatory question remains unresolved. That can be a sensible operational choice under its own authority. The system should record the policy owner, scope and reason for the restriction. It should not attribute the restriction to an SFC sentence unless the interpretation supports that attribution. Otherwise later reviewers cannot tell whether a change requires new legal

analysis or a policy decision by the bank.

Likewise, a manual review route must have a defined effect. “Refer to compliance” is incomplete if no owner, service expectation, required evidence or transaction state is specified. An unresolved assessment may hold an automated approval, route the case for review or invoke an already approved conservative policy. The choice belongs to the bank’s operating design. The translator should never invent a transaction permission merely because its retries have reached a limit.

The product’s ambition is thus substantial but specific. It should improve the quality of the questions reaching a human reviewer, preserve alternatives that would otherwise disappear, and verify execution of the interpretation that is eventually adopted. The next chapter follows the second circular through that entire route. Its apparent simplicity is useful: a small formula is sufficient to expose why independent source judgment, explicit scope and uncertainty cannot be replaced by a large number of successful program evaluations.

Chapter 2

The second circular, from source to an unreleased control

2.1 The source packet and the question it supports

The running example begins with an identifiable public document: SFC circular 23EC46, issued on 24 October 2023, concerning additional returns and other services or arrangements offered when marketing SFC-authorized funds (SFC, 2023). The retained experiment selected its paragraph-10 gift restriction. The circular as a whole discusses more than that restriction. A reader needs to know both what was selected and what remains outside the control before a result from the control can be interpreted.

The experiment preserved the official source response, an extracted text derivative and anchors connecting the selected rule to its source passages. On 22 September 2026, a fresh retrieval matched the retained raw bytes. This check establishes that the retained response matched what the retrieval route returned at that time. It does not establish the complete current legal position or resolve the effective versions of incorporated sources. A hash identifies bytes; it does not by itself establish their authority, completeness or legal effect.

The selected assessment concerns one proposed incentive component, offered by a distributor in promoting a fund within the declared profile. It is a question about a specific restriction. It is not a whole-offer compliance verdict. The distinction is visible in the result vocabulary. A true result means that the selected prohibition trigger is established. A false result means that this trigger is not established as true on the supplied, resolved facts under the specified semantics. It supplies no general permission to market the offer.

The retained packet explicitly leaves the applicable Code paragraph and cited FAQ unresolved. This matters twice. Those sources may settle the meaning or scope of a term; they may also establish that the selected profile is narrower than the underlying requirement. Their absence is therefore a release-blocking dependency, not a bibliographic inconvenience. The example is useful precisely because it continues to expose that absence after its executable tests pass.

The book uses this historical, selected packet to teach implementation and assurance. New commercial offers, cashback definitions and mixed-component scenarios introduced below are synthetic investigations. They are not reported SFC rulings. The unresolved references are not silently filled using a language model's recollection of a Code

provision. Doing so would convert a visible gap into an untraceable premise.

2.2 Reading the surrounding circular before isolating the rule

A translation that begins by copying the most convenient sentence can miss why that sentence is there. The circular's opening establishes its distribution context. The next passages describe observed practices, including additional features, standing investment arrangements and marketing. Those observations help identify the activity being discussed, but they are not automatically individual executable prohibitions. A source inventory should retain their function without manufacturing a rule for every paragraph.

The discussion then turns to additional returns and their possible regulatory consequences. Some questions concern the structure of an arrangement and public promotion. Others concern impressions created about the underlying fund. The gift discussion appears within that broader context. Consequently, a finding that one separate classification is absent does not clear the gift issue. The gift control is not conditional on first finding that every other regulatory concern applies.

Later passages address redemption arrangements, administrative cutoffs, advertising and the description of services. Some concerns require event or workflow evidence; others require qualitative assessment of communication. Those mechanisms differ from the paragraph-10 Boolean control. An adequate implementation of the whole circular would need to preserve those differences. It cannot simply accumulate a larger expression around the gift predicate and call the circular complete.

The existing disposition inventory has 18 numbered paragraphs and four footnotes. Its assignments are agent-authored and unreviewed. They include context, separate controls, qualitative review and unresolved dependencies. Paragraph 10 is the only numbered passage with an executable control in this exercise. A future independent reviewer should challenge both the assignments and the completeness of the inventory. Merely approving the final formula would leave the earlier decision about what to implement unexamined.

This provides an immediate requirement for the ensemble. One path must inspect the source independently of the candidates and ask what obligations, exceptions and dependencies need a disposition. Another path may develop the selected rule. Comparing those paths creates a place for an omitted requirement to appear. If both start from the same already-filtered list of paragraphs, the apparent redundancy begins too late.

2.3 Naming the six inputs precisely

The executable example has two scope inputs and four body inputs. Their names are convenient for software, but their definitions do the substantive work. The first scope

input records whether the assessed actor is a distributor in the declared profile. The second records whether the activity concerns promotion of an SFC-authorised fund within that profile. Both are reviewed classifications. The prototype does not infer them from the actor’s name or the presence of the word “fund” in a document.

The first body input records whether the proposed component is classified as a gift. The second and third record a promotional link to a specific product and to a particular product type. A link to funds from a particular fund house requires an explicit classification where the relevant example is used; the engine does not assume that every mention of a fund house creates such a link. The fourth body input records whether the entire assessed component falls within the fee-or-charge-discount exception.

The word “entire” in that final definition is an engineering expression of the proposed component-level reading. It prevents an assessed voucher from inheriting the classification of a separate fee waiver. It does not prove that the decomposition is legally correct. A reviewer must establish the relevant unit of assessment from the source and context. The definition and the review issue must travel together until that question is resolved.

For a compact mathematical account, let A denote established actor scope and S the fund/activity scope. Let G denote the gift classification, P the specific-product link, T the product-type link and D the discount exception. For now suppose these are known Boolean values. The body condition is

$$B = G \wedge (P \vee T) \wedge \neg D. \quad (2.1)$$

It asks three things together: is the component a gift, does either promotional route hold, and is the exception absent? The two alternatives occur inside one parenthesized condition. Replacing that disjunction by a test of P alone changes the rule for offers linked only to a product category.

Scope is assessed separately. When both scope inputs are true, the body answers the selected question. When a scope input is known false, the limited profile returns `OUT_OF_SCOPE`. When scope is unresolved, it returns `UNKNOWN`, even if a body condition might otherwise be false. The distinction prevents an unresolved applicability question from being concealed inside an apparently definitive in-scope result.

The current runtime also has a conservative conflict policy. It checks the static dependency closure of the requested rule before evaluating branches. Conflicting referenced evidence produces `CONFLICT`, including when a later scope branch would otherwise make the body irrelevant. This policy is part of the declared engineering semantics. It should not be attributed to the wording of the SFC circular. Chapter 3 explains why such a policy must be fixed before comparing implementations.

2.4 The named cases are an argument about the rule

The 22 named cases are most useful when read as a sequence of challenges to an interpretation. They do not merely fill rows in a test fixture. Each changes a fact or evidentiary condition that a plausible implementation might mishandle. The baseline case

is a component classified as a gift, linked to one specific fund, with no fee-discount exception, and known in-scope actor and activity.

2.4.1 The alternative promotional route

Case g01 establishes the baseline prohibition trigger. Under equation (2.1), G and P are true and D is false, so the body is true. This is a necessary example but a weak test on its own. An incorrect expression $G \wedge P$ would also return true, even though it omits both an exception and an alternative route.

Case g02 changes the promotional facts: no individual fund is named, but the component is linked to a fund category. Now P is false and T is true. The disjunction remains true. A candidate that omitted the product-type route would return false, making this case a concrete witness to the omission. The source's inclusion of that route supplies the interpretive objection; the different program outputs make its consequence inspectable.

Case g15 uses the fund-house example through a supplied reviewed product-type classification. Its purpose is not to write a text classifier for fund-house mentions. It checks that the chosen formal vocabulary has a place to represent the classification when review establishes it. This distinction prevents a source example from being silently generalized into a universal inference from an entity name.

Case g09 establishes a specific-product link while leaving the product-type link unknown. The known route is sufficient for the disjunction. Unknown information about a second route does not erase the first. Case g14 reverses the situation: the specific-product route is false and the remaining route is unknown. Here the unresolved fact can change the outcome, so the correct result under the declared semantics is unknown.

Taken together, these cases explain the meaning of “either” in an executable rule with partial evidence. The system must neither require every route to be known true nor convert every missing route to false. This is a semantic distinction, not a preference about how verbose the explanation should be.

2.4.2 The exception and the object it qualifies

Case g03 changes the baseline component to one classified as solely a fee or charge discount. The exception settles the selected body condition as false. The result says that this particular gift restriction is not triggered under the supplied classification. It does not resolve advertising, authorization or any other requirement associated with the offer.

Case g04 returns to a voucher while another component of the same offer is a fee waiver. The assessed component remains the voucher, so the waiver does not set its D value to true. This case challenges an implementation that stores a single offer-level discount flag and reuses it for every component. If that flag is the only available data, the adapter has insufficient information to apply the proposed component-level rule. It should request a finer assessment rather than derive a convenient value.

Case g07 leaves the discount classification unresolved for a component otherwise meeting the trigger. If D is false, the prohibition is triggered; if D is true, the exception applies. Both possibilities remain consistent with the supplied evidence. Unknown is therefore the informative answer. A system that turns this unknown into false would remove the exception by default; one that turns it into true would invent the exception.

Case g16 provides known discount evidence while leaving the gift and promotional classifications unknown. Within known scope, a true D makes $\neg D$ false, which settles this conjunction as false. The case demonstrates that every missing fact need not prevent every narrow conclusion. Its safety depends on the specific conclusion being reported. It would be unsafe to broaden the result into a complete clearance of the marketing arrangement.

The contrast between g04 and g16 is particularly useful for reviewer training. An established exception can settle a condition, but only for the object to which it applies. A precise value attached to the wrong subject is not better evidence than an unknown value attached to the right one.

2.4.3 Classification is not supplied by the formula

Case g08 concerns additional returns whose gift classification has not been settled. The circular's qualified treatment of such returns motivates an assessment; it does not establish G for every arrangement. The proposed candidate must retain unknown when the remaining facts would make that classification decisive. Replacing the missing judgment by an unconditional "additional return implies gift" would create a new interpretive rule.

Cases g06 and g22 establish that no gift is offered. In g06 the discount classification is also missing. The false gift predicate settles the body as false under the declared Boolean semantics. Case g22 makes the point with a known product link: a promotional link alone does not establish that a gift exists. These cases catch a candidate that assumed G from the context instead of requiring evidence for it.

Case g05 establishes that neither product-link route holds. This also makes the body false, while leaving other regulatory questions outside the result. It does not establish that every incentive unrelated to an identified product is permissible under all applicable requirements. The selected condition is only one element of a broader assessment.

Case g17 separates regulatory classifications. A separate assessment finds no structured product, but the component is a non-discount gift with a relevant product link. The gift trigger remains true. The case challenges a workflow that exits early after resolving one concern and never evaluates another. Correct integration therefore requires an inventory of outstanding controls, not a single reassuring flag named `regulatory_check_passed`.

2.4.4 Applicability precedes an in-scope answer

Cases g10 and g11 establish that, respectively, the actor and the product/activity are outside the selected profile. The response is out of scope. The underlying Code requirement may have a broader reach; the example cannot answer that broader question. Treating out of scope as false would erase the distinction between an in-scope condition that is not triggered and a question the selected control does not purport to decide.

Case g12 leaves the fund/activity scope unknown. The system cannot establish whether the profile applies. Case g13 leaves actor scope unknown while establishing no gift. A Boolean programmer might be tempted to use the false gift fact to return false immediately. The declared profile instead reports unknown scope. This preserves the fact that the system has not established the context in which its body answer would be meaningful.

This is a design choice that an independent implementation must reproduce. Another language could define a different scope-combination policy, but it would then implement a different specification. Differential testing only has a clear target after the policy has been made explicit. A reviewer must also decide how an unknown scope result affects the transaction workflow.

2.4.5 Contradiction, expiry and later knowledge

Case g18 supplies conflicting gift evidence. The result is conflict, not unknown and not the value favored by the latest-arriving record. Unknown means the evidence does not determine a value. Conflict means admissible records assert incompatible values. These require different remediation: obtaining missing evidence in the first case, and resolving or superseding contradictory evidence in the second.

Case g19 combines an out-of-scope actor with conflicting discount evidence. The current static-closure conflict policy returns conflict before scope evaluation. This is deliberately conservative. The example must teach the policy honestly because a reviewer might otherwise expect out of scope and mistake the difference for a Java bug. The policy is recorded as engineering semantics, subject to separate operational review.

Case g20 makes the gift evidence stale at assessment time. The rule does not continue treating an expired classification as current. Case g21 places the recording time after the historical knowledge cutoff. A historical assessment cannot use that later knowledge. Both become unknown when the missing gift classification remains decisive, but their reason codes and repair actions differ. Refreshing an expired record does not justify rewriting what was known at an earlier point in time.

All 22 cases therefore contribute a distinct aspect of the specified behavior. Some concern the wording of the selected restriction. Others concern evidence, scope and operational conventions needed to execute the reading. The case packet should label those sources of expected behavior separately. Otherwise an implementation convention can acquire the apparent authority of a legal sentence simply because both are stored in the same JSON file.

2.5 What the 729 combinations cover

Each of the six abstract inputs takes one of three values: true, false or unknown. The number of assignments is therefore

$$3^6 = 729. \quad (2.2)$$

This is a statement about a finite abstraction. It does not count legal interpretations, marketing documents, evidence histories or real client offers. The enumeration is exhaustive for that abstraction because it visits every combination of the six declared values.

The independent truth oracle handles scope first. For the four body facts it replaces each unknown by both possible Boolean values and evaluates the formula under every completion. If every completion returns true, the oracle returns true; if every completion returns false, it returns false; otherwise it returns unknown. The code does not import RuleIR or the reference evaluator. This makes it useful for detecting a defect in how the expression was encoded or evaluated.

For this particular body, each input occurs once and with a single polarity. The completion calculation agrees with the current three-valued operations. That agreement should not be generalized to every expression. For example, a formula containing a variable and its negation can be true under every Boolean completion while a compositional three-valued evaluator returns unknown. The next chapter derives that difference. The oracle's representation is independent of the candidate AST, but its semantic scope remains specific.

Conflict, stale evidence and later-recorded evidence are not extra members of the three-value enumeration. Their normalization and precedence are exercised by named cases and other application checks. The count 729 must not be used to imply exhaustive testing of those mechanisms. Nor does it exhaust numerical inputs, dates, arbitrary rule graphs or the Java runtime's entire implementation. A result summary should always name the finite object that was exhausted.

The comparison checked full semantic responses and relevant traces, while respecting each backend's own identity. It would be wrong to require the Python and Java engine identifiers to be identical: those identifiers describe different implementations. It would also be wrong to ignore all metadata in a comparison, because source, evidence and rule identities are part of an explainable decision. Chapter 7 specifies the distinction between semantic agreement and implementation-specific provenance.

2.6 Mutations ask whether the tests can notice an error

Successful cases can be uninformative if they would also pass a wrong program. Mutation testing addresses this by deliberately changing the rule and asking whether the checks detect the change. A mutation is not a plausible legal interpretation merely be-

cause it can be compiled. It is an instrument for testing the sensitivity of the evidence.

The first retained mutation removed the fee-discount exception. Cases g03, g07, g16 and g19 detected the change. Some of these detections concern the logical exception, while g19 also exposes the effect of changing the static dependency closure. Removing an expression can remove a referenced fact from conflict analysis. A mutation report should therefore explain why a case detects a change rather than treating every detection as the same kind of evidence.

The second mutation omitted the product-type route. Cases g02, g14 and g15 detected it. Their relationship to the source is direct: they isolate the route that the mutation removed, including a case where that route is unresolved. A baseline case with a named fund could not provide this evidence.

The third mutation assumed that a gift existed without requiring its evidence. Cases g06, g08, g18, g20, g21 and g22 detected the resulting change. The set includes known absence, missing classification, conflict, expiry and later knowledge. This shows why evidence-handling cases are necessary even for a small logical condition. A wrong classification default can alter outcomes across several superficially different scenarios.

The fourth mutation ignored the selected actor/product scope. Cases g10 through g13 detected it. The expected difference includes both out-of-scope and unknown results. If the test harness compared only a final allow/deny bit after flattening all other statuses, it could conceal these errors.

All four mutations were compiled into Java before comparison. The result thus concerns executable altered candidates, not a textual inspection of a proposed patch. Still, detecting four mutations does not establish detection of every possible implementation or interpretation defect. Mutation operators are chosen by people or software and can themselves omit an important failure mode. The ensemble evaluation must maintain a documented fault model and extend it when new failures are discovered.

2.7 A new discrepancy: what counts as a discount?

The existing tests assume that a reviewer can supply the discount classification. To investigate the missing interpretation step, consider a synthetic offer in which a customer pays a fee and later receives cash back. Two candidate definitions might be proposed. One restricts the exception to a reduction of the fee initially charged. Another includes a documented refund of a fee that was actually paid. The source packet alone has not established that both readings are legally tenable. Their purpose here is to expose a question whose answer affects implementation.

Fix the actor, activity and promotional link, and assume for the comparison that the component otherwise meets the gift condition. Let the fee paid be HK\$100 and the later rebate also HK\$100. Under the first proposed definition, the later transfer is outside the defined exception. Under the second, it may fall within the exception if the required refund relationship is established. Those conditions must be derived from authority or review; the example does not introduce them as new SFC requirements.

The next action should be a targeted investigation of the applicable definition, Code and FAQ, not a request for the same model to vote again on the word “discount.” A source that explicitly includes the relevant refund treatment would support refinement of the second candidate. A source that excludes it would defeat that candidate in the applicable scope. A source discussing another product, actor or period may be informative without settling the question. The retrieval result needs an applicability assessment before it can resolve the discrepancy.

Now alter the cash amount to HK\$150. If a candidate definition treats only a refund of an actual HK\$100 fee as falling within the exception, the extra HK\$50 raises a further classification question. It does not follow that the system can automatically split the transfer into a lawful and unlawful part. The legal unit of assessment and treatment of mixed benefits must first be settled. An arithmetic decomposition is easy; its legal significance is the question under investigation.

A second variation changes the evidence. The marketing material describes the transfer as a fee refund, but no record establishes that a fee was paid. The candidate’s definition may be clear while its decisive factual premise remains unknown. This discrepancy belongs to fact classification or evidence availability, not necessarily to interpretation. The resolution controller should request the relevant evidence or return unresolved, rather than changing the definition to make the missing record unnecessary.

A third variation combines a refund with a separate voucher. A whole-offer exception would remove the need to assess the voucher independently. The proposed component-level interpretation would retain it. This is a different branch in the hypothesis space from the timing of a refund. Preserving separate issue identifiers prevents a source that resolves the timing question from being incorrectly treated as resolving the mixed-offer question as well.

These variations show how an ensemble can make review more precise. It should produce a small set of materially different definitions, the source reasons for each, and the smallest scenarios on which they diverge. The reviewer then decides an identifiable question. A long narrative saying that an offer appears reasonable would provide less actionable evidence.

2.8 A complete dry run of the proposed workflow

The following dry run specifies proposed behavior. It is not a report that the new ensemble has already run. Start by creating an assessment whose context contains the retained circular revision, declared distributor/fund scope, selected effective interval and an inventory of dependencies. The historical engineering example uses a synthetic September 2026 interval; the workflow must not infer a regulatory commencement date from that setting.

The source-review path inventories the circular and flags the incorporated Code and FAQ as unresolved. The interpretation path proposes the component-level gift rule. A second interpretation path omits the product-type route. A definition-focused path

asks whether cashback is a discount and whether a combined offer changes the assessed unit. At this stage the system has a candidate set and several issues, not a winner.

The comparator identifies a decision difference between the complete promotional condition and the candidate omitting the product-type route. It constructs a case with a gift linked only to a fund category and no exception. The source-grounding check points to the relevant paragraph. The candidate author can propose a repair adding the missing route. The system stores the repaired candidate as a new revision, reruns the discrepancy case and the retained regression cases, and preserves the original candidate and its rejection reason.

The cashback definition issue follows a different route. Retrieval attempts to obtain the incorporated sources with the right identity and version. If the sources are unavailable, another language-model explanation cannot close the issue. The controller records the failed acquisition and the remaining dependency. It may retry an authorized retrieval route within its budget, or stop automatic work because no permitted action can supply the missing authority. The issue remains unresolved either way.

The mixed-offer issue can produce a useful witness even before legal resolution. The comparison holds the voucher facts fixed and toggles the presence of a separate fee waiver. If a candidate's voucher decision changes solely because of the other component's exception, the report explains the dependence. A reviewer must then confirm the unit to which the exception applies. The witness does not grant the controller authority to make that legal decision itself.

The accepted formal candidate can be compiled and checked in parallel with this research. Candidate compilation is valuable because it exposes implementation defects early. It does not change the issue status. The report may legitimately say that the candidate passed all specified execution checks and remains unavailable for release because source interpretation is unresolved. Keeping those two observations together is more informative than suppressing either.

When automatic work reaches its finite limit, the controller freezes a report. It identifies the source revision, surviving definitions, rejected encodings, discriminating scenarios, completed actions, failed actions, unresolved dependencies and the exact effect of each unresolved issue on the control. There is no final vote to select a legally preferred answer. The state is ready for targeted human review or blocked on a named dependency, according to the reason for stopping.

A designated reviewer can later resolve an issue using newly obtained authority. That creates a new context revision. Dependent candidate scores and comparisons are invalidated where appropriate, and the formal rule and tests are updated. The earlier report remains a faithful account of what was unresolved before the new material arrived. This is essential for reproducibility: a current reviewer must be able to see why the earlier system abstained.

2.9 From the JSON oracle to independent adjudication

The filename `oracle.json` is an engineering term. It designates a source of expected results for tests. It does not establish institutional authority or legal independence. The file states that the same agent authored it and the candidate. A future user interface should display that authorship rather than letting the label “oracle” imply a gold standard.

Independent adjudication should begin from the retained source packet and declared task, before exposing reviewers to the proposed formal expression. One reviewer identifies requirements, definitions and difficult cases. Another can independently develop or challenge the interpretation. Disagreements are recorded and, where possible, resolved with source reasons. A third role may adjudicate material disagreement. Independence here concerns information and responsibility as well as the identity of a person or model.

After that source-based stage, reviewers can inspect the candidate rules and their discriminating cases. This order reduces anchoring on the generated answer. It does not make reviewers infallible. The reference packet must allow an unresolved category, later correction and competing defensible readings. If a reviewer cannot determine the treatment of a cash rebate without the referenced FAQ, the correct reference outcome is not a guessed Boolean chosen to make benchmark scoring convenient.

The original 22 cases should remain frozen as historical engineering evidence. New adjudicated cases belong in a separately versioned packet with its own authors, source context and review decisions. A changed expected outcome should explain whether the earlier interpretation was rejected, the scope changed, new evidence became available or the evaluation semantics were corrected. Those causes imply different repairs and different consequences for existing releases.

The chapter has now followed a source to a rule, tested the rule, exposed unresolved definitions and specified how a future ensemble should handle them. The remaining task is to state precisely what each formal check can prove. Without that precision, a solver’s success could once again be mistaken for the missing judgment about meaning.

Chapter 3

What can be proved, and under which assumptions

3.1 Giving a correctness claim a precise target

The second-circular exercise leaves us with two questions that sound similar but have different evidentiary requirements. Does the program compute the proposed gift rule correctly? Does the proposed gift rule express a justified reading of the circular? The first question becomes mathematical once the rule and its inputs have precise meanings. The second includes deciding which meanings the words, context and authority support. A proof of the first cannot silently include a proof of the second.

To see why, suppose a programmer writes a formula that checks gifts promoting a specific product and omits the product-type route. A proof assistant can verify that an implementation returns exactly that formula's value for every Boolean assignment. The proof is correct about its target. The target is incomplete relative to the proposed source interpretation. Making the proof longer, using a more powerful solver or verifying another implementation will not add the missing route. The source-to-formula mapping has to be challenged directly.

This is not a special weakness of legal language. Formal verification always depends on the proposition that was formalized. The virtue of a formal method is that it makes the conditional claim exact and inspectable. It does not remove the need to establish that the proposition is the one the institution intended to rely on. Catala's separation between legal authoring and compiler semantics is useful precisely because it keeps these responsibilities distinct (Merigoux et al., 2021, core language and compilation sections).

Let E denote a versioned source packet and S a controlled specification adopted for a declared interpretation. Let F be a RuleIR program and J the generated Java artifact. Let A contain the explicit vocabulary definitions, domain restrictions and operational assumptions. The input domain D_A consists of facts and histories allowed by those assumptions. We can now state a proposed preservation obligation:

$$\forall x \in D_A, \quad \pi(\text{eval}_{\text{CNL}}(S, x)) = \pi(\text{eval}_{\text{IR}}(F, x)) = \pi(\text{eval}_J(J, x)). \quad (3.1)$$

The projection π selects the semantic result components that the contract requires to agree. These include the decision category and the specified explanation and evidence behavior. They exclude fields whose purpose is to identify different backends. Chapter 7 makes that projection concrete. Equality in equation (3.1) is a proposed formal

target for a supported language fragment, not a theorem already proved about the delivered application.

The claim that S is justified by E remains a separate interpretation claim. It can be supported by source anchors, explicit alternatives, reviewed definitions, adversarial cases and an attributable judgment. When some part of E has itself been translated into a formal language, a theorem can relate those formal objects. The choice that made the formal source representation faithful still needs justification. Repeating that translation boundary does not eliminate it.

3.2 Quantifiers change which entity carries the obligation

The phrase “for each” introduces a choice that a Boolean expression can easily hide. To see it without assuming knowledge of predicate logic, consider a synthetic requirement: every relevant benefit in an offer must satisfy a specified condition. Let $B(o)$ be the finite set of benefit components in offer o , and let $P(b)$ mean that component b has the property under investigation. The universal reading is

$$\forall b \in B(o), P(b). \quad (3.2)$$

It requires every component to satisfy the condition. An existential reading,

$$\exists b \in B(o), P(b), \quad (3.3)$$

requires only one. These expressions are not two styles of writing the same rule. For an offer with two components, one satisfying P and one not, the universal reading is false and the existential reading true.

The distinction becomes more consequential when an exception is involved. Suppose $G(b)$ identifies a gift and $D(b)$ an excepted discount. A component-level prohibition trigger might be

$$\exists b \in B(o), G(b) \wedge \neg D(b). \quad (3.4)$$

An alternative whole-offer encoding might be

$$(\exists b \in B(o), G(b)) \wedge \neg (\exists b \in B(o), D(b)). \quad (3.5)$$

The first asks whether any component is a non-excepted gift. The second suppresses the trigger whenever any component is a discount. A voucher combined with a fee waiver distinguishes them immediately: the second formula can let the waiver cancel the voucher’s effect, while the first continues to assess the voucher. These are proposed logical readings for analysis, not a legal determination about mixed offers under 23EC46.

A source interpretation must decide what the variable ranges over before a proof about either formula is useful. Is the subject a benefit, a contract, a campaign, a transaction

or a client relationship? Are components defined by economic substance, contractual terms or an operational product record? The formal notation exposes this question because $B(o)$ cannot remain undefined in an implementation-ready specification.

The current RuleIR fragment has scalar facts rather than unrestricted quantified collections. A reviewed adapter can create one snapshot per component and a host orchestration can aggregate results under an explicit rule. That aggregation is another specification obligation. It must define how true, false, unknown, conflict and out-of-scope component results combine, and how the host knows the component list is complete.

An empty list deserves attention. In classical logic, a universal statement over an empty set is true and an existential statement is false. That convention is mathematically coherent, but a missing component inventory must not be treated as an established empty offer. The adapter needs an evidence status for collection completeness. Otherwise a data omission can pass a universal check by vacuity. The same pattern occurs when no argument extensions are returned or no test cases are selected: an empty computation can look successful unless existence and coverage are checked separately.

Unknown classification also affects aggregation. If one component is known to trigger a prohibition and another is unknown, the selected logical aggregation may already establish a trigger. Yet the unknown component may remain relevant to another obligation or to the completeness of the offer review. A top-level known result should not erase the record of missing facts. The distinction between missing and blocking inputs in the runtime supports this more informative report.

3.3 Necessary conditions, sufficient conditions and permissions

Another frequent translation error changes the direction of implication. Consider a synthetic statement that a process may use a streamlined route only if consent has been obtained. Let S mean use of the streamlined route and C mean established consent. The necessary-condition reading is $S \Rightarrow C$: using the route requires consent. It does not follow that $C \Rightarrow S$ or that consent alone authorizes the route. Other eligibility conditions may still apply.

A truth-table example makes the difference concrete. With consent true and an unmet financial condition, the necessary-condition statement is not violated by refusing the streamlined route. An implementation that treats consent as sufficient would proceed. Both programs may mention the same two words, but their decisions represent different logical relationships.

Likewise, an exception to one prohibition is not a general permission. If the selected condition R is false, we know only that this particular trigger is not satisfied under the accepted facts and semantics. We do not infer that every other prohibition is false or every positive obligation fulfilled. The host must compose independently applicable controls without upgrading the absence of one trigger into universal clearance.

In a closed, complete formal theory, one can define permission in terms of the absence of a prohibition if that is the intended logic. Legal systems and practical source packets often do not provide the completeness needed for that move. Explicit permission, lack of evidence of prohibition and a bank's operational choice should therefore have different representations. The normative-language literature examined in Chapter 4 provides several treatments; RuleIR's narrow condition results do not choose a universal deontic logic.

The words “unless” and “except” also require a structural decision. A simple propositional rewriting may express an exception as a negated condition, while a default system represents a defeasible conclusion that can be overridden. These can agree on a selected complete fact pattern and differ under missing evidence, conflicting exceptions or additional rules. The translator must identify which behavior the approved specification requires.

For the circular example, using $\neg D$ inside the selected body is an explicit modeling decision tied to the reviewed factual definition of discount. If D is unknown, the program should not act as though the exception is certainly absent. The three-valued semantics makes that uncertainty visible. A closed-world database query that treats a missing discount row as false would encode a different assumption, which requires separate evidence and approval.

3.4 A proof obligation should expose its assumptions

A useful proof statement includes the source-independent mathematical property, the accepted semantic definitions and the domain assumptions. For example: for every well-typed Boolean expression in a specified grammar and every normalized fact assignment without conflict, the reference and target evaluators produce the same three-valued result. This statement is precise enough to prove by structural induction. It does not include the claim that a particular English paragraph was mapped to the right expression.

The interpretation assumption can be represented separately as a reviewed proposition: under source packet S , profile P and definitions D , expression e represents the selected requirement. A preservation proof then establishes that target program $J(e)$ behaves like e . Composing these steps is useful, but the first premise remains a source and review judgment unless a restricted controlled language and its translation semantics are themselves the authoritative input.

A proof assistant will faithfully carry an unjustified premise into a theorem. If we assume that every cashback is a discount, it may prove the corresponding exception applies. The theorem is conditional on that assumption. It is not a discovery that the assumption is legally correct. The proof report must display such premises in language the reviewer can understand.

The same principle applies to abstraction. A model may replace a complex offer with six Boolean inputs. The proof can establish behavior for every assignment of those inputs.

To apply it to the real offer, the adapter must establish that the six inputs faithfully represent the relevant facts and that the omitted features cannot change the selected requirement within the declared scope. That relationship is an abstraction argument, not an automatic consequence of exhaustive truth-table testing.

The workbench should therefore attach an assumption register to every formal result. Each assumption names its provenance, operational meaning and review status. A proof that depends on an unresolved definition is retained as a valid conditional engineering result, while release remains blocked. This preserves the value of formal work without allowing its precision to conceal an uncertain starting point.

3.5 Expressions, types and meanings

A formal language is a collection of permitted expressions with rules for deciding what they mean. It need not look like a conventional programming language. A decision table, a restricted English sentence or a state-transition diagram can have formal semantics if each permitted construct has a precise interpretation. An unrestricted paragraph usually has more interpretive freedom than such a language allows.

The simplest fragment needed for the gift example contains Boolean literals, references to named facts, conjunction, disjunction and negation. In a compact grammar, an expression e is generated by

$$e ::= \text{true} \mid \text{false} \mid \text{fact}(a) \mid \text{not}(e) \mid \text{all}(e_1, \dots, e_k) \mid \text{any}(e_1, \dots, e_k), \quad k \geq 1. \quad (3.6)$$

Here a is the identifier of a declared Boolean fact. The grammar specifies which trees are valid expressions; it does not decide whether the fact denotes a legally appropriate classification. That meaning belongs to the declaration and its source-linked interpretation.

Types prevent several avoidable errors. A Boolean condition is different from a monetary amount. A date is different from a number of days. Money denominated in HK cents is different from a foreign-currency amount before conversion. The current RuleIR scalar types are Boolean, integer, HKD money and date. A comparison accepts compatible ordered values. Addition preserves a numeric type. No implicit conversion turns a missing Boolean into zero or a dollar amount into cents.

Typing is valuable because it can reject an entire class of malformed specifications before any client case is evaluated. It is also limited. Two variables can both have the Boolean type while referring to different subjects or definitions. A type checker cannot infer that an offer-level discount flag was wrongly substituted for a component-level classification unless the type and declaration system explicitly represent that distinction. The proposed extension should therefore retain subject, scope and definition identities alongside the scalar type.

An expression's semantics is its evaluation rule. For known Boolean inputs, `all` is true exactly when every operand is true, and `any` is true when at least one operand is true. The gift body is a tree whose root is an `all`, with children for the gift fact, the two-route

disjunction and the negated exception. The tree makes parentheses explicit. A source sentence can be attached to the root and narrower evidence anchors to its children.

The chosen language is deliberately finite and acyclic. A reference cycle such as rule A depending on B while B depends on A is rejected in the current profile. This avoids an undefined recursive evaluation. Richer legal formalisms can assign meaning to recursive or mutually defeating rules, but doing so requires a specified semantics. A compiler must not guess that an unsupported recursive construct means the same thing as the current acyclic profile.

3.6 Unknown is an information state

The bank often lacks a decisive fact when a rule is evaluated. In the gift example, the evidence may not establish whether an additional return is a gift. The program should distinguish that absence from evidence that no gift exists. We therefore need an unknown value, written U, alongside known true T and known false F.

The current Boolean operations follow strong-Kleene tables. The name identifies a precise three-valued calculation, which can be taught through the two cases already encountered. If a product-specific promotional link is true, then the disjunction of that link with an unknown type link is true. If the specific link is false and the type link is unknown, the disjunction is unknown. A true route suffices; an absent route cannot settle the other one.

<i>a</i>	<i>b</i>	$a \wedge b$	$a \vee b$
T	T	T	T
T	F	F	T
T	U	U	T
F	F	F	F
F	U	F	U
U	U	U	U

Table 3.1: The omitted rows follow by exchanging the operands. Negation swaps true and false and leaves unknown unchanged. The table describes logical values; evidence conflict and errors are handled separately.

An established exception can make the gift conjunction false even when another body fact is unknown, because one false conjunct is enough. That is the logic behind g16. It is not a general instruction to ignore missing information. Unknown scope is handled before the body, and conflicting evidence has its own precedence. The evaluation result must retain which question was answered and why the remaining unknown body facts did not change that answer.

A convenient symbolic encoding represents a three-valued Boolean by two ordinary Boolean indicators: one means “known true” and the other “known false.” The un-

known value has neither indicator. If t_a and f_a encode a , then

$$t_{a \wedge b} = t_a \wedge t_b, \quad f_{a \wedge b} = f_a \vee f_b, \quad (3.7)$$

$$t_{a \vee b} = t_a \vee t_b, \quad f_{a \vee b} = f_a \wedge f_b, \quad (3.8)$$

$$t_{\neg a} = f_a, \quad f_{\neg a} = t_a. \quad (3.9)$$

The domain excludes $(t_a, f_a) = (1, 1)$. That pair would assert both truth and falsity, which belongs to a different conflict mechanism in the current design. The equations reproduce Table 3.1: for example, true and unknown have true indicators 1 and 0, so their conjunction cannot be known true; neither has a false indicator, so the result is unknown.

This encoding allows a solver to search disagreements while preserving missing information. Replacing each unknown by an arbitrary ordinary Boolean inside one execution would ask a different question. It could produce a definite answer chosen from a possible completion without showing that other completions differ. A comparison tool must name the semantics it encoded, not merely report that it used a solver.

There is a subtler distinction between compositional three-valued evaluation and evaluating every Boolean completion. Let p be unknown. Strong-Kleene evaluation gives

$$p \vee \neg p = \mathbf{U} \vee \mathbf{U} = \mathbf{U}. \quad (3.10)$$

Under every consistent Boolean completion, however, the expression is true: if p is true the first term holds, and if p is false the second holds. A completion-based semantics can therefore return true. Neither calculation is ambiguous once its rules are specified; they are different semantics.

For the read-once gift body, the retained truth oracle's completion method agrees with the compositional calculation. That particular equivalence does not make the completion oracle a general replacement for RuleIR semantics. A future ensemble should treat an unexpected disagreement as a possible semantics mismatch before assuming that one interpreter contains a bug. This example is also a reason to test repeated variables when extending the comparison fragment.

3.7 Conflict, scope and errors are not extra Boolean values

Unknown truth and conflicting evidence should not be combined casually into one truth table. The current runtime first normalizes records by validity, recording time and explicit supersession. Two admissible non-superseded records with incompatible values produce conflict. That result records a problem in the evidence supporting a fact. It does not ask the Boolean evaluator to choose which record is right.

The static dependency closure contains every fact reachable from the requested rule through its scope, body, referenced rules and exception branches. Conflict in that clo-

sure prevents evaluation from treating a convenient branch as a way to evade contradictory evidence. A conflict in an unrelated declared fact does not automatically block the requested rule. This distinction makes the conservative policy bounded and reproducible.

After successful structural validation and conflict handling, scope determines whether the body is an appropriate question. A known-false scope produces out of scope. An unknown scope produces unknown with a scope reason. Only a known-true scope permits the in-scope body result. Thus an out-of-scope result is a statement about the relation between the query and the selected rule, not a false answer to the rule's substantive condition.

Malformed data and unsupported constructs introduce a further distinction. An input with an invalid date or duplicated conflicting identifier may be an error, not legal uncertainty. If the system cannot parse a request, it cannot honestly report that the regulatory condition is unknown on a well-defined request. The API should identify the boundary error and the location of the malformation. Operationally both may prevent automatic action, but their causes and repair routes differ.

We can therefore describe the result as a tagged value whose tag identifies the kind of answer. The current contract includes true, false, unknown, conflict, out of scope and error, together with diagnostics and provenance. The tags are not interchangeable labels for a single confidence score. The ensemble discrepancy taxonomy must preserve them so that a failed parser is not processed as a legal disagreement and an unresolved definition is not processed as a transient network failure.

The precedence is part of the specification. Reordering checks can change both a result and its explanation. The g19 example deliberately returns conflict even though an actor scope fact is false. A Java implementation that short-circuits on scope before consulting the required conflict closure would disagree with the current target. That disagreement is wrong relative to the declared execution semantics, independently of whether the bank later chooses to revise the policy.

3.8 A readable specification with a fixed translation

Controlled natural language keeps the familiarity of English while restricting how sentences are formed. The restriction matters because the reader should be able to determine the formal structure from the sentence without asking a model to interpret it again. Logical English provides one developed example of this approach through explicit templates, variable binding and restrictions on ordinary linguistic ambiguity (Kowalski et al., 2023, language and law examples).

For our narrow gift example, a specification can state: "For an assessed component in the established distributor/fund scope, the selected gift restriction is triggered if the component is a gift, at least one of the two specified promotional links holds, and the component does not meet the defined discount exception." The specification must then define each term and its evidentiary state. This sentence is reviewer-facing exposition.

A machine translation needs an accompanying restricted structure in which the three conjuncts, disjunction and negation are explicit.

An appropriate authoring interface might present one condition per line with typed connectives and named definitions. The reviewer sees the English label and can inspect the exact expression beneath it. Free text remains available for reasoning and caveats, but it does not secretly alter the executable condition. A change to a definition or connective creates a new specification revision. Editing an explanatory paragraph must not bypass the formal diff.

The controlled specification also records the legal character of the source. A statement framed as an obligation, a prohibition, a permission, a discretion or an expectation should retain that category. Translating all of them into a Boolean called `allowed` would destroy distinctions before verification begins. The current RuleIR chiefly computes conditions. A reviewed operational policy determines how those conditions affect a bank action. Richer normative and temporal languages are introduced in Chapter 4.

A fixed translation from restricted syntax to RuleIR is a valuable target for proof. Every permitted construct has a known translation rule; unsupported sentences are rejected or left as unresolved authoring work. If a reviewer writes an unrestricted sentence outside that syntax, a model can propose a restricted rendering, but that proposal returns to the interpretation process. The system cannot claim deterministic controlled-language compilation merely because an LLM usually produces valid JSON.

Roundtrip verification supplies an additional check. Translate a formal candidate into readable language, formalize the rendering again, and compare the two formal candidates. A mismatch can reveal drift. Agreement can show stability across the tested transformations. The same omitted source condition can survive both translations, so the roundtrip result remains one component of the evidence, as acknowledged by Amrollahi et al. (2026, method and limitations). It cannot replace the independent source inventory established earlier.

3.9 How a preservation argument is constructed

The phrase “prove the compiler correct” becomes more useful when the reader can see the structure of such a proof. Consider only the finite Boolean expression grammar in equation (3.6). Let its source evaluator use Table 3.1. Suppose a translation maps literals, fact references and each connective to corresponding target operations. Assume the target fact lookup returns the same normalized tagged value for every declared fact and that its primitive operations implement the same tables.

Proposition 3.1 (Preservation for the Boolean expression fragment). *Under those assumptions, translating any well-formed finite expression in equation (3.6) preserves its three-valued result for every normalized fact assignment.*

Proof. Use induction on the construction of the expression. A true or false literal has the same value by the literal translation. A fact reference has the same value by the lookup

assumption. For a negation, the induction hypothesis gives the same child value in both evaluators, and the target negation table maps that value to the same result. For a conjunction or disjunction, the induction hypotheses give equality for every child. Applying the identical connective table to the identical sequence of values gives the identical parent value. These cases cover every constructor in the grammar. The finite syntax tree ensures the induction reaches a base case on every path. $\square$

This small proof is exact, but its scope is intentionally visible. It assumes correct target primitives and lookup. It does not establish the current Java implementation satisfies those assumptions. It does not cover scope, conflict normalization, numeric operations, exceptions, errors or traces until separate lemmas include them. And it says nothing about whether the expression is a justified interpretation of paragraph 10.

The next proof obligations follow the same pattern. A normalization lemma must show that both implementations select the same admissible evidence under the same times and supersession relation. A scope lemma must preserve the tagged result and the decision to skip the body. A conflict lemma must preserve the static closure and conflict precedence. A trace lemma must connect each evaluated or skipped node to the required source and evidence identifiers. Only after these claims are assembled can the implementation claim be expanded from a Boolean value to the full semantic result.

The actual generated Java class currently binds an immutable validated bundle and invokes the Java semantic runtime. The proof boundary must follow that architecture. Proving a small emitted wrapper correct while assuming an unchecked runtime would leave most of the computation outside the theorem. Conversely, a proof about a generic runtime still needs to show that the generated class embeds the intended bundle and that the host invokes the correct class. The theorem's trusted components should be named rather than hidden under the word "compiler."

There is a useful discipline here for an implementation agent. Begin with a small formal fragment whose semantics and generated behavior can be inspected. Prove or exhaustively check its components, then extend the fragment with new constructors and corresponding obligations. A failed lemma is information about the specification or implementation. It is not a reason to weaken the claimed target without explaining the change.

3.10 Counterexamples and the domain they live in

To compare two candidate interpretations, the system first needs a common question and compatible inputs. Let F_1 and F_2 compute tagged results on the same declared input domain. A solver can search for an assignment satisfying

$$D(x) \wedge (\pi(\text{eval}(F_1, x)) \neq \pi(\text{eval}(F_2, x))) . \quad (3.11)$$

Here D states the permitted facts and relationships. In the product-type example, a witness sets gift true, specific-product link false, type link true and discount false, with

scope established. The assignment has different consequences under the complete and omitted-route rules.

The first solver check should establish that D is satisfiable. If the domain simultaneously requires a fact to be true and false, the difference query is unsatisfiable for an unhelpful reason: there are no admissible inputs. The correct response is `INCONSISTENT_DOMAIN`, not equivalence. This precheck is a concrete safeguard against a formally successful but meaningless verification result.

If the domain is satisfiable and the difference query is satisfiable, the assignment is a counterexample to equivalence in that domain. Before displaying it to a reviewer, replay it through the ordinary interpreters. That replay checks that the solver encoding and the runtime agree on what the witness means. If replay disagrees with the solver prediction, the comparison harness has failed and the witness cannot be used to adjudicate the candidates.

If the difference query is unsatisfiable, the supported conclusion is absence of a counterexample under the encoded semantics and domain. The strength of that result depends on the solver procedure and any checked certificate. A machine-checked certificate provides a different assurance level from trusting an unchecked solver response. A timeout, unsupported construct or unknown solver result remains unresolved; it must never be converted to successful equivalence because no witness was printed.

Domain assumptions can hide exactly the error we hoped to find. If a comparison asserts that every product-type promotion also names a specific product, the omitted-route mutation may become indistinguishable from the proposed rule. That assumption would need a source or business justification. It cannot be introduced merely because it makes the proof succeed. The report should show which constraints exclude a witness and where those constraints came from.

Minimal witnesses are useful for human review, but “minimal” also requires a definition. One option minimizes the number of facts changed from a familiar baseline. Another minimizes monetary amounts within meaningful bounds. Neither guarantees that the resulting scenario is legally or operationally feasible. Source and domain validation remain necessary. The smallest formal assignment may be a poor real-world question if it combines classifications that a reviewer would never jointly accept.

3.11 Proving a workflow property

Some of the strongest useful guarantees concern the review process rather than the legal reading. For example, the proposed ensemble should never mark an assessment releasable while a material discrepancy remains unresolved. This is a property of explicit records and transitions. It is amenable to a state-machine argument once the terms “material,” “unresolved” and “releasable” have precise stored meanings.

Let Q be an assessment state, $M(Q)$ its set of unresolved material issues, and $R(Q)$ the

predicate that a release is authorized. A required invariant is

$$R(Q) \implies M(Q) = \emptyset. \quad (3.12)$$

The invariant does not say that an empty issue set proves the interpretation correct. It says that known unresolved material issues cannot be ignored by the release transition. A faulty detector may fail to create an issue, which is why independent coverage and detector evaluation remain necessary.

To establish the invariant, the initial state has no release. Every transition that creates, reopens or materially changes an issue must invalidate dependent release eligibility. The only transition that authorizes release must check the issue set and all other required conditions in the same transaction that records the authorization. If a concurrent worker can add an issue after the check but before the release without a version check, the implementation does not preserve the invariant. Database transaction boundaries are therefore part of the proof assumptions, not an incidental engineering detail.

A second property concerns termination of automatic processing. Suppose a run has a finite number of reserved actions, and every dispatched action consumes one reservation even if the worker fails. If all automatic paths either consume a reservation or enter a terminal state, then no run can dispatch more actions than its budget. This argument requires crash and retry accounting; counting only successful responses would permit unlimited failed calls. Chapter 6 gives the exact controller and its termination measure.

These guarantees directly support the user's requirement. They can establish that the system reports remaining uncertainty and respects its retry limit. They cannot establish that the detectors discovered every possible uncertainty. The two responsibilities should be tested separately: detection quality against independent cases, and workflow integrity against state-transition and failure injection tests.

3.12 Temporal duties require histories

A Boolean assessment of an offer is not enough for every circular requirement. Some duties become active after an event and must be performed before a deadline. Others must remain satisfied throughout an interval. Consent can be granted, withdrawn and granted again. A formal specification for these cases needs histories and states, not just a snapshot of current fields.

Consider a hypothetical duty activated at time a with an exclusive deadline d . Under a declared non-preemptive achievement interpretation, a matching performance at time t satisfies the timing condition exactly when

$$a \leq t < d. \quad (3.13)$$

The inequality states an engineering profile; a source may require a different boundary or permit performance before activation. Those choices must be reviewed and encoded explicitly. The event must also match the duty's actor, action and subject. A timely

action by the wrong party is not satisfaction.

Failure to find a performance record is not by itself evidence that the duty was breached. The observation stream may be incomplete. The current event runtime therefore requires a completeness assertion through the deadline before inferring breach from absence. That assertion has provenance and a recording time. A process clock cannot create it. This mirrors the source-coverage problem: an empty retrieved set does not prove that the relevant object does not exist.

Late performance and breach are also distinct. A duty can be performed after its deadline without erasing the fact that timely performance was absent under complete evidence. A later correction may establish that an earlier performance was missed; that creates a new assessment with a different knowledge cutoff. The original assessment remains available. Normative logic and contract-state languages help express these distinctions, but their semantics must be selected to match the intended duty (Governatori and Rotolo, 2006; Hashmi et al., 2016; Crafa et al., 2021).

The current profile supports a limited event model. It does not automatically implement every waiver, sanction, compensatory duty or maintenance obligation that a legal document could contain. Unsupported temporal constructs should remain explicit implementation gaps. A translator that invents a deadline or converts a continuing duty into a single check would create a new rule while appearing to simplify the software.

3.13 An assurance claim is a conjunction of different evidence

We can now state the overall structure without pretending that one theorem settles the entire process. Source retention establishes which material was reviewed. Interpretation review establishes an attributable decision about that material within a declared context. Formal checks establish consequences and preservation properties for specified languages and domains. Integration checks establish how a result reaches a bank action. Each supports a distinct premise in the justification for using a control.

This structure prevents two opposite mistakes. The first is to dismiss formal verification because it cannot prove the meaning of unrestricted English. Preserving a reviewed specification across implementations is valuable, and workflow invariants can rule out important classes of operational failure. The second is to present formal verification as if it eliminated the interpretation and evidence premises. A perfectly verified formula still needs the right subject, source version and fact classifications.

For the bank's reviewers, the practical outcome is a set of answerable questions. Which parts of the interpretation are accepted and by whom? Which formal properties have actually been established? What lies outside the supported fragment? Which data mappings remain unresolved? What would reopen the conclusion? Those questions can be supported by exact records rather than an undifferentiated confidence statement.

The next chapter examines existing languages and methods in that light. Each is intro-

duced by the problem it solves: readable authoring, exceptions, alternative interpretations, events, or symbolic checking. Their combination must preserve those distinctions instead of assuming that choosing one formal language supplies the whole assurance argument.

Chapter 4

Languages that make interpretation inspectable

4.1 Choose the question before choosing the language

The gift example needs a condition over classified facts. A consent example needs a history of grants and withdrawals. A payment agreement may need parties, assets and deadlines. A dispute about the meaning of an exception needs alternative readings and their supporting reasons. These are related questions, but a representation designed for one does not automatically solve the others. The purpose of reviewing formal legal languages is to identify which structures help make each question precise.

A formal legal language is a language in which statements about a legal or normative domain have defined syntax and computational meaning. The word “legal” identifies the intended subject matter and useful abstractions. It does not mean the language has legal authority, that a program written in it is enforceable, or that translation into it establishes a correct reading. Authority still comes from the relevant source and institutional decision.

For our product, a useful language must support at least three forms of inspection. A reviewer needs to see how a condition corresponds to source text. An engineer needs to know exactly what values and events the program expects. A verification tool needs unambiguous semantics. A language that is expressive enough to represent any program may make verification harder; a language that permits only simple decision tables may fail to represent important temporal or exception structure.

This chapter develops the relevant mechanisms through examples. It does not choose a universal replacement for the bank’s Java system. The recommended architecture keeps a bounded executable representation, a richer interpretation record and explicit adapters for specialized analyses. The relationship between those representations must itself be checked. Adding more languages without defining that relationship would create new translation boundaries rather than redundancy.

4.2 Rules do not all behave like ordinary implications

An ordinary material implication says that if a proposition is true, another proposition is true. A legal norm often says that, under specified circumstances, someone ought to do something. The existence of the duty does not establish that the action occurred.

A rule requiring a document to be supplied cannot be used as evidence that the document was supplied. The distinction between facts and normative outputs is central to input/output logics (Makinson and van der Torre, 2000, secs. 2–4).

For a simple illustration, suppose an adopted specification says that a distributor must provide a named warning when a condition holds. The condition can create a duty record. Performance requires separate evidence of the warning being delivered to the correct recipient in the required context. If the program inserts the duty’s content into its factual database as though it had happened, every obligation can appear automatically satisfied.

Prohibitions and permissions also need care. Failure to derive a prohibition may mean the act is permitted under one explicitly defined logic; in a bank workflow it may instead mean the relevant sources or facts are missing. The literature distinguishes strong and weak permissions under particular defeasible-logical definitions (Governatori et al., 2012, secs. 2–4). Those definitions are not a general license to treat an empty query result as authorization. The selected RuleIR gift result remains a subcondition, with permission supplied only by a separate approved decision policy.

Many legal rules are defeasible: they normally support a conclusion, but an exception or stronger opposing rule can override that support. A defeasible rule is not the same as an ordinary implication with an informal comment attached. Its evaluation must account for applicable opposing rules and priorities. Defeasible-logic systems formalize different ways to do this, including rules that support conclusions and defeaters that block opposing support (Antoniou et al., 2000, sec. 2).

The engineering implication is simple but demanding. “Has exceptions” is not a complete semantics. The specification must say what happens when no exception is known, one is established, two compete, or an exception remains unknown. It must also say whether a priority is supplied by legal authority, an explicit interpretation decision or a programming convention. File order is not an adequate hidden source of legal priority.

The early logic-program treatment of the British Nationality Act demonstrated how a selected body of legislation could be decomposed into rules with inspectable reasoning (Sergot et al., 1986). That work is valuable here as an example of disciplined formalization, including attention to time and negative information. It should not be read as a claim that ordinary legal prose can always be converted automatically into a complete logic program.

4.3 Controlled English: readable structure with restricted freedom

Ordinary English permits pronouns, elliptical references, implied subjects and several possible attachments of a qualification. Those features make writing compact, but they leave a translator with choices. Controlled English restricts some of that freedom so a sentence can be mapped into logic by explicit rules. Logical English develops this

approach for law and education (Kowalski et al., 2023, language specification and examples).

Consider the sentence “If it is a gift and is linked to the product or its type, it is prohibited unless discounted.” The sentence is short, but “it” could refer to the offer or component, “its” needs a binding, and “discounted” does not identify the exception. A controlled rendering should repeat the subject where necessary and name each definition. Repetition in the language is useful if it removes a legally consequential ambiguity.

One possible authoring form for our own restricted profile is:

Listing 4.1: Illustrative LegalMath authoring notation. This is a proposed template, not claimed to be valid Logical English or Catala source.

```
subject: component C in offer O
scope:
  actor(0) satisfies distributor_profile
  activity(0) satisfies authorised_fund_promotion_profile
condition:
  C satisfies gift_definition
  AND (
    C has specific_product_link
    OR C has product_type_link
  )
  AND NOT (C satisfies fee_discount_definition)
consequence:
  selected_gift_restriction_triggered
```

The form makes four decisions inspectable: the subject, the scope, the condition and the named consequence. Every definition identifier refers to a versioned record. A reviewer can ask why a definition applies without reverse-engineering the syntax tree. An engineer can translate the connectives deterministically once the template is fixed.

Variable binding is a practical source of error. If one occurrence of C refers to the voucher and another to the whole offer, the sentence has changed its subject midway. The authoring language should enforce a declared variable and type environment. A reference to an undeclared subject should be rejected. An association such as “component C belongs to offer O ” should be explicit when the rule uses facts about both.

Negation requires similar precision. “There is no evidence that C is a gift” is not the same assertion as “evidence establishes that C is not a gift.” Logical-programming systems may distinguish explicit negation from failure to prove a proposition. Our evidence model records known false, unknown and conflict separately. A controlled sentence that asks about missing evidence should use that information state explicitly; it should not rely on a model to guess which sense of negation the author intended.

Controlled English improves the boundary between reviewed meaning and implementation. It does not remove the earlier task of choosing the controlled sentence from the original circular. A reviewer can still choose the wrong definition or omit a phrase. That is why the product retains source anchors, alternative templates and contrasting

scenarios alongside the readable rule. The language makes a chosen interpretation precise enough to challenge.

4.4 Catala: defaults, exceptions and source-linked authoring

Catala was designed around a recurring pattern in legislation: a general rule defines a value, and explicit exceptions may replace it. Its core calculus gives that structure a formal meaning; its authoring approach keeps legal text near the corresponding code (Merigoux et al., 2021, secs. 2–4). For a reviewer, this can be more intelligible than a long sequence of ordinary conditional statements whose priority is implicit in their order.

A hypothetical policy might normally require an explanation and waive a particular step under a reviewed condition, while restoring it if the client requests assistance. The important structure is the relationship between the general rule and its exceptions. A nested exception can express a priority that differs from two sibling exceptions. Moving a condition one level in that structure can change the result even when every word remains present.

For a small teaching example, suppose a base value is false and one exception returns true when g holds. If g is established, the exception supplies the value. If g is refuted, the base applies. If the specification supplies two sibling exceptions and both are active, the system needs a conflict rule. Catala’s core treatment distinguishes an absent applicable value from a conflict between exceptions; more than one active exception can conflict even when the values agree. Agreement of the values does not prove that their legal priority has been resolved.

LegalMath borrows explicit exception structure but adds its own treatment of incomplete evidence. In the present RuleIR profile, every sibling guard is evaluated. Two true guards conflict. One true guard with an unknown competitor remains unknown because the competitor may become active. One true guard with all competitors false selects its value. When all guards are false, the base is selected. This is a separately specified design; it is not a claim that RuleIR implements the whole Catala calculus.

The distinction matters for ensemble comparisons. A Catala model and a RuleIR model can be useful independent representations, but they do not become equivalent merely because both have an “exception” construct. The adapter must define how missing evidence, no applicable value and competing exceptions are represented. A comparison outside that common fragment should return unsupported or expose the difference, not choose a convenient interpretation.

Catala’s compiler work also illustrates how formal claims should be scoped. The paper establishes properties for a core translation; the production compiler and surrounding tooling are separate implementation objects. The retained repository inspection also records an admitted supporting lemma in the formalization notes. This project has not replayed those proofs. We borrow the explicit language structure and preservation discipline, while requiring independent evidence for any concrete adapter used in the

bank product.

Huttner and Merigoux (2022, sec. 4.2) discuss legal/programmer collaboration as a way to keep source meaning and executable rules connected. The useful lesson is organizational as well as technical. A legal reviewer can challenge an exception's interpretation while an engineer checks that its computational consequence matches that interpretation. Neither role should be expected to certify the other's entire domain from a single code review.

4.5 Metadata is part of the meaning: LegalRuleML

A formula alone does not say which provision it interprets, which jurisdiction it concerns, when it applies, or whether a competing formula is also defensible. LegalRuleML addresses representation of legal rules together with such contextual and provenance information. The work on legal interpretations explicitly represents alternative readings associated with source material (Athanasopoulos et al., 2014, secs. 2–4); the later OASIS specification provides a broader standardized vocabulary (OASIS LegalRuleML Technical Committee, 2021).

For the fee-discount issue, two candidate rules can point to the same source span while naming different interpretation records. One record might restrict the exception to an initial reduction; another might include a documented refund. Their source identity is shared, but their assumptions and consequences differ. The system should not assign a single interpretation identifier to both merely because the source paragraph is the same.

The representation must also preserve the authority of a selection. A legal reviewer may adopt one interpretation for a defined product and date range. That selection is an event with an author, reason and scope. It does not erase the alternative or establish that the alternative was syntactically invalid. A future source clarification may require reconsidering the choice. The provenance record is what lets the system determine which decisions depend on it.

LegalRuleML is a representation standard, not a turnkey engine for resolving every legal dispute. A serialized permission or priority still requires a specified evaluation semantics. Different reasoning engines may support different fragments or make different assumptions. For the prototype we should borrow the discipline of explicit context and alternatives without requiring the entire standard to become the first runtime dependency.

This leads to a useful separation in the proposed storage model. A source span identifies words. An interpretation claim identifies an adopted or proposed meaning. A formal rule identifies a computation. A review decision identifies who accepted which claim under which context. Keeping those identities separate prevents a hash of the source text from being used as if it also identified the meaning chosen from that text.

4.6 Stipula: a contract as parties, resources and transitions

Stipula becomes relevant when a legal arrangement changes through actions over time. Its original paper describes agreements, parties, fields, assets, functions, states and scheduled events (Crafa et al., 2021, secs. 3–6). A function can be available only to a specified party in a specified state, with a precondition on the current data. Executing it changes fields or resources and moves the contract to another state. This is a much richer object than a single decision-table row.

To understand the mechanism, consider a synthetic service agreement. It begins in a state awaiting agreement on terms. After the designated parties agree, it enters an active state. An authorized party can request a permitted service only while the agreement is active. A withdrawal changes the state so that the same request is no longer available under that agreement. A timed event may move the agreement into an expired state. This example illustrates a state model; it is not a claim about the legal effect of any particular bank consent.

The state matters because the same action name can have different consequences at different points in the history. A function called `requestService` cannot be understood by its input parameters alone if its availability depends on whether consent is active. A current Boolean copied from a stale database may also be inadequate. The state should be derived from an attributable history or a reviewed snapshot plus a complete subsequent event stream.

Stipula’s agreement construct explicitly identifies who participates and who must agree on the initial terms. This is useful as a modeling discipline: the identity of the party agreeing is not an untyped annotation on a field. For LegalMath, the analogous review question is who has authority to approve a meaning, data mapping or release. We borrow the importance of explicit parties and transitions, not a claim that a regulatory circular is itself a private contract between the bank and regulator.

Assets in Stipula are distinguished from ordinary fields. An ordinary numeric field can be copied as information; an asset transfer must account for the resource leaving one location and arriving at another. In the original semantics, asset movement prevents creating a negative quantity and preserves the relevant resource through transfer. This supports questions about duplicate spending, locked resources and the availability of contractual actions. It is not a model of market liquidity or a substitute for the bank’s general ledger.

For a synthetic escrow, suppose a contract holds a resource amount b and transfers q to a recipient. A local arithmetic description is

$$b' = b - q, \quad r' = r + q, \quad 0 \leq q \leq b, \quad (4.1)$$

where r is the recipient’s modeled holding. Adding the first two equalities gives $b' + r' = b + r$. This is a conservation property of the modeled transfer. It says nothing about whether the underlying asset exists off-system or whether the transfer is legally au-

thorized. Those are separate premises. The example shows how a formal language can support a useful exact property without claiming to settle the whole legal arrangement.

Scheduled events add another dimension. A Stipula event can schedule a continuation at a time and execute it only if the contract remains in a specified state. An action taken before the event can change whether that continuation is applicable. The ordering of an event and a function call at the same time is consequently significant. The inspected original semantics gives scheduled events precedence in the relevant equal-time situation. A bank adapter must not assume that this is the legally correct ordering for its own deadlines. The source, clock precision and event-order authority need review.

This example also distinguishes observation from permission. A formal contract may describe which function calls its own execution platform permits. A bank audit system must still record actions that occurred outside a permitted workflow. Deleting an observed transaction because the policy would have forbidden it would erase evidence of a possible violation. A normative monitor therefore needs a clear separation between advice about an action and recording its actual occurrence.

4.7 A small state contract that a reader can execute by hand

A complete miniature model helps distinguish the legal-language idea from its implementation in a particular tool. Consider a synthetic escrow agreement in which a payer deposits a fixed amount, a recipient may receive it after an accepted delivery, and the payer can recover it after an expiry if delivery has not been accepted. The example is chosen to teach states, parties, resources and time. It is not a model of an actual bank product or a proposed interpretation of the gift circular.

Let the contract state be one of UNFUNDED, HELD, PAID and REFUNDED. Let b be the amount held by the contract, p the payer's modeled external holding and r the recipient's holding. All amounts are nonnegative integer units. Let d be the agreed deposit, τ the expiry time and t the current event time. The parties and the meaning of accepted delivery are agreed inputs to this model.

The state transition table is the executable meaning of the miniature contract. It says which caller can perform which action, under what guard, and how the state and resources change. It is deliberately written in a neutral notation; it is not presented as tested Stipula source code.

Table 4.1: Synthetic escrow transitions, inspired by state-contract languages.

From	Action and caller	Guard	Effect and next state
UNFUNDED	Deposit by payer	$p \geq d, d > 0, t < \tau$	$p' = p - d, b' = d, r' = r$; enter HELD.

From	Action and caller	Guard	Effect and next state
HELD	Release by authorized acceptance process	Accepted delivery and $t < \tau$	$r' = r + b, b' = 0, p' = p$; enter PAID.
HELD	Expiry event	$t \geq \tau$	$p' = p + b, b' = 0, r' = r$; enter REFUNDED.
PAID or REFUNDED	Any repeat transfer	No enabled transition	No second transfer is permitted by this model.

Start with $p = 100, b = 0, r = 0$, and an agreed deposit $d = 40$. A valid deposit produces $(p, b, r) = (60, 40, 0)$ and state HELD. If delivery is accepted before expiry, release produces $(60, 0, 40)$ and state PAID. If expiry occurs first, the refund produces $(100, 0, 0)$ and state REFUNDED. The same deposit has two possible future paths, determined by events and their timing.

A reader can now check a conservation property without trusting a compiler. Define $W = p + b + r$, the total modeled resource. For the deposit transition,

$$W' = (p - d) + d + r = p + r = W, \quad (4.2)$$

because the initial contract holding is zero. For release, $W' = p + 0 + (r + b) = W$. For refund, $W' = (p + b) + 0 + r = W$. Each permitted transition therefore preserves the total. Since the initial holdings are nonnegative and the deposit guard requires sufficient funds, these transitions also preserve nonnegativity. This is a local derivation about the stated miniature model.

The terminal states prevent a second transfer. After release, no transition from PAID can release the same balance again. A Java implementation that updates balances but forgets the state change could violate that property under a repeated call. This is why the state transition is part of the contract, not just a user-interface label. An idempotency mechanism can support repeated requests, but must be specified consistently with the state model.

Time is equally substantive. The table uses $t < \tau$ for release and $t \geq \tau$ for expiry. At exactly τ , only expiry is enabled. If the intended agreement allowed delivery at the deadline, the guards would need a different boundary or an explicit ordering rule. A test performed one minute before expiry cannot establish the exact-deadline behavior. The Stipula literature's event ordering is therefore a modeling choice to understand, not a universal answer for every contract (Crafa et al., 2021).

The model also exposes a dependency hidden by ordinary prose: what establishes accepted delivery? It might be an authorized person's recorded acknowledgment, an external system event or a disputed factual assessment. The transition table assumes that this predicate is supplied correctly. Conservation and no-double-transfer proofs do not establish that assumption. If acceptance can be forged, the formal transfer properties may hold while the wrong person receives the resource.

The distinction between safety and progress is now visible. Conservation is a safety property: no permitted step creates or destroys the modeled resource. The ability to reach PAID or REFUNDED is a progress question. If the platform never processes the expiry event and delivery is never accepted, the resource can remain held. Proving eventual refund requires assumptions about time advancement and event execution, not merely the transition table's arithmetic. The contract-liquidity literature concerns such availability of actions and resources; it should not be confused with market liquidity (Laneve, 2023).

This miniature also clarifies what a Java/JML translation would need to preserve. A precondition specifies the caller, state and guard. A postcondition specifies the new balances and state. An invariant states nonnegative holdings and any conserved total. The verifier reasons about all allowed calls under those conditions. It must also account for the event mechanism and any loop abstraction used by the translation. The inspected Stipula-to-Java work provides a relevant route, with the restrictions already described (Hähnle et al., 2025).

For LegalMath, the corresponding benefit is a disciplined way to model review, consent and obligation lifecycles. A report awaiting review is not an approved report; a withdrawn consent is not an active generation; an opened duty is not fulfilled merely because a document was eventually delivered late. Explicit states and transitions make these distinctions executable and testable.

The correspondence is methodological rather than a claim that every circular is an escrow contract. Regulatory obligations can apply even when a party never agreed to a private contract, and actual events must be recorded even if a formal platform would reject them. A bank's monitoring system therefore separates the normative question of what should occur from the factual record of what did occur. The state model helps express that separation only when it is designed for the intended purpose.

A practical specification exercise follows directly. Before implementing a new history-dependent control, write its states, actors, events, guards and effects; execute one normal path, one boundary path and one violating observation by hand; identify the facts that the model assumes; and state which invariants and progress properties matter. Then choose a language or verifier that supports those properties. This sequence gives Stipula, Symboleo, eFLINT or a restricted custom monitor a concrete role rather than selecting a language because its name occurs frequently in the literature.

4.8 What contract verification does and does not buy

Once a contract has a transition semantics, tools can ask whether certain states are reachable, whether a resource can become trapped, or whether two contracts have corresponding observable behavior. Stipula's legal-bisimulation work gives a formal account of selected observable equivalences (Crafa et al., 2021, behavioral equivalence section). A bisimulation relates states so that an observable step on one side can be matched by a corresponding step on the other. It is a statement about modeled behavior under that definition of observation.

For the bank, an analogous question might ask whether a revised consent implementation preserves the reviewed behavior for grants, withdrawals and transactions. The observation set must include what matters: subject, category, time, permitted use and recorded violations. If the comparison ignores withdrawal, equivalence of the remaining observations would not establish the property the bank needs. Formal observation boundaries deserve the same scrutiny as ordinary input-domain restrictions.

Hähnle et al. (2025) demonstrate a translation from Stipula contracts into Java with JML specifications and verification using KeY. JML expresses properties of Java behavior, such as preconditions, postconditions and invariants; KeY reasons about whether the Java program satisfies them. This is especially relevant to our Java delivery target because it illustrates a route from a specialized contract language into a general-purpose verification environment.

The inspected work has explicit restrictions. Its event conditions are modeled symbolically, and the automated encoding keeps certain loop inputs constant across iterations. Its examples establish feasibility for the examined models, not complete fidelity to arbitrary calendars or external event streams. The official translator source was inspected in the earlier project review; no KeY proof was replayed. Adopting a similar route would require a precise mapping from our RuleIR and event profile to the Java/JML semantics, including the unknown and conflict results that ordinary Boolean contracts omit.

General verification cannot be promised for every expressive contract language. Delzanno et al. (2026) establish undecidability results for clause reachability in Stipula fragments and identify a decidable fragment under particular restrictions. The restrictions concern disjoint function/event source states and immediate events; they should not be confused with the different disjoint-cycle condition used in the Java/JML translation work. A shared word in two papers does not identify a shared theorem.

The practical consequence is to define bounded questions and supported fragments. A tool can report that no bad state was found up to a specified depth, or prove a property under a restricted transition model. It cannot turn a finite search into a complete reachability decision for a language whose general problem is undecidable. The report must retain the bounds so a reviewer can understand which behaviors were left unexplored.

Stipula's amendment literature is also relevant. A contract can change its declarations, state and permissible behavior through a specified amendment mechanism (Laneve et al., 2023, secs. 3–5). For regulatory controls, an amendment must similarly identify whether existing obligations keep their opening source version or migrate to a new one. Hot-replacing code without a state-migration interpretation can change the treatment of historical events. The versioned control and its transition obligations need separate approval.

4.9 Other normative languages clarify different distinctions

Symboleo models legal contracts using concepts such as obligations and powers, with lifecycle distinctions that are easy to lose in a Boolean implementation (Sharifi et al., 2020, secs. II–IV). Creating a duty is not always the same as activating it. A power can change a normative relationship, for example by suspending or terminating an obligation. These distinctions are useful when a bank workflow has review, activation, withdrawal and corrective actions with different legal effects.

The retained Symboleo paper and software inspection do not provide a complete implemented axiom set for LegalMath. Some supporting references in the early paper are not fully available in the retained edition. We can use the lifecycle questions to improve requirements, while treating any direct engine adoption as a separate interface and semantics audit. A familiar conceptual diagram is not a substitute for an executable contract.

eFLINT separates facts, actions and duties in a normative specification (van Binsbergen et al., 2020, secs. 3–5). A particularly useful distinction is that an impermissible action may still occur, change state and create a violation. This matches the difference between a bank’s pre-transaction advice and its post-event audit. A system that refuses to represent an unauthorized observed action cannot explain what happened after a control failure.

The eFLINT literature also shows why language versions matter. The later stable-model semantics provides a particular formal grounding and can produce no model or several models (Esterhuyse et al., 2025, translation and evaluation sections). Those outcomes carry information; selecting one model silently would conceal ambiguity or inconsistency. The newer language survey discusses implementations whose semantic guarantees differ (van Binsbergen et al., 2026). A proof or test result for one version should not be attributed to every interpreter bearing the same name.

s(LAW) combines legal formalization with logic programming that can explain positive and negative outcomes and explore possible models (Arias et al., 2024, secs. 2–5). This makes assumptions visible in a way that a single opaque classification may not. However, a model containing a hypothetical fact does not establish that the fact is true for the client. Counterfactual reasoning should propose what would change an answer, while the evidence service establishes whether the corresponding condition actually holds.

These methods suggest an interface in which normative classification, state transition and factual evidence are separate inputs to a decision. The implementation should reject an unsupported duty kind explicitly. A maintenance obligation, which must hold throughout an interval, cannot be implemented by checking that it was true once. A non-preemptive achievement obligation cannot be discharged by a performance before it came into force. The taxonomy in Hashmi et al. (2016, sec. 3) helps identify these choices; the precise profile still needs a source-grounded interpretation.

4.10 Decision tables and business-rule engines

Decision tables remain useful when their rows and hit policy have a precise meaning. A table can show, for example, that a known discount exception changes the selected trigger while an unknown exception leaves a material question. The reader must know what happens when several rows match: choose the first, require a unique row, collect results or apply a priority. DMN makes such hit policies explicit ([Object Management Group, 2024](#), sec. 8.2.11).

That explicitness is the valuable part. A spreadsheet whose row order silently resolves competing legal exceptions is harder to review because its legal priority is hidden in presentation. Changing the sort order can change the answer. A unique-hit table can reject overlap, but it cannot determine whether the missing row represents an omitted legal situation. Independent source coverage and boundary cases remain necessary.

Drools, OPA and other rule engines can execute policy logic and integrate with enterprise applications. The question for this product is not whether they are capable software. It is whether the selected engine's semantics, evidence model, explanation behavior and deployment interface match the adopted specification. Unknowns, conflicts, exception priorities and event ordering need explicit mappings. A backend name is not an assurance claim.

For the prototype, a small RuleIR is easier to specify and compare than a large engine with many unsupported features. Other engines can serve as independent reference encodings for a supported fragment after that fragment is defined. The comparison must include the behavior that ordinary Boolean examples leave out. Engine diversity without semantic alignment can produce many discrepancies that reflect adapter choices rather than flaws in the legal interpretation.

4.11 A solver can improve reasoning after a translation

A neural-symbolic system separates tasks that language models and formal engines perform differently. A language model proposes a formal representation of prose. A symbolic engine evaluates or reasons about that representation. The engine can give a reproducible answer to a precise logical question and can expose inconsistency, unsupported syntax or a counterexample. The translation that supplied its premises remains a separate source of error.

Logic-LM uses language-model formulation followed by an appropriate solver and feedback-driven refinement ([Pan et al., 2023](#), sec. 3 and prompt appendices). LINC combines language-model formalization with first-order theorem proving and analyzes failures at the translation and reasoning boundary ([Olausson et al., 2023](#), method and error analysis). These methods support the design choice that a model should not be asked to simulate every logical calculation in prose. They do not support treating successful solver execution as evidence that a legal requirement was fully captured.

A syntax repair illustrates the benefit and the limit. Suppose a candidate uses an unde-

clared symbol in the gift rule. The parser identifies the symbol and rejects the encoding. A repair prompt can ask for the missing declaration or corrected reference while preserving the intended interpretation. The repaired candidate is then validated again. This process may solve a concrete representation error. It does not justify adding a new legal assumption merely because doing so makes the formula easier to satisfy.

ARc develops policy-model creation, human vetting, structured inspection and symbolic checking in a combined system (An et al., 2025, sec. 3 and appendix A). Its policy creation procedure divides a document into units, proposes types, variables and constraints, and composes them into a model. Its vetting mechanisms include linting, template-based English inspection and testing. These are relevant to our product because they make the proposed model available for correction before individual answers rely on it.

Composition creates its own interpretation problem. Two units may use the same word for different legal concepts, or different words for the same concept. ARc uses embedding-based clustering in its variable-unification procedure. For our high-stakes specification, such a merge should be a proposal with provenance and validation. Merging “offer discount” with “component discount” because their descriptions are similar could remove the distinction that g04 was designed to test. A similarity score cannot establish identity of legal concepts.

ARc’s answer-verification procedure also uses redundant translations and symbolic agreement checks. Its reported finding categories distinguish ambiguous translation, impossible premises, invalid conclusions, entailed conclusions and satisfiable but unentailed claims. The distinction is useful: if a model permits both an outcome and its opposite under incomplete premises, the system can show scenarios for each rather than invent a definitive answer. For LegalMath, these scenarios become review questions tied to the candidate set and the source context.

The word “soundness” in ARc’s empirical evaluation needs careful translation into a product claim. The paper defines an empirical quantity based on false positives across requests; it is not a theorem that the English policy was formalized correctly. The denominator differs from precision among approvals. The manuscript therefore borrows the inspection and disagreement mechanisms without importing a claimed legal-error probability. Chapter 9 defines the denominators our own evaluation would need.

The tax-reasoning work of Jurayj et al. (2026) further illustrates the importance of task boundaries. Supplying gold rules, retrieving annotated cases and extracting facts are different assistance conditions from generating rules from a new circular. A comparison must account for those inputs and the human effort that produced them. A system given a reviewed rule library should not be presented as evidence that the same system can independently interpret unreviewed prose.

4.12 Roundtrip verification and targeted repair

Roundtrip verification asks whether a formal representation survives a journey through another representation. In the method of Amrollahi et al. (2026), source English is formalized, the formula is translated back into English, and the reconstructed English is formalized again. A solver compares the first and last formulas. If they differ, diagnosis identifies a likely translation stage to repair, and dependent stages are regenerated.

This is attractive for our user interface because the intermediate English can make a formal candidate easier to inspect. Suppose the original formula contains both product-link routes but the back-translation mentions only a specific product. Re-formalizing that sentence may lose the product-type route. The resulting counterexample exposes a defect in the explanation or translation chain. A reviewer should not be shown that explanation as if it faithfully described the formal rule.

The same construction can miss a shared omission. If the first formalization already omits the product-type route, a faithful back-translation of that incorrect formula and a faithful re-formalization will agree. The roundtrip then passes. What survived is the first formula, including its error relative to the source. The paper explicitly recognizes this limit; our ensemble must retain a source-grounding path that does not simply inspect the same back-translation.

Repair therefore needs a named target. If the formal rule is justified but the explanation drops a condition, repair the explanation and its dependent roundtrip outputs. If the original rule omits a source condition, create a new interpretation candidate and revalidate its downstream artifacts. If the two formulas use incompatible vocabularies, repair the mapping before claiming the formulas disagree about one question. A generic “make these consistent” instruction does not distinguish those cases.

The stopping rule is equally important. A roundtrip procedure can run for a bounded number of attempts and still end in disagreement. The product should report the original and revised formulas, the counterexample, diagnosis, actions attempted and the remaining issue. It should not keep repairing until some pair agrees and then discard the path that made the original source objection visible. The ensemble controller extends this principle from one translation chain to several methods and issue types.

An explanation generated by deterministic templates provides a useful additional path. It is constrained by the formal syntax and can be checked for completeness of its displayed operands. It may be less elegant than free prose, but it offers a stable view of what the program actually says. A freely generated explanation can still help readers if it is compared against that view and does not replace it as the sole account of the rule.

4.13 Evidence spans and distant qualifications

An answer supported by a source span is easier to review than an unsupported assertion, but the span itself may be insufficient. A qualification can appear in a footnote, definition or distant section. ContractNLI studies document inference with evidence an-

notation and highlights challenges involving references and exceptions (Koreeda and Manning, 2021, task, annotation and error-analysis sections). For our product, the relevant lesson is to retain the supporting and limiting passages needed for the inference, rather than a single convenient quotation.

The distinction between “not mentioned” and “false” is particularly useful. If a retained source does not settle the definition of a fee refund, the absence should remain visible. A retrieval engine returning no matching paragraph does not establish that the exception excludes refunds. It may have searched the wrong document, missed a synonym or lacked the referenced source. The evidence report should distinguish unsuccessful retrieval from reviewed absence within a specified source set.

Source spans also require stable identity. Character offsets depend on the extraction version, while page numbers may not survive a changed layout. Retain raw-byte identity, derivative identity, exact span text or hash, and human-readable location. A reference to paragraph 10 without a source version can become misleading after an amendment. A matching text hash without its source context may identify a sentence copied into a secondary summary.

The formalization boundary should distinguish source claims from supplied facts. A sentence defining a gift exception is part of the normative context. A bank record establishing that a fee was paid is factual evidence. A reviewer’s classification relating the record to the exception is an interpretive application. Combining all three into an undifferentiated “evidence” list makes it harder to see where disagreement actually arises.

Recent faithfulness work analyzes how language models and formal solvers can diverge when implicit assumptions enter formalization (Wang et al., 2026). The retained paper also has annotation and denominator questions that prevent us from transferring a general accuracy figure. Its examples nevertheless reinforce a practical requirement: an implicit assumption that changes the answer should become an explicit, challengeable premise. The system must show where a model supplied information that the source packet did not.

4.14 Testing paths, dates and inconsistent constraints

Other methods contribute different checks once a formal model exists. CUTECat uses concolic execution to explore paths in Catala programs (Goutagny et al., 2025, secs. 2–5). Concolic execution combines a concrete run with symbolic constraints describing the path taken. By changing a path condition, the tool seeks an input that exercises another branch. For our gift example, this is analogous to moving from a named-product case to the case where only the product-type route can determine the answer.

Path-directed generation can expose an exception branch that ordinary examples never visit. It cannot visit a branch that the formal program omitted entirely. Independent source-derived cases remain necessary. The reported costs on larger legal examples also show why a prototype needs explicit resource limits; small-program speed should

not be treated as a promise that every circular can be exhaustively explored.

Date arithmetic is another place where a precise program can embody a wrong choice. Adding a month to a date near the end of a month may require a rounding convention. The choice can affect deadlines. [Monat et al. \(2024\)](#) formalize date arithmetic and analyze ambiguity in legal applications. The transfer to this project is to make calendar rules, boundary conventions and their source basis explicit. A verified date library does not decide which rounding convention a particular provision requires.

ContractCheck analyzes consistency and executability of structured contract models ([Khoja et al., 2025](#), secs. 3–8). A satisfying assignment shows that a modeled set of conditions can jointly hold. An unsatisfiable set can help locate conflicting clauses or assumptions. Neither result proves that the original legal document has been translated faithfully. The study’s manual translation is an important part of its task boundary.

An unsatisfiable core is a useful diagnostic: a subset of constraints that already suffices for inconsistency. It is not necessarily the smallest such subset unless a minimization procedure establishes that property. For a reviewer, a small and source-linked explanation is valuable, but the product should label a returned core accurately. It must also distinguish an encoding inconsistency from legitimate defeasible conflict that the chosen logic was intended to resolve.

These analyses fit into the ensemble as specialized questions. Does this representation have a hidden path? Does a date convention alter a boundary case? Are the supplied assumptions jointly possible? Does a revised rule preserve the reviewed behavior? Their evidence can challenge a candidate without claiming to replace the source interpretation process.

4.15 A synthesis for the first implementation

The immediate product should keep a small executable core whose semantics are fully specified. Around it, the interpretation record should be richer than the executable language: it needs alternative readings, source versions, definitions, unresolved qualitative judgments and supporting arguments. Controlled templates give reviewers a readable view of formal structure. Specialized analyses can then inspect declared fragments through adapters.

This choice avoids forcing every legal question into the first compiler. A paragraph requiring judgment can become a reviewed assessment input or manual procedure, with explicit evidence and responsibility. A temporal duty can use an event profile whose limits are stated. A disputed definition can remain in the interpretation set until review resolves it. The system can still generate Java for a reviewed subset while keeping the broader assessment visibly incomplete.

Every adapter needs a semantic contract. It declares supported constructs, result categories, unknown and conflict behavior, vocabulary mapping, resource bounds and how a returned witness is replayed. A paper’s name or a repository’s existence does not supply that contract. A failed or unsupported analysis is reported as such and cannot be

counted as an additional agreeing ensemble member.

The literature thus supplies complementary mechanisms: source-linked authoring, precise exceptions, contextual alternatives, normative state, symbolic consequence, path-directed testing and translation repair. The new contribution proposed here is the controller that combines these mechanisms without allowing one successful check to conceal another unresolved question. Before specifying that controller, we need to explain how competing interpretations are generated and defended. That is the task of the next chapter.

Chapter 5

Constructing and comparing competing interpretations

5.1 The object of search is a legal choice

Suppose three models explain paragraph 10 in different words. The first says that product-linked gifts are prohibited unless they are fee discounts. The second calls the same arrangement a marketing incentive and repeats the exception. The third turns the sentence into an identical decision table. We have three presentations but possibly only one interpretation. Their agreement tells us little about the alternative that a mixed offer should be assessed component by component, or about the evidence required to classify a cashback payment. An ensemble that rewards the number of agreeing explanations may therefore become more confident without becoming more informative.

A useful hypothesis changes a consequential choice. It might change the unit of assessment from a whole offer to each benefit, distinguish reimbursement of a paid charge from an unrelated reward, or require an incorporated definition before applying a term. The hypothesis must also explain why that choice is supported. Merely deleting the words about a product type creates a different program, but an unsupported deletion is a test mutation, not an equally respectable interpretation. The system needs both kinds of variation and must label their roles accurately.

We will use a candidate record with five substantive parts: a proposition about the source; the assumptions under which it holds; a formal rule that expresses its consequences; supporting and opposing reasons; and a description of cases that distinguish it from another candidate. The candidate also carries source versions and an immutable identifier. This makes a claim such as “this cashback is a discount” answerable: a reviewer can inspect the proposed definition, identify the facts it requires, and observe the transactions on which a competing definition would disagree.

The search objective follows from that representation. We seek a useful set of supported alternatives, together with reasons for rejecting other proposals and questions that remain unanswered. Finding a single high-scoring sentence is a different objective. The former fits the bank’s need to know where judgment is required; the latter can conceal the judgment in an optimization procedure.

5.2 Legal theories are more than predicted outcomes

Theories of legal reasoning provide a richer starting point than generic text search. Bench-Capon and Sartor (2003, sections 3–5) represent a theory through cases, legally relevant factors, rules, preferences between rules, and values that explain the preferences. A factor is a legally significant characteristic, not simply a word that occurs in a judgment. A rule relates such characteristics to a conclusion. A preference explains why one applicable rule should prevail over another when both cannot determine the outcome together.

To understand the distinction, consider an invented training example about a reimbursement policy. One decision allows reimbursement because the expense was necessary for an authorized task. Another refuses it because the claimant had already been reimbursed. A theory containing only the first rule predicts payment whenever necessity is established. A theory containing both rules, but no priority, encounters a conflict in the double-payment case. A theory in which the bar on double payment defeats the necessity rule explains both outcomes. This is a logical illustration, not a statement about an actual court or SFC policy.

The theory's values might explain a preference by reference to preventing double recovery. That explanation is itself contestable: it must be connected to applicable authority rather than invented as the bank's preferred commercial outcome. The theoretical machinery helps represent such an explanation; it does not decide which values an institution is legally entitled to prioritize.

For the circular, the analogous factors include a product link, a type-of-product link, the character of the benefit, and evidence of a charge reduction. A theory might treat an incentive's relationship to the subscription charge as decisive; another might require that the charge itself is reduced in the contractual fee schedule. Their predictions may coincide for a direct fee waiver and diverge for cash paid after settlement. The divergence identifies an interpretive question that a bare outcome label would obscure.

The source paper does not claim to recover complete judicial reasoning from raw opinions. Section 5.4 explicitly discusses limits of the representation. Our implementation must preserve that limit: converting a judgment to factors is another interpretive task, with evidence spans, assumptions and review. A model that extracts a convenient factor list and then proves a conclusion from it has not bypassed the original source problem.

5.3 What case-based argument contributes

The history surveyed by Bench-Capon (2017, sections 2–3) is particularly relevant to the user's concern about a single reading. HYPO organizes exchanges in which one side cites a favorable case, an opponent distinguishes it or presents a counterexample, and the first side replies. This exchange encourages a search for facts that make an apparent analogy fail. The purpose is not to collect the largest number of vaguely similar

documents.

Imagine that an earlier, applicable authority addresses a benefit described as a fee rebate. A candidate relying on it must state the shared features: perhaps a charge was first incurred, the later payment could not exceed that charge, and both arose from the same transaction. An opposing candidate may distinguish an offer in which the payment exceeds any charge or can be earned without incurring one. These are questions about material similarities. A search engine's textual similarity score does not answer them.

HYPPO's claims lattice orders relationships between cases using legally relevant dimensions. A lattice is a partial ordering: some comparisons remain incomparable. Two precedents can each support a different aspect of the present case, without either being globally "more similar." Preserving this structure is useful for an interpretation workbench. A numerical embedding can nominate cases for inspection, while the argument record explains which feature makes a case relevant and which feature might defeat the analogy.

CABARET, as described in the retrospective, combines statutory rules with case-based reasoning about difficult terms. BankXX searches a network of pieces of legal argument. These suggest a division of labor for the circular: apply explicit logical structure directly, and investigate open-textured classifications through sources and arguments. They do not justify allowing analogy to override an unambiguous express restriction without an applicable legal reason. The original CABARET and BankXX implementations have not been audited for this monograph; the historical claims here come from the inspected retrospective.

A production case record therefore needs more than a citation and a summary. Record the issuing body, jurisdiction, date, procedural context, proposition relied on, relevant factual features, passage that supports the proposition, and known treatment by later authority. Distinguish a court's holding from an advocate's submission or an incidental observation. Those classifications must remain reviewable. An invented or misclassified precedent is more dangerous when attached to a polished formal argument than when presented as an uncertain search result.

This book supplies no actual case-law ruling on the cashback example. Its role is to specify how such authority would enter the system if found. The immediate 23EC46 exercise remains tied to the circular and its unresolved incorporated materials. A reviewer should never have to infer that a hypothetical analogy is a reported judicial decision.

5.4 AGATHA: explicit construction of rival theories

AGATHA supplies a direct precedent for searching over competing legal theories. Chorley and Bench-Capon (2005, sections 2–4) define moves that extend or challenge a developing theory: analogy, distinction, counterexample and changes to preferences. Rather than asking a model to "think harder," the system asks what legal move would change the current account and why. This makes the search history inspectable.

In our running example, an analogy constructor could propose that two payments belong to the same classification because both reverse a charge. A distinction constructor could split them by whether a charge was actually paid. An exception constructor could attach the fee-discount qualification at a particular point in the rule. A counterexample constructor could search for a scenario that the candidate treats as permitted despite an express product-type link. Each move should identify its parent, the changed proposition and the source support.

AGATHA evaluates theories using explanatory power, simplicity and completeness. Its search is a modified A* procedure, and the paper explains why the modification can miss the best theory. We should not import the word A* as an optimality certificate. The evaluated cases already have factor descriptions. The reported exercise is therefore substantially easier than discovering the factors, resolving cross-references and determining current authority in raw banking materials.

The paper's adversarial lookahead is also instructive. A theory can look strong because a limited opponent fails to find the appropriate counterargument. Optimizing for a victory against that opponent can favor a weak justification. For a bank, the objective must be resistance to informed challenge, not persuasion of a convenient critic. The system should preserve the strongest located objection even when an advocate agent prefers to stop discussing it.

We borrow the explicit construction operators and the record of competing reasons. We do not adopt the paper's heuristic weights as calibrated legal confidence, nor its advocacy objective as the bank's release criterion. A successful local adaptation must be evaluated on whether it finds material alternatives and omissions within a declared source perimeter. That evaluation has not yet been performed.

5.5 Arguments, attacks and unresolved conclusions

An argument is a chain from premises to a conclusion. Some steps are strict: under the stipulated meaning of conjunction, two true components make the conjunction true. Other steps are defeasible: evidence that a payment reverses a charge may support a classification while remaining vulnerable to a contrary source or missing condition. ASPIC+ separates these kinds of inference and formalizes ways to challenge them (Prakken, 2010, sections 2–3).

There are three useful attacks. An attack on a premise disputes a starting fact, such as whether the charge was paid. A rebuttal supports a contrary conclusion, such as classifying the payment as an additional benefit. An undercut attacks the inference itself: even if the payment and charge have equal amounts, equality of amount alone may not establish that one legally discounts the other. These attacks require different remedies. Retrieving a payment record may settle the first; retrieving a definition may address the third. Repeating the same classification prompt need not address either.

Let an argument graph contain arguments a and b that defeat each other, with no stronger argument resolving the dispute. Under the usual preferred-extension seman-

tics, the graph has two alternative acceptable sets, $\{a\}$ and $\{b\}$. Neither argument appears in both sets. Under grounded semantics, neither is accepted in this simple graph. The result expresses unresolved competition; it does not say both conclusions are legally true.

An extension is a set of arguments satisfying a specified acceptability condition. A conclusion is credulously accepted when at least one extension supports it, and skeptically accepted when every relevant extension supports it. A conclusion may be supported by different arguments in different extensions; requiring one identical argument to appear everywhere would be a stronger and different condition. The selected semantics and treatment of premises must be stored alongside the result.

There is an implementation trap at this point. If a semantics yields no extensions, code that applies a universal predicate to an empty list may report that every proposition passes. The inspected Carneades Dung implementation illustrates why callers must handle the existence of extensions explicitly. The workbench will require a nonempty computed collection before reporting skeptical support. If no extension exists, or enumeration is incomplete, the result is a separate unresolved status. It cannot become a positive conclusion through vacuous truth.

Argumentation computes consequences of the supplied arguments, attacks and preferences. It cannot establish that all relevant arguments were supplied or that the legal preferences are justified. Moreover, ASPIC+ rationality results depend on conditions on rules and knowledge bases; the author's correction to definition 6.8 must accompany any implementation relying on those results. A free-form graph of model-generated sentences does not automatically inherit the paper's theorems.

5.6 Burdens, assumptions and evidence in Carneades

Carneades distinguishes ordinary premises, assumptions and exceptions (Gordon et al., 2007, sections 3–6). An ordinary premise needs support. An assumption may stand in a particular dialogue until challenged. An exception can defeat an otherwise supported argument. These distinctions explain why absence of evidence has different effects in different reasoning arrangements.

For example, the workbench might require affirmative evidence that a transaction falls within the bank's activity profile. It must not silently assume that fact because most transactions in a training set did. Conversely, a procedural review may temporarily accept an explicitly declared factual assumption to compare the consequences of two legal readings. That temporary acceptance is a property of the review exercise, not a permission to use the assumed fact in production.

A burden of proof specifies what support is needed for a proposition in a given context. The 2007 paper includes illustrative proof standards and explicitly notes that they have not been validated as legal proof standards. It would be wrong to rename one of its numerical or dialectical conditions "SFC approval" and install it as policy. The bank needs its own reviewed rules about which questions require legal adjudication, what

factual evidence is sufficient, and which roles can accept residual operational risk.

The original model is acyclic. Newer Carneades software has a different computational model and cannot be treated as an executable copy of every result in the paper. Our source inspection of the newer implementation found useful argument utilities but also potentially expensive extension enumeration. A prototype should first support small, bounded graphs with explicit semantics and known reference examples. Introducing a general argumentation engine before its resource and semantic behavior are understood would create another opaque component in the assurance chain.

For the first release, even a simple structured argument register is valuable. Require each candidate to name its supporting premises, distinguish assumptions from evidence, identify opponents and specify what would defeat it. This can be implemented before automatic extension computation. The eventual engine then formalizes an already visible review practice rather than replacing an undefined practice with a numerical score.

5.7 Breadth before expensive investigation

The initial search should cover named dimensions: actor and activity scope, subject identity, definitions, logical attachment of exceptions, modality, quantification, time and incorporated context. These dimensions are prompts for possible differences, not a Cartesian product of equally plausible answers. If a choice has no source support, retain it as unsupported or as a negative control; do not inflate the shortlist with implausible alternatives.

Consider three dimensions with two alternatives each. Exhaustive combination would generate eight candidates, but some combinations may be inconsistent. A whole-offer interpretation combined with a component-only factual schema may not be executable without a transformation. A candidate that uses a definition from one historical version and an exception from an incompatible later version may not describe any applicable regime. Rejecting these combinations requires an explicit reason, because an encoding defect could otherwise be mistaken for a refutation of the underlying reading.

A practical initial frontier reserves room for each material family before ranking refinements within a family. The reservation is a search policy, not a claim that all families have equal legal merit. If the budget permits only six candidates and eight supported families are identified, the report must say that two families remain unexplored. It should not display a six-candidate shortlist as exhaustive. Increasing a beam width can improve exploration, but no finite width establishes semantic completeness for unrestricted legal language.

The search tree records how candidates were created. The argument graph records shared dependencies. These are different structures. Two branches may rely on the same disputed definition, and a later clarification should affect both. Representing all dependencies only as parent-child edges would duplicate facts and make invalidation unreliable. A directed acyclic graph of immutable candidate revisions can coexist with

a possibly cyclic argument graph, provided each structure has its own semantics and resource limits.

Deduplication also needs layers. Exact duplicate bytes can be merged by hash. Canonical syntax can identify expressions that differ only in irrelevant formatting. Formal equivalence can establish equal decisions over a declared domain. None of these establishes that source assumptions are identical. Two candidates with equivalent Boolean bodies but different definitions of “gift” remain separate interpretive commitments. Their shared body may be stored once, while their source and factual bindings remain distinct.

5.8 Tree search and the meaning of a reward

Tree of Thoughts separates candidate generation, evaluation and search (Yao et al., 2023, section 3). Its inspected breadth-first implementation retains a beam selected by model-based values or votes. That is bounded search, not exhaustive breadth-first enumeration. String deduplication in the code does not ensure that the retained thoughts express different legal hypotheses.

Language Agent Tree Search combines tree search, model judgments, reflection and environment feedback (Zhou et al., 2024, sections 3.2–4 and appendix A). A commonly used selection expression illustrates the tradeoff. For a parent visited N times and child i visited $n_i > 0$ times, let $\bar{r}_i$ be its observed mean reward. A UCT-style selector evaluates

$$U_i = \bar{r}_i + c \sqrt{\frac{\log N}{n_i}}, \quad c > 0. \quad (5.1)$$

The first term favors a child that has produced useful outcomes. The second encourages investigation of less-visited children. A separate rule handles unvisited children; setting their denominator to zero is not an implementation. The constant c , reward definition and visit accounting are material design choices and must be recorded.

For a game, the reward can be a terminal win. For a software task, a reward may include tests. For interpretation, neither model agreement nor passing a candidate’s own tests is an external truth signal. A useful action reward could instead record a verified dependency retrieved, a replayable disagreement scenario found, or a previously open question resolved by authenticated evidence. Such a reward measures progress in an investigation. It must not be displayed as the probability that the resulting legal reading is correct.

The distinction is especially important when reflection uses the model’s own explanation of a failure. A model can learn to satisfy a flawed test or to agree with its earlier answer. The resolution controller must keep the authoritative question and evidence fixed while allowing the candidate to change. The oracle cannot be rewritten merely to improve the candidate’s reward.

For the first prototype, a deterministic scheduler with family coverage and explicit ac-

tion priorities is easier to validate than a learned MCTS policy. MCTS can later be an optional scheduler under the same action, budget and release constraints. A comparison would ask whether it finds more independently judged material issues at equal cost. It would not ask whether it accumulates a larger self-assigned score. Chapter 9 specifies the needed experiment.

5.9 Choose questions that separate the surviving readings

Suppose four supported candidates remain. Three agree on a direct fee waiver, but they differ on a payment larger than the fee, a payment made without any fee, and a mixed voucher-and-waiver offer. Rechecking the direct waiver provides little discrimination. The system should identify the question whose answer is most likely to separate the remaining positions.

Generalized binary search formalizes a related problem for a finite hypothesis class and an answer oracle (Nowak, 2011, Figure 1 and sections I–II). Its identification guarantees have assumptions about the hypothesis space, queries and oracle. Legal interpretation may violate all three: a defensible reading may not yet be generated, a question may not admit a definitive answer, and the reviewer can be uncertain. We borrow question selection without claiming those guarantees.

A transparent heuristic uses pairwise disagreement. Let H be the retained candidates, $w_h \geq 0$ descriptive priority weights, and $o_h(x)$ the candidate’s predicted outcome for a proposed factual scenario x . Define

$$D(x) = \sum_{\{h,j\} \subseteq H} w_h w_j \mathbf{1}\{o_h(x) \neq o_j(x)\}. \quad (5.2)$$

Each unordered pair contributes when the scenario makes its predictions differ. With equal weights, this simply counts separated pairs. Dividing by the expected cost of answering the question can help scheduling, but the cost estimate is itself uncertain. Neither version converts the weights into posterior probabilities.

For two candidates, the useful scenario is any feasible one on which they differ. An SMT solver can sometimes construct such a scenario, subject to the domain conditions explained in Chapter 3. For classification disagreements, the solver may expose a variable assignment but cannot manufacture evidence that makes it legally meaningful. A witness with a “discount” flag set to true is not an answer to the question of what establishes that flag.

The review question should therefore contain the facts, the rival conclusions, the source passage responsible for the disagreement, and the additional evidence or authority sought. “Which answer is right?” is usually too vague. “Does the incorporated definition require a reduction in the contractual charge, or can a later reimbursement qualify when it is capped at the paid charge?” identifies the missing premise. If the available authority does not resolve it, that is a substantive result that the report should preserve.

5.10 Uncertainty about meaning is not a vote count

Self-consistency samples multiple reasoning paths and aggregates their answers (Wang et al., 2023, section 2). Its evaluations show benefits on the studied reasoning benchmarks. The method deliberately distinguishes diversity of paths from a single greedy trajectory, but the samples still share a model and prompting context. Forty sampled answers are not forty independent legal experts. The paper does not establish omission detection or calibrated legal confidence for SFC circulars.

Semantic uncertainty goes further by grouping answers by meaning before estimating uncertainty (Kuhn et al., 2023, sections 3–4). This is helpful because ten paraphrases of one answer should not look like ten separate answers. In the idealized formulation, strings expressing the same meaning belong to a class c , and the probability of the class is the sum of the probabilities of its strings. The class entropy is then

$$H(C) = - \sum_c p(c) \log p(c). \quad (5.3)$$

A concentration on one class produces lower entropy than an even distribution across several classes. This measures uncertainty of the model’s answer distribution under the selected grouping. It is not a proof that the concentrated answer is correct.

The paper approximates semantic grouping with natural-language inference. Its appendix discusses nontransitivity and sensitivity to clustering order. Our inspection of the official source identified a further implementation issue: the examined clustering condition tests the absence of label zero in either direction, rather than explicitly requiring entailment in both directions. The retrieved model configuration maps label zero to contradiction, one to neutral and two to entailment: the inspected code therefore also permits neutral relations. The model configuration was retrieved separately from the pinned code; its identity must be pinned as well before reuse. Even a perfect implementation of the stated mutual-entailment test would remain an empirical language-model judgment, not formal equivalence of bank controls.

The workbench can use semantic similarity to propose clusters for review. It must not delete a dissenting candidate merely because an NLI model assigns a high score. Formal equivalence can justify merging execution paths only within its checked domain, while source assumptions and classification definitions stay visible. Chapter 9 develops calibration and set-valued reporting without assuming that a model’s internal distribution is a distribution over legal truth.

5.11 What can be reused from the adjacent projects

DynareMCP contains useful patterns for explicit hypotheses, diagnostics, falsifiers and limits of claims. In the inspected legacy audit writer, `write_hypothesis_node` records a hypothesis, source support, code support, a diagnostic, a falsifier and a supplied rank score. These fields map naturally to interpretation candidates. The supplied score is

not automatically calibrated; its meaning depends on the caller.

The inspected legacy scheduler chooses among eligible frontier nodes by a stored priority and identifier. It is not a general UCT engine hidden behind the word “tree.” Reusing its bookkeeping ideas is sensible; importing it and claiming that a proven Monte Carlo search now verifies legal meaning would be wrong. The new controller needs its own issue identity, source lineage, legal review states and resource accounting.

MathDevMCP is useful once a proposition is precise enough to prove. A formal obligation might concern preservation of a Boolean expression, absence of an unbounded loop in a finite controller model, or equivalence of two encodings over a declared domain. ResearchAssistant supports retrieval, parsing and retention of the literature and regulatory evidence. Neither project supplies a privileged interpretation of English. They strengthen different links in the chain.

The design that follows therefore combines several structures: a source inventory to detect missing material, a candidate set to expose choices, an argument graph to record reasons, a bounded search to investigate questions, and formal checks to compare executable consequences. The diversity is useful because each structure can reveal mistakes that the others overlook. Its value will depend on how discrepancies move between them, which is the central subject of the next chapter.

Chapter 6

An accountable ensemble with bounded resolution

6.1 The design objective and its limits

The proposed system must make a dangerous path difficult: a plausible reading is accepted, translated correctly, and released without anyone discovering that it omits a material qualification. Multiple generators alone do not close that path. They may share a source extraction, a vocabulary, a training distribution and the same mistaken assumption. The design therefore uses redundancy across different activities, not just repeated answers to one question.

One activity inventories the source and its dependencies. Another identifies normative statements. Several candidate constructors express different readings. An opponent seeks a missing condition or counterexample. A formal evaluator checks consequences. A separate reference process supplies expected decisions for reviewed scenarios. A Java verifier checks preservation. Each activity has a defined responsibility and a defined way to fail. A failure or discrepancy is recorded even when every other member appears successful.

The main output is an interpretation dossier. It contains the proposed control, the source it covers, the readings considered, the reasons for their dispositions, the tests and formal checks, and the uncertainty that remains. The controller may prepare this dossier for review. It does not acquire authority to approve legal meaning merely because it has completed its work.

There are three success conditions at different levels. An engineering run succeeds when it follows the specified state machine and produces a complete report within its limits. An interpretation investigation succeeds when it resolves its declared questions with adequate evidence or precisely identifies what requires judgment. A release succeeds only after the required people approve the exact interpretation and executable package. A run can satisfy the first condition while ending with an unresolved legal question. That is a valid and useful outcome, not an excuse to fabricate certainty.

No finite ensemble can establish that it has found every possible reading of unrestricted language. The operational objective is more precise: cover a reviewed source perimeter; require explicit treatment of every inventoried unit; search a declared set of interpretive dimensions; investigate every detected discrepancy under finite limits; and prevent a material unresolved question from being silently converted into release. These obligations can be implemented and tested. Their empirical effectiveness at discovering

unknown omissions requires the evaluation in Chapter 9.

6.2 Why diversity has to be designed

Repeated sampling can reduce some random mistakes while leaving a shared mistake untouched. A simple probability model makes the distinction clear. Let q be the probability of a common failure that makes every member wrong. Conditional on that failure not occurring, assume for this illustration that each of m members has independent error probability p . The probability that all members are wrong is then

$$\Pr(\text{all wrong}) = q + (1 - q)p^m. \quad (6.1)$$

The two events are disjoint: either the common failure occurs, or it does not and all residual errors occur together. This derivation is an illustrative model, not an estimated error law for language models or lawyers.

If $q = 0.02$, $p = 0.1$ and $m = 3$, the expression gives 0.02098. Treating the three members as unconditionally independent would instead give 0.001. Adding members reduces the residual term in this example, but the common failure remains. A missing footnote in the shared extraction is a concrete way that the common failure could arise. Three careful interpretations of the same incomplete text may agree perfectly.

A second calculation concerns the variability of a mean of member indicators. Suppose X_i records whether member i makes a particular error, all have variance σ^2 , and every pair has correlation ρ . Expanding the variance of their mean gives

$$\begin{aligned} \text{Var}\left(\frac{1}{m} \sum_{i=1}^m X_i\right) &= \frac{1}{m^2} (m\sigma^2 + m(m-1)\rho\sigma^2) \\ &= \frac{\sigma^2}{m} \{1 + (m-1)\rho\}. \end{aligned} \quad (6.2)$$

The expression explains why counting correlated members exaggerates the amount of independent information. It does not identify their correlation in practice, and it does not supply a legal error probability. Its role is to motivate measurement of shared failure and to discourage confidence claims based on panel size alone.

Diversity should therefore be recorded along several axes. Members may use different model families, prompts, representations, source acquisition routes, retrieval methods, factual schemas or verification methods. These differences are opportunities for complementary errors; they do not prove independence. Two vendors can share training material. Two parsers can both omit an image. Two human reviewers can both receive a leading case summary. The record should describe what differs and what remains shared.

The strongest inexpensive separation is often procedural. Let source inventory and initial interpretation happen before either sees the other's preferred conclusion. Freeze the initial outputs. Only then expose disagreements for investigation. This preserves evi-

dence that one member independently noticed a condition. If all members see the first answer before responding, later agreement may merely reflect anchoring. Deliberation can still be useful, but it must not rewrite the history of independent proposals.

6.3 The ensemble members and their responsibilities

The prototype uses roles rather than assuming that every role is a language model. The source custodian acquires the official document, retains the exact bytes and metadata, and records incorporated materials. A document inventory process enumerates paragraphs, footnotes, tables, attachments and visible cross-references. A second inventory method compares the enumeration with the rendered document. Both methods may be automated initially, but unexplained mismatches remain review questions.

The normative reader identifies who must or may do what, under which conditions, with which exceptions and dates. A controlled-language reader expresses the same material using a restricted template. Their outputs are compared at the level of actor, activity, modality, object, condition, exception, time and dependency. This comparison can reveal a missing phrase even when the two prose summaries sound similar.

A counterinterpretation reader is required to propose consequential alternatives where the source supports them, and to say when it found none. It is not required to invent a controversy for every sentence. Its task includes seeking an omitted qualification, an alternative attachment of an exception, or a classification that changes a boundary case. Unsupported alternatives are preserved as such, which helps distinguish deliberate testing from legal disagreement.

The formal checker validates types, references and explicit semantics, compares candidate rules over supported domains, and replays witnesses. It has no permission to promote a source assumption to a fact. The scenario designer constructs examples that separate readings and includes unknown, conflicting and out-of-scope evidence. Its expected outcomes come from an independent review process or remain labelled provisional.

The implementation checker compares the accepted rule with the Java package and verifies the package's exact identity. Finally, the dossier assembler reconciles all required outputs and creates the review report. It may summarize, but it may not suppress a minority objection, change its severity or mark it resolved without a linked resolution record.

A role can fail in several ways: unavailable, malformed output, unsupported claim, resource limit, incompatible source version or abstention. Abstention is an informative response when a role lacks the evidence needed for its question. It is not a vote for the majority. If a role is mandatory for the configured coverage claim, its missing output prevents that claim from being marked complete. A replacement role must be explicitly authorized by the configuration and its independence limits recorded.

Table 6.1: Responsibilities that create different opportunities to detect error.

Role	Question answered	Failure that must remain visible
Source inventory	What material is present and incorporated?	Missing attachment, unmatched paragraph, inaccessible dependency
Normative reader	What obligations, conditions and exceptions are expressed?	Omitted modality, scope or qualification
Alternative reader	What supported choice changes a consequence?	No supported alternative found; unexplored family
Argument critic	What defeats the premises or inference?	Unanswered objection or unverified authority
Formal checker	What follows from this specified rule?	Invalid encoding, unknown solver result, limited domain
Reference reviewer	What is expected for this scenario, and why?	Disagreement, incomplete facts, shared authorship
Java checker	Does this package preserve the accepted specification?	Mismatch, unsupported operation, stale build
Dossier assembler	Are all required results and dispositions present?	Missing member, open issue, broken evidence link

6.4 Build a source perimeter before interpreting it

The source perimeter is the set of documents and versions the investigation claims to cover. It begins with the named circular, but incorporated definitions, annexes and transition provisions may expand it. Each dependency has a reason: explicit incorporation, definition of a term, amendment, background explanation, or a proposed analogy. These relationships must not all be collapsed into a generic “relevant document” link.

For 23EC46, the retained inventory has eighteen paragraphs and four footnotes. The executable slice covers paragraph 10. That is a legitimate prototype scope only if the report makes the remaining material visible and explains whether it is contextual, another obligation, a dependency, or outside the selected slice. Labelling all unimplemented paragraphs “not applicable” merely because the prototype lacks code would be wrong. Implementation capability is not a legal applicability test.

The inventory assigns each source unit a stable locator within a particular source hash. It records the extracted text, its position, and whether the unit contains a possible normative statement or reference. The second inventory can use a different parser or rendered-page inspection. A mismatch creates a source coverage discrepancy before any interpretation ranking takes place.

The controller expands dependencies until each required edge reaches a retained source, a justified out-of-scope disposition, or an unresolved dependency record. Cycles are legal as document relationships: a code and a guidance note can refer to each other. The traversal uses visited version identifiers to avoid infinite retrieval, while preserving both edges. A retrieval depth limit stops processing but leaves the unexpanded edge visible. It does not prove that the dependency is unimportant.

Publication date, effective date and retrieval date occupy separate fields. A circular published today may describe a later transition or an earlier arrangement. The source custodian cannot infer commencement solely from the HTTP response time or the first date in the text. The retained source packet is a historical evidence object. A current applicability assessment requires a separate search for amendments and supersession, with its own cut-off date.

This inventory is an important defense against common omissions, but it is not an oracle. An extractor might enumerate a footnote without understanding its legal effect. The requirement that each unit receives a reasoned disposition connects inventory to interpretation. A coverage check should fail if a unit is present but has no explanation, or if an explanation claims executable coverage without identifying the rule and verification evidence that provide it.

6.5 Freeze blind proposals before reconciliation

An initial proposal request contains the frozen source packet, the selected activity profile, the vocabulary already approved for that profile, and the member's task. It excludes other members' proposed conclusions. The response is stored with model or tool identity, configuration, prompt-template version, source hashes and completion status. Raw output is retained subject to the bank's access and retention policy; the structured candidate is a validated derivative.

A candidate cannot refer to a source version it was not given without opening a new retrieval request. This rule prevents a model's recalled description of an unseen amendment from entering as if it were inspected evidence. Recalled material can nominate a search, but the supporting claim remains unverified until the source is acquired and checked.

The blind phase ends when every required member has responded or reached a terminal failure under the initial-phase budget. The controller records that boundary. It then compares the candidates and inventories. Later repairs may share all relevant evidence, because their purpose is informed resolution. The record must continue to distinguish the initial independent proposals from responses influenced by the shared discussion.

This separation prevents a misleading report such as "five independent members agreed" when four were shown the first member's answer. It also makes a useful experiment possible: measure which defects were caught before deliberation and which required shared investigation. A system that produces agreement by suppressing dissent should look different from one that resolves an objection with a missing official definition.

Candidate revisions are immutable. A revised rule points to its parent, states what changed, and links the evidence or diagnostic that motivated the change. The original remains available. Reviewer approvals attach to a particular candidate hash, not to a mutable display name such as "latest rule." If a source, assumption or rule changes, the old approval cannot silently authorize the new candidate.

6.6 Detect discrepancies at the right level

A discrepancy is a structured difference that may affect coverage, meaning, execution or evidence. It need not be a pair of contradictory prose answers. One member may omit an input that another requires; two members may agree on a rule but cite different versions; one parser may miss a footnote; the Java runtime may return the same decision with an incomplete audit trace. All deserve a recorded disposition appropriate to their consequence.

The comparator first checks identity: source versions, activity profile, subject unit, candidate dependencies and evaluation times. Comparing answers to different questions creates false disagreement. A model interpreting a whole offer and one interpreting a single benefit must be identified as differing in subject identity before their truth values are compared.

It then compares structured meaning: actor, action, object, modality, conditions, exceptions, quantifiers, temporal boundaries and references. Exact structural matches are cheap evidence of agreement about those fields, but source coverage still needs a separate check. Differences in presentation can be classified as nonmaterial only with an explicit rationale, such as identical canonical syntax or a domain-qualified equivalence result.

Formal outcome comparison comes next. A counterexample is especially useful when two apparently similar rules disagree only at a boundary. For paragraph 10, omitting product-type linkage is exposed by $P = F, T = T$, with the other conditions established. Omitting the discount exception is exposed by a genuine fee discount. Neither example resolves whether a particular cashback is a fee discount; that is a different discrepancy with a different evidence need.

The comparator also checks omissions against independent inventories. Every required source unit, dependency and ensemble role must have a disposition. An all-agreeing set of candidates can therefore still produce discrepancies. This is the intended behavior: agreement among candidates cannot erase an uncovered paragraph or a failed source acquisition.

A discrepancy record includes its origin, affected source units, candidate identifiers, issue family, factual or legal question, observed difference, materiality, evidence needed, permitted next actions and current status. It also stores the consequence of leaving it unresolved. A classification issue might block a particular incentive category; a corrupted source hash might invalidate the entire run. The scope of the block must be explicit and conservative when impact cannot yet be determined.

6.7 An issue needs an identity that survives retries

Naive retry systems often attach the attempt counter to a model response. When the response creates a revised candidate, a new counter starts. A difficult issue can then consume unlimited attempts by repeatedly changing its wording or creating descen-

dants. The proposed controller attaches budgets to the run and to an issue root that survives candidate revisions.

An issue root identifies the source context, the substantive question and the affected subject or rule family. It does not include a candidate hash as the only identity, because candidate hashes necessarily change during repair. The system assigns an immutable root identifier when the question is first recorded. Subsequent discrepancies can link to that root or create a child issue. A child inherits the root's budget account and the run's limits. Creating a new display record never creates new spending authority.

Automatic deduplication proposes that two issues have the same root when they concern the same source unit, semantic dimension and consequence. A reviewer or strict deterministic criterion must resolve uncertain merges. An erroneous merge could hide a second problem; an erroneous split could waste work. Global action limits protect against unlimited spending even when deduplication is imperfect, while the report shows possible duplicates for later review.

Issue identity also separates changed evidence from repeated reasoning. If an unavailable FAQ becomes available during the same run, the controller can attach the new version and continue within the remaining budget. If a later circular changes the applicable rule, that is a new source context and normally a new run. The old run remains closed with its original result. Opening the new run requires a recorded trigger and authorization; a worker cannot evade exhausted limits by automatically renaming the investigation.

A candidate that depends on several issues keeps all of their identifiers. A resolution of one issue cannot clear the others. The dependency relation also supports invalidation: retracting a definition affects every candidate that used it, even if those candidates were reached through different search branches. The service must query this relation directly rather than trying to reconstruct it from narrative summaries.

6.8 Statuses describe evidence, not optimism

The issue lifecycle begins at OPEN. An issued action moves it to INVESTIGATING. It may then return to OPEN with additional evidence, move to AWAITING_EVIDENCE, or require HUMAN_REQUIRED. These are unresolved states. They differ in the next useful action, not in whether the system may assume the preferred answer.

Resolution has three main forms. RESOLVED_EVIDENCE means that an inspected source or factual record establishes the missing premise under the accepted review rules. RESOLVED_ADJUDICATION means that an authorized reviewer has made and explained the interpretive decision. RESOLVED_EQUIVALENCE means that the apparently different executions have been shown equivalent over a specified domain and that no distinct source commitment remains at issue. These statuses require different evidence and cannot substitute for one another.

For example, an SMT equivalence result cannot resolve a dispute about whether the word “may” establishes a universal classification. Both candidates may have been en-

coded with the same unjustified classification flag. Conversely, an explicit source passage that defeats a missing-exception candidate need not wait for a probabilistic vote. The source-based resolution identifies the clause, updates the candidate and requires the affected formal and Java checks again.

An issue may become terminal while remaining unresolved. The terminal reason can be `BUDGET_EXHAUSTED`, `DEADLINE_REACHED`, `NO_PROGRESS`, `DEPENDENCY_UNAVAILABLE`, `CANCELLED`, or an integrity failure. Terminality means automatic work has stopped. It says nothing favorable about the candidate. The report must therefore store resolution status separately from processing status; a single Boolean `done` is inadequate.

Materiality is another separate field. An issue is material when its unresolved answer could change applicability, a required action, the decision, the timing, the explanation required for review, or the validity of the evidence chain. If impact is unknown, it is treated as material until assessed. Downgrading an issue requires a reason and authority appropriate to the configured review policy. A generator cannot label its own objection immaterial simply to unblock release.

A resolved issue can be reopened by new evidence, a source amendment, an invalid witness or a discovered dependency. Reopening creates a new event and invalidates results that depended on the old resolution. It does not erase the earlier record. If the original run is already closed, the successor investigation links to it and states which conclusion is being reconsidered.

6.9 Choose a remedy that can answer the question

Each discrepancy type has a permitted action set. A missing source calls for retrieval or escalation to the source custodian. A parsing disagreement calls for inspection of the original page, an alternative parser or manual transcription with provenance. A scope disagreement calls for analysis of the actor and activity language, incorporated definitions and the selected bank profile. A formal mismatch calls for a replayable witness and a correction to the relevant encoding or runtime component.

A useful action has a question, an expected evidence type, a bounded execution method and a success condition. “Ask another model” is incomplete. “Ask the alternative reader to identify source-supported reasons why the exception might attach to the whole offer, using only the retained paragraph and definition” is a bounded interpretive task. Its result is still a proposal, but the task makes clear what new information it seeks.

The controller should prefer actions with a plausible path to resolution. If the question is whether a fee was paid, querying transaction evidence may be more useful than sampling another legal explanation. If the question concerns the meaning of an incorporated term, more transaction data may be irrelevant. This matching of remedy to issue is where structured uncertainty becomes operationally valuable.

An action can also deliberately seek a falsifier. For a proposed rule, request a feasible scenario in which it contradicts an express source condition. For a classification, request

a fact pattern that the definition treats counterintuitively and ask which source justifies that boundary. For a claimed equivalence, ask the solver for a difference rather than asking a model whether the programs look similar. Falsification attempts must retain their domain and assumptions.

Repeated actions are allowed only when their rationale changes or a bounded retry policy permits recovery from a technical failure. Retrying an unavailable server can make sense; asking the identical question with no new evidence is unlikely to resolve a legal ambiguity. The report should show which retries addressed transport failure and which explored a substantive alternative. Both consume resources, but they mean different things for the investigation.

A repair is validated against the original discrepancy and against the unaffected requirements. Fixing the type-of-product omission must not remove the fee-discount exception. A local witness becoming consistent is necessary evidence about that repair, but insufficient evidence about the whole rule. The controller runs the dependent validation set, records any new discrepancies, and keeps the old issue open until its resolution criterion is actually satisfied.

6.10 A resolution matrix prevents the wrong kind of closure

The controller needs a typed relationship between a discrepancy and the evidence that can close it. Without that relationship, every issue can eventually be “resolved” by another fluent explanation. The matrix below is a proposed engineering contract for the first implementation. It states the minimum kind of evidence, not a substitute for legal judgment about whether that evidence is persuasive in the particular case.

Table 6.2: Discrepancy-specific actions and closure evidence.

Discrepancy	Appropriate next action	What can justify closure
Missing source unit	Compare original rendering and independent inventory	Retained unit, corrected inventory and reviewed disposition; agreement among summaries is insufficient.
Unavailable dependency	Retrieve the named version or seek authorized clarification	Inspected applicable source, or an explicit adjudication of the remaining question; a guessed definition is insufficient.
Different source versions	Establish the assessment cut-off and amendment relationship	A documented version selection and reconsideration of affected candidates.

Discrepancy	Appropriate next action	What can justify closure
Actor or activity scope	Inspect addressee, operative language and profile facts	Reviewed applicability proposition with evidence for the actual bank activity.
Disputed classification	Identify definition and distinguishing factual evidence	An accepted definition and sufficient facts, or an attributable adjudication; a Boolean label alone is insufficient.
Exception attachment	Construct component and whole-arrangement examples	Source-grounded structural choice, with contrary reading disposition and regression cases.
Necessary versus sufficient condition	Compare implication directions on boundary scenarios	A reviewed direction of dependence and a rule that preserves other required conditions.
Timing or transition	Separate publication, commencement, valid time and knowledge time	Reviewed interval and transition rule with endpoint examples.
Factual conflict	Obtain correction or apply an approved evidence-priority rule	Attributable supersession or reviewed normalization; newest arrival alone is insufficient.
Formal mismatch	Produce and replay a feasible difference witness	Validated repair or domain-qualified equivalence, with source assumptions retained.
Unsupported operation	Inspect whether a supported encoding preserves meaning	Reviewed encoding plus conformance evidence, or an approved runtime extension; compiler rejection does not defeat the reading.
Java mismatch	Compare full semantic results and trace	Corrected exact package with repeated affected verification and new build identity.
Missing ensemble member	Retry under budget or use an authorized replacement	Required role output with documented independence limits; silence is not agreement.
Search truncation	Investigate retained frontier or narrow the claimed scope	Completed declared finite search or explicit reviewed restriction; a high best score is insufficient.
Broken approval lineage	Reconstruct exact source, report, bundle and build links	Fresh valid approvals for exact identities; a matching display title is insufficient.

Some issues need more than one row. A cashback classification can depend on an unavailable definition and an uncertain factual relationship to a fee. Retrieving the defini-

tion resolves the dependency but may leave the factual question open. The controller should represent these as linked issues so that progress is visible without overstating closure. A single combined issue can be acceptable if its resolution criterion explicitly requires both conditions.

The matrix also guides action rejection. If a proposed action cannot produce the needed evidence, the scheduler should normally decline it and explain why. A new model paraphrase is not an appropriate remedy for a corrupted source digest. A solver equivalence query is not an appropriate remedy for a missing commencement date. This does not prohibit exploratory work; it requires that exploratory work be labelled as such and charged to the investigation budget rather than counted as resolution.

Closure is evaluated against the issue's original question and current source context. Suppose a model responds to a component-level question with a correct whole-offer explanation. The response may be useful, but it has not answered the original question. The controller creates or links the relevant candidate and keeps the original issue open. This guards against a subtle form of apparent progress in which the task changes until the model can answer it confidently.

6.11 Resolution can reveal that the first question was wrong

A structured controller must still permit substantive reframing. The initial question may assume a distinction that the source does not make. For example, the team may ask whether a cashback is a discount when the relevant provision instead classifies the entire arrangement through another defined concept. The correct response is to record the changed understanding, identify which source supports it and replace the candidate with a new revision. The old question is dispositioned as superseded by a reasoned reframing, not silently deleted.

Reframing does not reset budgets. The successor issue links to the original root or consumes the same global account, and the report explains the relationship. If the reframing expands the source perimeter beyond the authorized run, the current investigation can end with a request for a successor run. This prevents a useful ability to change direction from becoming an unbounded search loophole.

A concrete example concerns subject identity. A provider initially treats an offer as one benefit. The independent inventory identifies a source passage that requires separate treatment of components. The controller creates a component model, revises factual requirements and reruns affected scenarios. It also records that prior results used a different subject abstraction. Merely changing the formula while leaving the whole-offer adapter untouched would not complete the repair.

Another example concerns modality. A candidate initially turns guidance about factors to consider into an unconditional prohibition. Investigation shows that the source requires an assessment rather than a predetermined outcome. The repair may need a workflow obligation, evidence of consideration and a review record, not another

Boolean threshold. If the current runtime cannot express the accepted requirement, the result is a supported interpretation with an implementation gap. That is more accurate than forcing the source into a familiar Boolean shape.

These cases explain why the issue register includes the affected decision and the kind of evidence required. The controller can recognize that the target has changed and invalidate dependent checks. A test suite for the previous target cannot automatically certify the new one, even if many examples happen to retain the same final status.

6.12 The investigation graph should preserve shared causes

The following diagram separates the objects that are often collapsed into a single chain of model messages. Several candidates can depend on one definition, and several discrepancies can affect one candidate. Actions investigate issues; results create evidence or revisions. The final report reconciles the graph rather than selecting the last message as the answer.

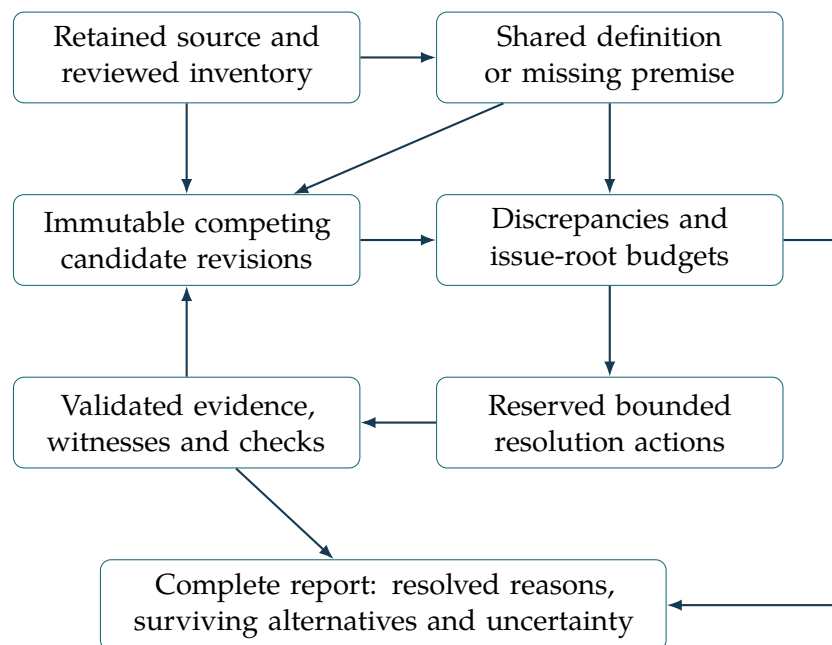


Figure 6.1: Search creates a history, while dependencies explain what must be reconsidered.

An important consequence is that one new source can resolve several issues without spending a separate retrieval action for each. The action is charged once, and each affected issue receives a validated link to the result. Their root budgets do not acquire extra credits. Conversely, one failed source acquisition can block several candidates, and the report should identify that shared cause rather than presenting a long list of apparently unrelated uncertainties.

The graph also prevents misleading independence claims. If four candidate members

all rely on the same extracted definition and factual mapping, the report can show that common dependency. Their reasoning paths may differ, but their evidence base is shared. This is a more useful account of redundancy than simply displaying four model logos beside an agreement percentage.

6.13 Finite budgets are part of the semantics

The user requires automatic processing up to a maximum number of times. To make that requirement implementable, we distinguish an initial proposal phase from resolution rounds. A resolution round consists of selecting currently open issues, reserving authorized actions, collecting bounded results and reconciling the resulting candidates. The round count increases when the controller commits the round's action plan, even if every external action later times out.

The configuration supplies a maximum number of resolution rounds, a maximum number of issued actions per issue root, a maximum number of issued actions for the run, and an absolute deadline. It also supplies per-action timeouts, candidate and graph-size limits, and any token or monetary limits. None is inferred from the fact that the existing drafting stub happens to use three attempts. That stub's retry behavior addresses a different contract.

A reference policy for deterministic fixtures uses three resolution rounds, three issued actions per issue root, and twelve external actions in total, including the initial proposal actions. It permits at most four initial external proposal actions, leaving at most eight for later investigation. These are engineering examples chosen to make boundary tests tractable. They are not empirically justified production defaults or recommendations about how much legal review is enough. Production configuration must be approved and evaluated for the selected task and cost constraints.

The reference policy also sets a ten-minute run deadline and a thirty-second external action timeout. Deterministic local checks have separate finite input and execution bounds. A mandatory check is never silently skipped when its bound is reached: it returns an incomplete result and blocks the corresponding claim. Token and monetary caps are nullable only when the provider integration cannot supply a reliable bound; such a configuration is labelled unsuitable for an unattended paid run. A paid deployment must supply conservative reservations.

Let B be the configured global action cap and a the number of actions already issued. Before dispatching any new external action, the controller must establish

$$a + 1 \leq B, \quad a_r + 1 \leq B_r, \quad t < t_{\text{deadline}}, \quad (6.3)$$

where a_r and B_r are the issued count and cap for the affected issue root. For an initial proposal action, a separate initial-phase account replaces the issue-root account. The guard also checks the round, token, cost and action-type constraints. Passing one limit does not override another.

Reservation occurs atomically before dispatch. The issued count increases even if the

worker crashes before learning whether the provider accepted the request. This conservative accounting prevents an ambiguous network result from becoming free work. A reservation can be cancelled without spending only before dispatch is durably committed and only through an atomic state transition that proves no worker can still send the action.

Provider SDK retries must be disabled or included in the same accounting. A single logical action that secretly makes five billable requests defeats the meaning of the cap. When a provider offers idempotency keys, the adapter uses the recorded action identifier. When it does not, the system cannot promise exactly once execution across a lost response; it reports the uncertainty, counts the attempt, and follows the configured retry policy.

6.14 No progress is a substantive stopping reason

A finite global budget guarantees that external actions cannot continue forever, but it need not be spent in full. A no-progress condition avoids cycling through identical proposals. The controller computes a progress signature from accepted source additions, newly supported assumptions, new feasible witnesses, issue resolutions and genuinely distinct candidate semantics. Cosmetic wording changes do not count as progress.

The signature is a deterministic summary of validated changes, not a model's claim that its answer improved. If a resolution round produces no such change, the controller records that fact. The reference policy stops after two successive rounds without progress, unless an already reserved action is still awaiting a result within the deadline. This is an engineering stopping rule for the fixture profile. A production policy may choose another threshold, but must disclose it and evaluate the cost of premature stopping.

New unsupported candidates are not automatically progress. Otherwise a generator could prevent termination by inventing fresh interpretations indefinitely. Similarly, splitting one issue into several differently worded issues does not reset the no-progress count or any budget. The controller retains those proposals for review when useful, but their existence alone does not establish a reason to continue automatic investigation.

A source becoming temporarily unavailable may justify a scheduled successor run, but not an endless loop in the current run. The report records the failed retrieval, the affected question and the conditions under which a new run would be useful. A human or an authorized source-change service can initiate that run with a new recorded budget. This preserves the distinction between bounded automation and an operational process that may legitimately continue over days.

No-progress termination is not a judgment that the research direction has failed. It says that the current actions, sources and budget have not resolved the question. A later phase designed to obtain independent authority may still be justified. The system must report whether it encountered an invalid source, broken implementation, insufficient evidence, or simply a candidate that failed its checks. Those outcomes call for different

next steps.

6.15 A controller that can be implemented and checked

The controller is a deterministic state machine around fallible external tools. Its durable state contains the run configuration, source snapshot, required role results, candidate revisions, issue roots, action records, budgets and report status. External results enter through validation. They do not directly mutate approvals or release state.

The following pseudocode specifies the main control flow. The functions named `commit` and `reserve` are transactions; they must either write all of their stated changes or none. The action worker uses the committed action record and cannot create additional actions on its own.

Listing 6.1: Main controller, with explicit terminal uncertainty.

```
start_run(source_packet, profile, required_policy):
    validate_source_identity_and_policy()
    commit(run = INITIALIZING, counters = 0)
    reserve_and_issue_initial_roles_with_global_budget()
    collect_until_complete_or_action_deadline()
    validate_outputs_and_freeze_initial_proposals()
    reconcile_inventory_candidates_and_required_roles()

    while has_unresolved_material_issue():
        if integrity_failure():
            close_as(FAILED_INTEGRITY); break
        if cancelled():
            close_as(CANCELLED); break
        if deadline_or_global_or_round_limit_reached():
            mark_remaining_issues_with_stop_reason()
            close_as(BLOCKED_UNRESOLVED); break
        if no_progress_limit_reached():
            mark_remaining_issues(NO_PROGRESS)
            close_as(BLOCKED_UNRESOLVED); break

        plan = choose_permitted_discriminating_actions()
        if plan.is_empty():
            mark_remaining_issues(HUMAN_REQUIRED)
            close_as(BLOCKED_UNRESOLVED); break
        commit_next_round_and_reserve_bounded_actions(plan)
        dispatch_only_reserved_actions()
        collect_results_until_each_action_is_terminal()
        validate_results_and_create_candidate_revisions()
        rerun_affected_checks_and_reconcile_all_open_issues()
        record_progress_signature()

    if not already_closed():
```

```

    check_required_roles_coverage_and_all_mandatory_checks()
    if any_requirement_incomplete():
        close_as(BLOCKED_UNRESOLVED)
    else:
        close_as(READY_FOR_REVIEW)
    write_complete_report_for_every_terminal_status()
    return report_reference

```

The final coverage check is essential. If no comparator detects a disagreement, the loop might never run. The controller must still require complete source coverage, mandatory members and verification results. “No open issue” and “all required evidence present” are separate conditions. A failed member must also create an issue, but the final check protects against a defect in that issue creation path.

Actions that finish after the run deadline are retained as late observations. They do not reopen a closed run or retroactively change its result. A successor run may use them after validation. This rule makes reports stable and prevents a late worker from changing the package that a reviewer has already inspected. Late evidence can still trigger a documented follow-up when it changes risk.

The controller must produce a report even when it fails. If the primary report writer is unavailable, the durable run and action records support recovery of a minimal failure report. An integrity failure prevents a positive result, but it must not erase the explanation of what was attempted. Report generation itself has bounded resource use and a recovery path; an exception thrown while rendering a large argument graph cannot make the investigation disappear.

6.16 Termination and the boundary of the proof

We can prove a limited but useful property of the proposed controller. Assume a finite input packet, a finite configured action cap B , atomic issue accounting, finite local reconciliation with explicit graph limits, and an enforced timeout for every dispatched action. Assume also that the storage and supervisor either respond within their operational bounds or move the run to a recoverable failure state. These assumptions concern implementation and infrastructure; the proof does not establish them merely by listing them.

Define the remaining action budget as $V = B - a$, where a is the durable number of issued actions. Initially V is a nonnegative integer. Each new dispatch requires an atomic reservation and reduces V by one. No transition increases V in the same run. Therefore there can be at most B dispatches. Between dispatches, reconciliation is finite by its input and execution bounds. Waiting is finite by the action timeout and absolute deadline. When no permitted action remains, the controller enters a terminal state. Under these assumptions, automatic processing terminates.

This argument remains valid when an action discovers additional issues. New issues do not replenish the global budget. It also remains valid when a worker restarts, provided

the durable counter is authoritative and a lease does not authorize a second unrecorded dispatch. The per-issue cap and round cap are useful additional constraints, but the global cap is what prevents an unbounded stream of newly named issues from evading termination.

The proof is about bounded processing, not resolution. A controller can terminate with every difficult issue unresolved. It is also not a proof of wall-clock availability under arbitrary storage failure. If the process is indefinitely suspended or the database never responds, no local algorithm can guarantee a report at a fixed real-world time without an external supervisor. The operational design must therefore monitor leases and deadlines independently and recover a failure report when infrastructure returns.

A second invariant concerns release. Let $M(r)$ mean that run r has an unresolved material issue, $C(r)$ mean that its mandatory coverage and checks are complete, and $A(r, b)$ mean that the required authorized reviewers approved exact bundle b on the evidence from r . The release service requires

$$\text{Release}(r, b) \implies \neg M(r) \wedge C(r) \wedge A(r, b) \wedge \text{Integrity}(r, b). \quad (6.4)$$

This is a design invariant to be enforced by the release transaction. The controller's `READY_FOR_REVIEW` state supplies neither A nor release itself. An implementation test must attempt to bypass each conjunct, including through a stale approval or a source update, and verify rejection.

The invariant also exposes the limit of the design. If the inventory misses a material source and no check detects it, $C(r)$ could be incorrectly recorded. Formal verification of the release transaction would still prove only that it uses the recorded evidence correctly. Independent source review and empirical omission tests address the reliability of that evidence. The book's separation of proof claims is therefore retained inside the ensemble rather than abandoned at its most important decision.

6.17 Durable actions, leases and ambiguous execution

An action moves through `RESERVED`, `DISPATCHED`, and one of `SUCCEEDED`, `FAILED`, `TIMED_OUT`, or `RESULT_UNKNOWN`. The last state is necessary when the controller cannot determine whether a remote service performed the action. A lost connection after submission is different from a request rejected before dispatch. Both can lead to missing results, but only the former may already have consumed remote work.

The reservation transaction records the action identifier, input hashes, issue root, round, budget charge, adapter identity, timeout and idempotency key. An outbox entry is written in the same transaction. A worker claims that entry with a lease and a fencing token. The token increases when ownership changes, so an old worker cannot commit a result after a newer owner has taken responsibility. A valid late result can be retained as an observation without changing the closed action's authoritative state.

The dispatch boundary requires care. If the outbox records only "not yet sent" and a worker crashes after sending but before updating the database, another worker may

send again. A provider idempotency key can make that retry refer to the same operation, but not every provider supports this guarantee. The adapter must state its capability. Without remote idempotency, a recovery policy can choose to leave the result unknown rather than risk another paid request, or authorize a new counted action. It cannot truthfully promise exactly once remote execution.

The budget system reserves an upper bound on tokens or cost when those resources are capped. Settlement records actual usage when known. A refund of unused monetary reservation may be legitimate after a confirmed terminal response, but it does not decrement the issued-action count. Otherwise a sequence of failed requests could evade the maximum number of attempts. If actual usage exceeds a provider's documented bound, the run records an accounting discrepancy and stops new spending until the integration is investigated.

A compare-and-swap version protects issue updates. The worker returns evidence against the issue revision it received. If the issue has changed, the controller revalidates whether the evidence is still relevant before applying a resolution. This prevents two simultaneous actions from overwriting each other's objections or resolving an issue using a candidate that is no longer current.

Cancellation stops new reservations and attempts to cancel outstanding work. Remote cancellation may be best effort. The report records outstanding or late actions and keeps their budget charges. Cancellation cannot convert an incomplete run into a clean result, and it cannot delete the source of a later adverse finding. These details matter because regulatory evidence must survive ordinary operational events, including a person closing a browser or restarting a worker.

6.18 What counts as a justified repair

A repair has two responsibilities: remove the identified defect and preserve the requirements that were already correct. For a missing product-type condition, the revised body must include the type route, and the previously verified gift, exception and scope behavior must remain intact. The repair record identifies the changed expression and the affected checks. It cannot simply report that a new model answer received a higher score.

The original discrepancy remains immutable. The repair links to it and supplies resolution evidence: the source clause, changed candidate, witness before and after, regression checks and any reviewer decision. A failed repair becomes another candidate revision with a failed status. This is useful evidence about the investigation and should not be removed from the dossier merely because a later revision passes.

A repair can be rejected at several layers. The source may not support its new condition. The controlled-language rendering may no longer match the formal rule. The formal rule may use an unsupported operation. The Java package may miscompute the accepted expression. These are different failures. A compiler rejection means the current encoding cannot be delivered through that compiler; it does not necessarily re-

fute the legal reading. The next action might be a supported encoding or a documented implementation extension rather than a new interpretation.

Reference cases require special protection. When a candidate disagrees with a frozen expected outcome, the system records the disagreement and investigates both sides. It does not assume the reference is infallible, particularly in our same-author 22-case exercise. But changing the reference requires an independent resolution record with evidence and reviewer authority. A model cannot rewrite the expected result and then claim that its unchanged candidate now passes.

The same rule applies to formal domains. If a witness exposes a defect, narrowing the domain to exclude it is a substantive change. The domain change needs a source or operational justification, not merely a desire to eliminate the counterexample. Every equivalence claim includes the domain hash and assumptions, so an excluded scenario remains visible to reviewers.

Finally, repair must preserve dissent. If one supported reading is selected after review, the rejected reading, its strongest argument and the reason for rejection remain attached. Future evidence may make that earlier objection relevant. A system that stores only the winning candidate forces the next team to rediscover the same ambiguity and cannot explain why the original team chose as it did.

6.19 Scores that assist investigation without pretending to be probabilities

A single confidence number combines incompatible questions. A source can be authoritative while its application is ambiguous. A candidate can be internally consistent while missing a clause. A Java artifact can be verified against an unapproved rule. The proposed output therefore uses a score vector and explicit statuses rather than a scalar legal-confidence percentage.

The vector records source coverage, dependency completeness, supported and unsupported assumptions, unresolved material objections, scenario agreement, formal-check status, implementation-preservation status and search coverage. Some components are counts or categorical states. Others may be empirical scores, such as retrieval relevance. Their definitions are stored with the run. A score is not allowed to compensate for a failed mandatory condition.

For example, a candidate might cover eighteen of eighteen inventoried paragraphs with dispositions, yet have two unresolved incorporated sources and one material classification question. Displaying only “100 percent coverage” would mislead. The report must state that source-unit disposition is complete while dependency and interpretation resolution are incomplete. Conversely, a lower model ranking need not disqualify a candidate whose source argument remains undefeated.

Search priority can be a weighted function of expected information gain, action cost and issue materiality. The weights are scheduling choices. They may be tuned under a

declared evaluation protocol, but they are not probabilities of legal correctness. Every displayed numerical score includes its purpose and calibration status. A field named `probability_correct` remains absent or null until an appropriate target-specific calibration study supports a precisely defined event.

A useful shortlist can be a Pareto set. Candidate h dominates candidate j only when it is no worse on every declared comparable dimension and better on at least one. Even then, dominance is an aid to presentation, not legal defeat. A candidate with a unique source argument should not disappear merely because another has more completed engineering checks. Unsupported candidates can be rejected for specific reasons; incomparable supported candidates remain visible.

Set-valued reporting is especially valuable when authority is genuinely incomplete. The dossier may present two conditional proposals: under definition D_1 , use rule R_1 ; under definition D_2 , use rule R_2 . It must identify who can resolve the definition and what operations remain blocked. This is more informative than selecting one proposal with a decorative confidence interval whose coverage has never been established.

6.20 A complete dry run: repairing an explicit omission

The following trace is a designed example for the future controller. It has not been executed by a live model and does not replace the retained second-circular results. The source packet contains the selected paragraph, its surrounding context and the unresolved dependency records. The activity profile and factual schema are fixed before proposals begin.

Four initial actions are reserved. The normative reader proposes $G \wedge P \wedge \neg D$, omitting product-type linkage. The controlled-language reader proposes $G \wedge (P \vee T) \wedge \neg D$. The alternative reader agrees that both product and type links require examination and identifies a separate classification ambiguity. The inventory member confirms the paragraph and footnote enumeration but reports an unavailable incorporated source. The run has issued four actions; eight remain under the reference global cap.

Reconciliation creates issue I_1 for the missing type route, I_2 for the classification ambiguity and I_3 for the unavailable dependency. The comparator does not average the two formulas. It constructs the known-fact scenario $G = \text{T}, P = \text{F}, T = \text{T}, D = \text{F}$, with scope established. The first candidate returns false and the second true. The source passage explicitly includes the type route, so I_1 has a concrete textual and behavioral basis.

In resolution round one, the controller reserves a source-grounded repair action for I_1 and a dependency retrieval action for I_3 . The issued count becomes six before either is dispatched. The repair produces a child candidate that adds T to the disjunction. Its controlled-language rendering, type checks and witness replay pass. The regression set checks that fee discounts remain excluded and scope uncertainty retains its established behavior. The issue is resolved by evidence with the original discrepancy and both candidate versions retained.

The dependency retrieval times out. Its action is terminal and counted; I_3 remains unresolved. The run does not claim complete circular interpretation just because I_1 was repaired. If the unavailable source could affect the selected classification, both I_2 and the relevant candidate remain blocked. The report can still state precisely that the explicit type-of-product omission was found and corrected.

This trace illustrates two kinds of progress. The repaired omission supplies positive evidence about the ensemble's ability to catch that seeded defect in this designed case. The timeout supplies operational evidence about the controller's accounting. Neither is a measured error rate on real circulars. The prototype acceptance test will reproduce these transitions with scripted members before any live-model comparison is attempted.

6.21 A complete dry run: an ambiguity that survives the budget

Continue with the hypothetical cashback question. One candidate treats a later payment as a fee discount when it reimburses an actually paid charge and is capped at that charge. Another requires a reduction in the contractual fee itself. Their rules agree on a direct waiver and disagree on a later reimbursement. Neither definition is accepted merely because it produces a convenient outcome for the bank.

The scenario designer creates three synthetic offers. Offer A waives a charge of HK\$100. Offer B charges HK\$100 and later pays HK\$100 back under a linked promotion. Offer C charges HK\$100 and pays HK\$150. The factual amounts are known, but the legal classification of the later payment remains disputed. The dossier shows the candidates' conclusions and the exact definition each requires. It also separates any excess payment into a proposed component analysis rather than silently treating the entire amount as a discount.

In round one, a retrieval action seeks the incorporated definition. If the source is inaccessible, the result is a failed retrieval, not support for either reading. In round two, a bounded argument action asks each candidate to identify the strongest source-based reason against its own definition. Suppose both return plausible but unresolved explanations and no new authority. The controller records the arguments, but a longer explanation alone does not constitute resolution.

In round three, a question-selection action prepares a focused review question with the contrasting offers and the missing premise. The action can successfully produce the question while the issue remains unresolved. After the third round, no further automatic resolution round is authorized. The candidate set is preserved, the issue receives a budget stop reason, and the run closes as `BLOCKED_UNRESOLVED`.

The report states which offers are affected, what the candidates agree on, where they differ, which sources were inspected, which source remains unavailable, and what an authorized reviewer must decide. It contains no assertion that three rounds make the leading reading probably correct. The unresolved classification is not replaced by a

false Boolean, and the Java release route remains blocked for the affected control.

A later authorized reviewer may establish the definition from an applicable source or decide that a conservative bank policy should govern the uncertain category. Those are distinct decisions. A bank policy that declines all cashback offers pending clarification can be operationally useful, but it must be labelled as the bank's precautionary choice. It cannot be reported as a proven SFC prohibition unless the source supports that proposition.

6.22 The uncertainty report is a first-class result

An uncertainty report must let a reviewer reconstruct the decision without reading raw model transcripts. It starts with the source and activity perimeter, the proposed control, the run's terminal status and the operational consequence. A blocked report should say what is blocked. "Low confidence" is not enough: the reader needs to know whether the uncertainty concerns an incentive category, an effective date, a missing definition or the integrity of the entire source.

The report then presents the surviving interpretations in comparable form. For each, give the controlled-language statement, formal rule identifier, assumptions, supporting sources, strongest objections and distinguishing scenarios. Use the same factual scenarios across candidates so that the difference is visible. Candidate ordering may reflect a declared presentation policy, but must not imply that the first row has been approved.

Every detected discrepancy appears in an issue register with its processing and resolution states, materiality, affected decisions, attempted actions and final reason. Resolved issues remain present because their history explains why the final candidate differs from the initial one. Unresolved issues identify the missing evidence or human decision. A report that shows only unresolved issues would conceal the repairs; a report that shows only repaired issues would conceal the remaining risk.

Coverage is reported against explicit denominators. Count source units inventoried and dispositioned, dependencies required and acquired, interpretation families identified and explored, mandatory roles completed, candidate comparisons attempted, and verification obligations passed or incomplete. The denominators must be retained even when the run exceeds a display limit. Large inventories can be paginated or stored as linked machine-readable records; they cannot be silently truncated to make the report look complete.

The execution account records configured limits, issued actions, rounds used, timeouts, unknown remote results, no-progress decisions, elapsed time, token and cost reservations, actual usage when available, and unspent budget. A run that stops early because it lacks authority should not look like a run that exhausts its resources. Both may be blocked, but their next useful actions differ.

A concise machine-readable summary might look as follows. Identifiers here are illustrative; the complete contract in Chapter 8 adds validated references and integrity fields.

Listing 6.2: A blocked result preserves alternatives and the next decision.

```
{
  "schema_version": "interpretation.v1",
  "run_status": "BLOCKED_UNRESOLVED",
  "source_ref": "23EC46-retained-version",
  "selected_slice": "paragraph-10",
  "candidate_ids": ["cashback-reimbursement", "cashback-fee-reduction"],
  "material_unresolved_issue_ids": ["classification-cashback"],
  "release_eligible": false,
  "processing_stop": "ROUND_LIMIT",
  "rounds_issued": 3,
  "probability_of_legal_correctness": null,
  "next_decision": "Establish the applicable fee-discount definition",
  "next_decision_owner_role": "LEGAL_INTERPRETATION_REVIEWER"
}
```

The report should also state the limits of its evidence. In the second-circular exercise, the expected cases and candidate were prepared by the same author. That fact belongs beside the verification summary. A reviewer should not have to find a footnote in a separate research note to discover that the apparent oracle was not independent. Similarly, a solver timeout must appear as incomplete verification, not disappear from an average pass rate.

Reports are immutable versions. A corrected report points to its predecessor and explains the correction. Approval binds the exact report, candidate, source packet and Java bundle. The report is thus both a readable explanation and an index into reproducible evidence. Its human readability matters because a technically complete JSON object that no reviewer can understand does not solve the original manual-interpretation problem.

6.23 Agreement can still conceal a missed obligation

The hardest example is not a disagreement. Suppose every generator ignores a footnote that changes applicability. All formulas, scenarios and Java results agree because they share the same omission. Repeated self-consistency sampling would reinforce that agreement. Roundtrip translation could reproduce the same shortened meaning. Differential execution would confirm that the wrong specification is implemented consistently.

The independent inventory creates another opportunity to catch the omission. It requires a disposition for the footnote. A normative reader responsible for that unit may identify the qualification even if the candidate generators did not. A source-to-rule coverage check then opens an issue because the footnote has no corresponding assumption or rule. The repair propagates through candidate comparison and implementation checks.

Now suppose the inventory also misses the footnote because both extraction paths use

the same defective text layer. Rendered-document inspection or a distinct acquisition route may detect it. If those are absent or also fail, the system can still miss the obligation. This remaining possibility is why the product's claims must be measured against independent source review and adversarial omission tests. Redundancy reduces opportunities for undetected error only to the extent that the redundant activities can fail differently.

A second common-mode failure concerns vocabulary. All members may agree that a payment is a discount because the prompt labels it "fee rebate." The remedy is to distinguish observed facts from proposed classifications. Present the payment, charge, contractual relationship and timing without inserting the disputed label as an established input. Then ask what evidence supports the classification. Blindness to the preferred answer is useful only if the factual packet itself has not already embedded that answer.

A third failure concerns evaluation. If the reference reviewer sees the model's answer before forming an expected outcome, the reference may inherit the model's mistake. The ensemble may then appear accurate against a contaminated oracle. Independent initial judgments and recorded adjudication are necessary to measure whether the product actually discovers errors. They are also part of a credible operational review process, especially for novel or high-impact controls.

6.24 Interaction between uncertainty and operational action

The interpretation service should not decide every operational response. Its job is to state what is known, what is disputed and which controls are justified by the approved specification. The bank's host system then applies an explicit operational policy to outcomes such as unknown, conflict or blocked release. Those policies may include stopping an offer, routing a case to review or using an already approved fallback control.

This separation avoids a dangerous ambiguity in "fail closed." Closing an uncertain marketing offer may be appropriate in a particular process, but stopping a payment, withholding a disclosure or disabling an existing control could create other obligations or harms. The responsible business and control owners must approve the action for the relevant workflow. The interpretation engine supplies a reasoned status; it does not invent a universal response for all banking operations.

For the selected gift rule, a true result indicates that the specified prohibition condition is satisfied. A false result says only that this condition is not established under the accepted facts and rule. It does not authorize the offer under every other requirement. The host must combine this result with other applicable controls, such as disclosure or suitability obligations, according to an independently reviewed orchestration policy.

An unresolved interpretation can sometimes be isolated. If the disputed cashback definition affects only one incentive category and the system can reliably identify that category, a reviewer may approve a limited control for unaffected cases while routing the disputed category to manual review. The scope restriction must be explicit, tested and

included in the approved bundle. It is not enough to state informally that the system will “avoid edge cases.”

If impact cannot be isolated, the block applies more broadly. The system should prefer an honest incomplete release proposal over an unsupported claim of separability. The uncertainty report gives the decision-makers the information needed to choose between narrowing scope, obtaining more evidence, retaining an existing control or postponing deployment.

6.25 Acceptance conditions for the central method

The first implementation should be accepted against deterministic scenarios before live model performance is assessed. A scripted member can deliberately omit the type route, supply a contradictory source version, fail to respond, or return a malformed candidate. These fixtures isolate controller behavior from model variability. Passing them establishes specified engineering behavior, not legal translation accuracy.

The controller must demonstrate that a unanimous but uncovered source unit opens an issue; that a supported minority candidate remains in the report; that a formal mismatch produces a replayable witness; that an unavailable dependency cannot be marked resolved by repeated explanation; and that a successful repair reruns affected checks. Each condition targets a concrete way an apparently successful ensemble could hide an error.

Budget tests must reach exact boundaries. With a cap of twelve, the thirteenth external dispatch must be rejected even if it is proposed by a new child issue. A crash after dispatch must not reset the count. A timeout must consume its issued action. A new worker with an old lease must fail to overwrite the current result. A provider retry must either reuse a verified idempotency key or consume a separately authorized attempt.

Report tests must verify completeness for successful, blocked, cancelled and failed runs. Every issue identifier must have a disposition; every unresolved material issue must appear in the blocking summary; every candidate revision must retain its parent and source references; and every displayed score must identify its purpose. Empty extension sets, solver unknowns and missing member results must remain explicit rather than being converted to success by default logic.

The release test is adversarial. Attempt release from `READY_FOR_REVIEW` without approvals, with an approval for a different bundle, after a source change, with an unresolved material issue, and after an integrity failure. All attempts must fail. A positive release fixture requires complete checks and authenticated approvals for the exact package. The local synthetic identity mechanism is sufficient to test the state transitions, but enterprise authentication and role enforcement remain a deployment requirement.

These conditions give an implementation agent a concrete target. They also preserve the intellectual purpose of the ensemble: investigate alternatives, expose missing evidence, and retain uncertainty when the evidence does not settle the question. The resulting product is useful because it makes responsibility and reasoning visible, not because a

collection of fallible components can be renamed a proof of English meaning.

Chapter 7

From an accepted interpretation to verified Java behavior

7.1 The specification becomes the comparison target

Once reviewers accept a particular interpretation, a different verification problem begins. The source question was whether the rule expresses the relevant regulatory requirement. The implementation question is whether the deployed program preserves that accepted rule for the inputs and histories it supports. Keeping these questions separate allows strong engineering evidence without misrepresenting it as legal proof.

The accepted object is larger than a Boolean formula. It includes the source versions, activity profile, factual definitions, subject identity, time policy, unknown and conflict semantics, exception structure, result interpretation and coverage limits. If any of these is omitted, two implementations can agree on a formula while giving different advice in practice. A Java adapter that labels all rebates as discounts has changed the rule's application even if its Boolean operator code is flawless.

The verification target must therefore name the complete bundle and its permitted input domain. The bundle is immutable and content-addressed. The comparison includes the decision, missing and blocking facts, errors, source references and execution trace. Different backends have different execution identities, so equality of their entire serialized result hashes is not the right criterion. Compare the specified semantic fields and independently verify each backend's own result identity.

The existing workbench already provides a finite RuleIR interpreter, a Java runtime, generated policy classes and a release workflow. The ensemble proposed in Chapter 6 is an extension before that delivery path. It should produce reviewed candidates that the existing validator can accept, rather than silently expanding the runtime language whenever a model proposes a new legal construct. Unsupported meaning remains an explicit engineering and review issue.

7.2 Facts are evidence-bearing values

A production fact is not simply a field such as `gift=true`. It has a type, value or uncertainty status, evidence references, validity interval and recording time. The validity interval says when the evidence applies to the underlying situation. The recording time says when the system knew it. These times answer different questions and both are

needed for historical reconstruction.

Suppose the bank learns on 10 October that a fee record effective on 1 October was incorrect. A decision replay asking what the system knew on 5 October should not use the later correction. A corrected assessment made on 10 October may use it to reassess the earlier event. The two assessments should coexist. Replacing the old record would erase the evidence needed to explain the original decision.

The existing normalization boundary selects records valid at `valid_at` and recorded no later than `known_at`. A correction names the evidence it supersedes. Two admissible, non-superseded records with different values create a conflict. Arrival order is not a source-priority rule. If both records agree, their evidence may be combined without inventing a conflict.

Intervals are half-open: $[a, b)$ includes the start and excludes the end. If a record is valid until midnight, it is not valid at that midnight. This convention avoids overlapping adjacent intervals when one record ends exactly as another begins. Tests must exercise equality at both endpoints, not merely dates well inside an interval. All assessment timestamps use the canonical UTC format with six fractional digits; legal calendar dates are separate values.

Unknown facts have reasons. Missing evidence, stale evidence, an unreviewed classification and unresolved event order can all lead to unknown, but they call for different remedies. A host should not display the same uninformative message for every case. Conflict similarly preserves the competing evidence identifiers. The evaluator does not decide which source is legally superior without a reviewed normalization rule.

This design puts factual interpretation where it belongs: at a visible boundary. For the circular, classifying a benefit as a gift or fee discount is an evidenced assessment. The decision engine consumes that assessment under its approved definition. If the ensemble has not resolved the definition, the adapter cannot supply a known value merely to satisfy a required input field.

7.3 Exact arithmetic prevents a second kind of ambiguity

The existing scalar types are Boolean, integer, Hong Kong dollar money in cents, and Gregorian date. Integer and money values use canonical decimal strings and arbitrary-precision arithmetic. A monetary value of HK\$100 is represented as "10000". The string representation avoids a transport layer converting the value through a floating-point number before the exact arithmetic begins.

Units are part of the contract. A number expressed in dollars and the same number expressed in cents differ by a factor of one hundred. A compiler cannot infer which was intended from the digits alone. The host mapping must identify the source unit, conversion and evidence. Declaration-specific bounds can reject implausible inputs, but a plausibility check is not a substitute for a reviewed unit mapping.

Consider proportional ownership of a monetary amount. If the accepted operation is exact scaling by a rational fraction n/d , the runtime computes the product and quotient using integers. Let x be cents. It checks

$$nx = dq + r, \quad 0 \leq r < d \quad (7.1)$$

for nonnegative nx ; for signed values the implementation's quotient and remainder convention must still make zero remainder the exactness test. A nonzero remainder means that the result is not an integer number of cents. The current contract returns `E_INEXACT_SCALE`; it does not choose a rounding policy on the user's behalf.

If a bank requires rounding, that is a substantive extension. The rule must state which quantity is rounded, at what stage, in which direction and under whose policy. Rounding each component before summation can differ from rounding the sum. Two correct implementations of different policies can therefore disagree. The ensemble should recognize this as a specification issue rather than treating one result as a numerical accident.

Foreign-currency conversion is outside RuleIR 0.1. A reviewed adapter supplies an HKD amount with rate, valuation instant and rounding evidence. A missing rate creates unknown evidence. It must not trigger a guessed conversion or reuse an unboundedly old rate. The bank's data owner is responsible for the freshness and valuation policy, while engineering verifies that the adapter implements it.

Dates require similar precision. A date is not a timestamp, and an obligation measured in business days needs a calendar policy. The current expression language compares validated dates but leaves date arithmetic to reviewed event scheduling. This restricted design prevents a generic arithmetic library from silently deciding whether a public holiday counts toward a regulatory deadline. The appropriate calendar must be versioned with the assessment.

7.4 Unknown, conflict and scope have an order

The three-valued Boolean rules in Chapter 3 govern unknown values, but they are not the entire runtime semantics. Before evaluating the requested rule, the existing implementation checks conflicting evidence in its static transitive dependency closure. That closure includes scope, body, exceptions and branches that might later be skipped. A conflict in the closure returns conflict even if another route could determine the Boolean result.

This is a conservative evidence policy. For example, an actor may appear outside the selected scope, but a conflicting fact in the rule's static closure still produces conflict before scope evaluation. The second-circular case that exercises this behavior is deliberately different from ordinary short-circuit intuition. The specification must teach it because an engineer using conventional Java `&&` or `||` could otherwise implement a different result.

After conflict checking, scope is evaluated. False scope returns `OUT_OF_SCOPE`; unknown

scope returns UNKNOWN with a scope reason; true scope permits evaluation of the body. A body that would be false does not turn unknown scope into a known result. The system is answering a scoped condition, and the applicability question remains unresolved.

Within the body, conjunction and disjunction evaluate all operands in declared order. This eager behavior retains complete evaluated evidence and exposes errors or conflicts even if an earlier operand would determine an ordinary Boolean answer. Missing inputs can appear in a known result, while blocking inputs are empty when the unknowns do not prevent that result. The distinction is useful: the bank may know the condition is false while still needing to repair a stale input feed.

A conditional expression behaves differently. It evaluates the condition and only the selected branch. An unknown condition returns unknown without evaluating either branch, even when the branches appear equal. Skipped branches remain type checked and part of the static dependency closure. Their trace roots are marked skipped so that the explanation does not invent an evaluation that never occurred.

Defaults add another layer. All exception guards are evaluated first. Two true guards produce conflict even if their values would agree. An unknown guard can prevent selection of another true guard because the unknown might become a competing exception. One true guard with all others false selects its value; all false guards select the base. Priority is expressed only by explicit nesting with a reviewed interpretation. Array order is not hidden legal priority.

These rules are not universal truths about law or logic. They are the existing workbench's specified execution choices. The ensemble may identify a regulation that needs different semantics. The correct response is to propose and review an extension or keep the case outside the supported fragment, not to reinterpret the current operators ad hoc.

7.5 Validation failures are part of observable behavior

A verifier must compare invalid-input behavior as well as successful decisions. Malformed JSON, duplicate keys, noncanonical numbers or unpaired Unicode surrogates can fail at the transport boundary before a semantic result can be hashed. Valid JSON with an invalid bundle or snapshot can instead produce a structured semantic error. These outcomes have different evidence and should not be conflated.

The current validation order is schema, references and duplicates, types, cycles, time, static-closure conflicts and evaluation. A deterministic order makes error reports reproducible when an input has several defects. It also prevents a runtime from accessing an invalid reference before discovering that the bundle should have been rejected. Fixed error codes and pointers are compared across backends; dependency-library exception text is not the public contract.

Resource limits are explicit. The present canonical JSON boundary is bounded in size and depth, and the rule graph has node and expanded-call-depth limits. An oversized acyclic expression is a resource error; a cycle is a cycle error. An implementation must not catch either and continue with a truncated rule. The same principle applies to the

new ensemble's graph and report limits.

Tests should combine defects when precedence matters. A snapshot can contain a conflict and an expired validity interval. A bundle can contain a type error in a branch that would otherwise be skipped. A default can have two applicable guards whose values contain errors that should never be evaluated. Isolated operator tests cannot establish the behavior of these combinations.

A useful specification test states both the expected result and why a tempting implementation would differ. For example, a short-circuit implementation might return false before encountering an error in the second operand. The accepted eager semantics requires the error. This counterexample teaches the contract and makes the test more durable than an assertion copied from the current function's output.

7.6 Independent oracles and their limits

An oracle is a source of expected behavior against which an implementation is compared. Independence has several meanings. A separately written evaluator can be independent of the production code while relying on the same mistaken interpretation. A human reference can be independent of the model's answer while sharing the same incomplete source packet. The verification report must specify which independence it has actually achieved.

The retained 22-case oracle was frozen before the candidate bundle, which protects against some forms of adapting expected outputs to the implementation. It was nevertheless prepared by the same author. The 729-case enumeration gives complete coverage of six ternary abstract inputs for the selected formula. Neither fact establishes independent legal interpretation or whole-document coverage.

A stronger reference process begins with independent source reading and scenario judgment. Reviewers state the expected result, assumptions, supporting passage and unresolved questions before seeing the candidate output. Disagreements are adjudicated with reasons or retained as alternative acceptable outcomes. The reference then has a defined relationship to the evidence rather than being a single unexplained label.

For engineering checks, a small executable oracle can be valuable when its method is genuinely different. The second-circular body has a read-once unate structure, which permits its Boolean-completion check to agree with the specified three-valued result. The general expression $p \vee \neg p$ shows why that argument cannot be extended to every RuleIR expression: under the specified semantics, unknown p yields unknown, while every classical completion yields true. An oracle based on the wrong generalization would reject correct implementations or bless incorrect ones.

The oracle itself therefore needs a stated domain and a small set of discriminating examples. The workbench should retain oracle author, method, source versions, review status and any shared components. A generated oracle that imports the same evaluator under a different function name supplies little independent evidence. A reference that is changed during repair remains useful only when the change is separately justified

and versioned.

7.7 Mutation tests measure what the checks can notice

A mutation deliberately changes a candidate or implementation in a way that represents a plausible error. If the verification process rejects it for the intended reason, the test demonstrates sensitivity to that error. It does not estimate the frequency of that error in real use, and a high mutation score does not prove complete correctness.

The second-circular exercise used four meaningful mutations: remove the discount exception, omit product-type linkage, assume gift classification and ignore scope. The retained named cases detect all four after compilation. This is more informative than mutating arbitrary characters because each change corresponds to a failure a reviewer or programmer could plausibly make.

The ensemble extension should add source and process mutations. Remove a footnote from one extraction; supply a stale dependency to one member; label a speculative classification as established; suppress a minority candidate in the report; reset an issue counter after a revision; or reuse an approval after a source change. Each mutation targets a distinct assurance boundary. Passing only formula mutations would leave the new controller largely untested.

Some mutations are equivalent under the selected domain. Changing an expression may not change any supported outcome, and failure to detect that mutation is not necessarily a defect. The mutation report must distinguish equivalent mutations, invalid encodings and live semantic defects. Counting all undetected mutations as failures, or silently excluding inconvenient ones, would distort the result.

A mutation is killed when a required property changes and the check catches it. A compiler crash is not always the intended detection: if the task is to test whether the oracle notices an omitted exception, a syntax-invalid mutant does not answer the question. Mutation construction should preserve validity where the research question concerns semantic sensitivity. Invalid-input tests remain a separate and necessary category.

7.8 Symbolic comparison requires a feasible domain

For supported expressions, symbolic comparison can search for inputs on which two candidates differ. The domain records admissible values, knownness conditions and relationships among facts. Before asking whether the outputs differ, the solver must establish that the domain has at least one model. Otherwise an inconsistent domain makes every equivalence query vacuously true.

Suppose one domain constraint says that a payment exceeds the fee and another says that it cannot exceed the same fee. A query finding no differing outcome under both constraints has taught us nothing about the real candidates. The workbench returns an inconsistent-domain result. The domain must be repaired or reviewed before any

equivalence claim is made.

A satisfiable difference query returns a witness. The witness must be decoded to a concrete fact snapshot and replayed in both the reference evaluator and Java. This replay checks that the symbolic encoding corresponds to executable semantics. A solver result alone can be wrong relative to the claimed runtime if the encoding mishandles unknown values, scope or arithmetic. The witness and both results belong in the report.

An unsatisfiable difference query establishes no differing result within the checked fragment and domain, subject to the solver and encoding assumptions. It does not establish that the candidates use the same legal definition, cover the same source material or behave identically outside the domain. The result label must therefore include the domain and comparison projection.

The current comparison fragment supports documented Boolean and exact integer/money reasoning with explicit knownness. It rejects unsupported date, scale, default and event constructs. The two solver checks share a bounded time budget, with a separate worker deadline. Unknown, timeout, missing Java replay or unsupported constructs never become equivalence. The ensemble may schedule another permitted check, but cannot relabel the incomplete result as a pass.

Counterexample-guided refinement is useful when a witness exposes an imprecise abstraction. The engineer identifies why the witness is infeasible or why the encoding omitted a required condition, changes the abstraction with a documented reason, and reruns the query. The discipline matters: excluding a witness because it is inconvenient is not refinement. The revised domain must correspond to a reviewed fact about the application.

7.9 Property tests should follow the semantics

Property-based tests generate many inputs and check a relation that should hold. They can explore combinations that named examples miss, but the property must be valid for the specified semantics. Classical Boolean identities cannot all be transferred to unknown or conflict behavior. Likewise, reordering operands may preserve a value while changing the required trace or error precedence.

Useful properties include deterministic evaluation for identical canonical requests, rejection of undeclared facts where the schema requires it, exact integer arithmetic, preservation of known values under irrelevant evidence additions, and stable source references under deterministic rendering. Each property needs a precise precondition. Adding a conflicting record in the static dependency closure is not an irrelevant addition.

Metamorphic tests compare related inputs when a direct oracle is difficult. If a known amount increases while all other facts remain fixed, a simple lower-bound threshold condition should not change from true to false. This monotonicity applies only to the stated threshold fragment. It need not hold for a default with an upper-bound exception or for a scope condition that changes at the same time. The test generator should derive

its applicable domain from the property, not assume the property is universal.

For evidence time, a carefully limited relation is useful: replaying an earlier `known_at` should exclude records first recorded later. Increasing `known_at` does not necessarily preserve the decision, because a correction or conflict may become visible. A test that demands decision monotonicity in knowledge time would impose a false property. The intended invariant concerns which records are admissible, not whether later knowledge always makes the same answer more certain.

Fuzzing transport boundaries is a different activity. It tests duplicate keys, size limits, malformed timestamps, unusual Unicode and nesting limits. Its success criterion is controlled rejection or valid deterministic behavior, not legal interpretation. These tests protect the integrity of the evidence and execution chain and should be included in the engineering report under that role.

7.10 The actual generated Java interface

The current emitter validates the bundle, canonicalizes it and embeds its exact bytes in a generated class. The class name is `hk.legalmath.Policy_` followed by the first twenty hexadecimal digits of the bundle hash. The class exposes the full bundle hash and delegates evaluation to the Java semantic runtime. This is a generated, immutable policy binding; it is not the older proposal's imagined record-based API or a proof that each legal rule has been translated to hand-optimized Java control flow.

The two current entry points are:

Listing 7.1: Implemented generated-policy API.

```
public static String evaluate(
    String snapshotJson,
    String ruleId,
    String validAt,
    String knownAt,
    String mode);

public static Map<String, Object> evaluate(
    Map<String, Object> snapshot,
    String ruleId,
    String validAt,
    String knownAt,
    String mode);
```

The string overload returns strict JSON. The map overload returns a deeply immutable result. The generated policy and Java runtime operate without Python, a language model, a solver or network retrieval. This property is important for a bank integration: the interpretation workbench prepares and verifies a package, while production decision execution can remain local and deterministic.

A concrete host compiles against the actual generated class name from the build manifest. It supplies a validated snapshot, exact rule identifier, both assessment times and the requested mode. The mode does not grant authority. A caller cannot turn a draft into an approved control by passing the word “production.” The release service and host must establish approval and applicability separately.

The conceptual integration looks as follows. The symbolic class name in the listing is replaced by the actual generated name when the integration fixture is built; the release lookup and transaction methods are host responsibilities.

Listing 7.2: Host responsibilities around deterministic evaluation.

```
ApprovedRelease release = releases.resolve(profile, validAt, knownAt);
release.verifyAuthorityAndDigests();
ConsistentSnapshot s = adapter.readSnapshotWithVersions(subject, order);
String resultJson = Policy_HASH.evaluate(
    s.strictJson(), release.ruleId(), validAt, knownAt, "production");
Decision decision = decoder.decodeStrictly(resultJson);
HostDisposition disposition = hostPolicy.map(decision);
transactions.commitIfVersionsUnchanged(
    s.versions(), order.id(), order.payloadHash(),
    release.identity(), decision, disposition);
```

ApprovedRelease, ConsistentSnapshot and the host transaction interfaces in this illustration are proposed integration concepts, not classes already supplied by the generated SDK. Their responsibilities must be implemented in the bank’s environment. The example makes the boundary explicit so an agent does not mistake illustrative types for an existing dependency.

The host must handle every tagged result. A true selected prohibition condition may require stopping the relevant offer; false permits only progression to other controls under the host policy. Unknown, conflict, out-of-scope and error each have explicit dispositions. No result string should be cast to a Boolean by a language’s truthiness rules. The decoder rejects unknown status tags so that a future runtime extension cannot silently take an old default branch.

7.11 Traces are part of the delivered meaning

A decision without an explanation is difficult to investigate. The runtime trace records node identifiers, operators, children, type, value or status, evidence references and source spans. Shared rule evaluations are memoized, and skipped branches have explicit markers. The trace is deterministic under the request and engine version, making it suitable for replay and comparison.

Trace fidelity is a separate verification obligation. Two programs may return true while one omits the evidence for the exception or incorrectly claims to have evaluated a skipped branch. A regulator-facing explanation assembled from that trace could be misleading even though the Boolean result matches. The comparison therefore

includes trace structure and evidence, not just the top-level status.

Natural-language explanations are generated from the verified trace and approved source mappings. A language model may help render a readable summary, but it cannot alter the computed result or invent a justification. If a summary mentions a source that the trace and bundle do not contain, validation should reject or flag it. The authoritative explanation remains reconstructible without the summary model.

The current source-span policy associates trace nodes with the owning rule's explicit citations and retains exception citations in the bundle. That is a coarser mapping than a separate legal proof for every arithmetic operation. The report should describe the granularity honestly. A future refinement may attach more precise spans, but a dense set of citations should not be mistaken for a formal demonstration that each mapping is legally correct.

Execution hashes identify the canonical request and result under a particular backend version. They help detect altered bytes and reproduce behavior. They do not authenticate a reviewer or prove that a document came from the regulator. Authentication, source acquisition and approval are separate controls. Confusing these roles is a common source of overclaiming in systems described as auditable.

7.12 Event histories add obligations that a snapshot cannot express

Some requirements concern sequences: obtain consent before an action, honor a withdrawal, provide a document within a period, or preserve a breach after late performance. A current Boolean snapshot cannot express all of these relationships. The existing event component replays finite attributed histories using a separately versioned Java entry point, `EventReplay.replay(Map<String, Object>)`.

A stream header binds subject, category, actor, action, profile and inception. Events belong to that header; an API path cannot redirect their meaning to a different subject. Duplicate identical events are idempotent, while conflicting payloads or sequence reuse are errors. Ordering authority is explicit. If event order is unresolved, the system must not invent a sequence based solely on arrival.

Consent illustrates why completeness matters. Absence of a grant in an incomplete history does not establish that no grant occurred. A completeness watermark or a reviewed initial state supplies the relevant observation boundary. A watermark cannot cover a time later than the recording instant of its assertion. A withdrawal ends the applicable consent generation, and a later regrant uses a new evidence identity rather than resurrecting a previously withdrawn record.

For an achievement obligation, activation opens a duty with a deadline. The selected contract uses the interval from activation up to, but excluding, the deadline for timely performance. Performance exactly at the excluded endpoint is therefore late under that contract. This is an engineering convention that must match the reviewed legal timing

rule; a different legal rule requires a different specified boundary, not a casual adjustment in the host.

Late performance does not erase the historical breach. The system can record both that a duty was breached and that it was later fulfilled. Similarly, an observed action is retained even if an advisory control would have prohibited it. Deleting inconvenient observations would make replay describe an idealized compliant world rather than the bank's actual history.

Each opened duty binds the source bundle that created it. A later amendment does not automatically migrate existing duties. Migration may be appropriate, but it requires an explicit legal and operational policy about which obligations change, which dates apply and how historical findings are retained. The current MVP has no automatic migration implementation. The ensemble can identify the question and prepare alternatives; it cannot assume the answer from the new publication date alone.

7.13 Build, verification and approval form an acyclic chain

A build manifest records the bundle, generated source, runtime source, toolchain, Java target, dependencies and JAR digest. A verification report then names the build and exact JAR it tested. A release manifest names that build and verification report, together with applicability and effective-time information. Approval binds the final release manifest and bundle.

The direction matters. Embedding a manifest that contains the JAR's own digest inside that JAR creates a circular identity problem. The current design keeps these records external and acyclic. A reviewer approves the exact tested bytes through the final manifest, rather than approving a mutable directory or a human-readable version label.

Candidate compilation occurs before legal approval, because reviewers need a concrete package and verification evidence to inspect. Compilation does not grant deployment authority. Export into a controlled release route occurs only after the required approvals and checks. This distinction lets engineers do useful reversible work without repeatedly asking for permission, while preserving the human decision at the point where it matters.

Reproducible builds strengthen the identity chain. Under the same declared toolchain and inputs, build in separate directories with fixed archive-entry timestamps and compare emitted source and binary hashes. A mismatch should be investigated and recorded. Reproducibility does not prove semantic correctness, but it reduces uncertainty about which bytes correspond to the reviewed source.

The local MVP uses synthetic identities and unsigned historical archives to exercise workflow. A bank deployment needs enterprise identity, role separation, key management and independently verified release signatures or equivalent controls. A hash proves equality of bytes under its cryptographic assumption; it does not prove who

authorized those bytes. The manuscript's implementation plan keeps that integration distinct from the local demonstration.

7.14 A correct decision can still race with the transaction

Suppose a control depends on consent and exposure. The host reads both, evaluates an order and then commits it. Between the read and commit, consent may be withdrawn or another order may consume the available limit. The decision was correct for its snapshot but is no longer valid for the transaction being committed. This is a concurrency problem outside the pure rule evaluator.

The host should read a consistent snapshot with version identifiers, evaluate exposure including the proposed order, and atomically validate the versions while reserving the exposure and recording the decision. If the versions changed, it rereads and reevaluates under a bounded transaction retry policy. That policy is separate from interpretation-resolution retries and must not share an ambiguous counter called simply "attempts."

Idempotency protects against duplicate submissions. Repeating the same order identifier and payload should recover the same committed result or reservation. Using the same identifier with a different payload is a conflict. The identity must include the legally relevant order content, not merely a user-interface request number. Otherwise a retry can accidentally authorize a changed transaction.

Release selection has a similar race. The host records the release identity used for the decision and verifies that the applicable release policy permits it at commit. Whether an in-flight transaction can finish under a prior release is a bank policy requiring explicit treatment. The rule engine cannot resolve that question by consulting its process clock or loading whatever JAR is newest.

A rollback also has legal meaning. Restoring a previous software build can repair an implementation defect if that build still expresses the applicable rule. Restoring a superseded legal regime is different. The incident procedure must separate software rollback, operational suspension and legal-rule selection. Historical packages remain available for replay even when they cannot be used for new decisions.

7.15 A verification dossier that supports a release decision

The final technical dossier links the accepted interpretation to its implementation through several kinds of evidence. Named scenarios demonstrate reviewed boundary behavior. Exhaustive finite checks cover declared small domains. Property and mutation tests examine broader classes of implementation mistakes. Symbolic comparison supplies domain-qualified results and replayed witnesses. Differential execution

compares Python and Java semantics. Build records identify exact bytes. Host tests exercise concurrency, idempotency and release selection.

These methods overlap but are not interchangeable. A compiler proof could cover an unbounded class of accepted expressions under stated semantics, while still assuming that the factual adapter and source interpretation are correct. A test suite can catch a concrete adapter error that lies outside that proof. An independent legal review can reject a perfectly verified expression because it misreads an exception. The dossier should state the role of each result so that one strong component does not hide a missing one.

For the current project, the retained engineering evidence includes the local MVP test suite and the separate second-circular exercise. The new ensemble is not yet an implemented live-model product. The appropriate next engineering step is to implement its controller and contracts, then run deterministic acceptance scenarios, then evaluate interpretation performance with independent reference judgments. Declaring the entire product ready because the Java path works would skip the central problem this book is designed to address.

A release reviewer should be able to answer four concrete questions from the package: what rule is being enforced; what evidence makes its inputs meaningful; what demonstrates that these exact bytes preserve the rule; and what happens when an assumption or input is uncertain. If any answer requires guessing from a model confidence score, the dossier is incomplete. Chapter 8 turns these requirements into service records, operations and ordered implementation tasks.

Chapter 8

An implementation contract for the interpretation workbench

8.1 Extend the existing product at a defined boundary

The current repository already has source intake, RuleIR validation and execution, event replay, Java generation, review records, build and release manifests, and a local service interface. Its drafting provider is a deterministic fixture. The new work is an interpretation investigation service between source intake and approval of a candidate bundle. Replacing the entire runtime would discard useful engineering evidence and enlarge the task unnecessarily.

The extension accepts a retained source packet, an activity profile and an explicit investigation policy. It creates inventories, candidates, arguments, discrepancies and bounded actions. Its output is a report and, when appropriate, one or more validated draft bundles. The existing review and release services remain responsible for approval and export, with an additional requirement that the interpretation report and its unresolved issues be checked.

This boundary has two advantages. The interpretation service can evolve its candidate-generation methods without changing deterministic production execution. The Java path can be verified independently of any particular model provider. A candidate that requires an unsupported operator is returned as an engineering question rather than forcing a runtime change through an unreviewed model output.

The companion files under docs / monograph / contracts specify a proposed interpretation.v1 extension. They are design contracts and examples, not a claim that the service is already implemented. The ordered task file under docs/monograph identifies dependencies and acceptance conditions. An implementation agent should read this chapter with those files and the existing v0.1 runtime contracts; it should not infer additional authority from illustrative code in the manuscript.

8.2 Separate durable records from generated prose

The service needs a small set of durable record types. A run binds the source packet, activity profile, policy and required ensemble roles. A candidate binds a proposed meaning to a formal representation and its assumptions. An issue records a discrepancy and its evidence need. An action records one authorized external operation. A report rec-

onciles the run's evidence and terminal status. Review decisions and source coverage records connect these objects to human authority and document completeness.

Generated prose is a field within these records, not their controlling structure. A model may write an explanation saying an issue is resolved, but the service changes the issue's resolution status only through a validated transition with the required evidence. A sentence saying "approved by compliance" has no effect unless an authenticated review record exists for the exact object.

Identifiers are stable opaque strings within a namespace; content hashes identify immutable payloads. These roles should not be confused. A run identifier remains stable while events accumulate, whereas each immutable candidate revision has a new content identity. A human-readable title can change without becoming the primary key. References use identifiers and, where integrity requires it, the expected content hash.

Timestamps use the project's canonical UTC format. Durations are integer milliseconds or seconds with explicitly named units. Counts are nonnegative integers. Monetary reservations use exact minor units and a named billing currency. No field called cost should leave the unit or estimated-versus- actual distinction implicit. This discipline prevents the controller from comparing incompatible budgets.

The schemas reject unknown properties. This catches misspelled limits and unrecognized status fields rather than ignoring them. Structural validation is only the first step: the service also verifies references, state transitions, source compatibility, budget arithmetic and role authority. JSON Schema cannot prove that a source passage supports an interpretation or that two referenced objects belong to the same run.

8.3 The run and policy records

A run records its immutable configuration hash, source packet hash, profile identifier, required roles, creation time, deadline and current lifecycle state. It also references its candidate, issue, action and report collections. Mutable projections such as counters are derived from durable events or updated transactionally with them; they are never trusted solely because a client submits a number.

The lifecycle begins with initialization, proceeds through initial proposals and resolution rounds, and ends in ready-for-review, blocked-unresolved, cancelled or failed-integrity. A ready-for-review run is complete enough to present, not released. A run that is blocked can still contain useful validated candidates. The API returns them with the block, rather than suppressing all information because the overall investigation is incomplete.

The policy supplies all material resource and coverage choices. It names required roles, maximum initial actions, maximum resolution rounds, per-root and global action caps, no-progress limit, action timeout, run duration, candidate count, argument-node count and dependency depth. It also declares whether monetary and token caps are enforced. Configuration validation checks internal consistency, such as requiring the initial-action cap not to exceed the global cap.

The reference fixture policy is deliberately explicit:

Listing 8.1: Reference limits for deterministic controller fixtures.

```
{
  "schema_version": "interpretation.v1",
  "policy_id": "fixture-policy.v1",
  "use": "DETERMINISTIC_FIXTURE_ONLY",
  "max_initial_actions": 4,
  "max_resolution_rounds": 3,
  "max_actions_per_issue_root": 3,
  "max_actions_total": 12,
  "max_no_progress_rounds": 2,
  "action_timeout_seconds": 30,
  "run_deadline_seconds": 600,
  "max_candidates": 24,
  "max_argument_nodes": 200,
  "max_dependency_depth": 6,
  "token_cap": null,
  "cost_cap_minor_units": null
}
```

The candidate, graph and dependency bounds make local processing tractable for fixtures. Reaching one opens or preserves an incompleteness issue. These numbers are convenience choices for engineering tests, not selected production defaults. A paid provider configuration must add enforceable cost and token reservations, and its deployment policy must explain why the chosen limits are sufficient for the intended workload.

A policy version is immutable once used. Changing a cap during a run requires an explicit authorized amendment event if that capability is later implemented; the first prototype should instead require a successor run. This keeps budget semantics simple and prevents a worker from extending its own authority. The successor links to the prior report and explains why additional work is justified.

8.4 The candidate record carries the interpretive commitment

A candidate has a parent revision, source references, subject unit, controlled- language statement, formal bundle reference, assumptions and interpretation family. The family identifies a consequential choice, such as component versus whole-offer assessment. It is not merely a cluster label assigned by a text embedding. A candidate can belong to several dimensions, but its changed choices must be explicit relative to its parent.

Each assumption has a statement, provenance and status. Possible statuses include supported by inspected source, factual evidence required, provisional modeling choice, disputed and rejected. A known fact in a runtime snapshot must not be created directly

from a disputed assumption. The candidate-to-fact mapping is a reviewed transformation with its own evidence requirements.

Supporting and opposing arguments reference source spans or other arguments. An argument distinguishes premises, inference rule and conclusion, and labels strict versus defeasible inference where that distinction is used. If an argumentation engine is not yet implemented, these records can still support a structured review interface. The service must not label the resulting graph as formally evaluated until the selected semantics has actually been run.

Candidate status distinguishes active, deferred, defeated, unsupported and selected-for-review. Deferred means not yet sufficiently investigated, not false. Defeated requires a recorded reason under the chosen argument and authority policy. Unsupported means the current record lacks adequate support. Selected for review is a presentation decision; only a separate authenticated decision can approve the candidate's meaning.

The record includes score components with definitions and calibration status. The initial contract permits a scheduling priority and categorical evidence states. It explicitly leaves probability of legal correctness null. Later calibration can add a versioned, narrowly defined score, but cannot reinterpret old uncalibrated priorities as probabilities after the fact.

Source and assumption identity survive formal deduplication. If two candidates share an identical RuleIR body, the body can be stored once. Their distinct assumptions and source mappings remain separate candidate records. This prevents a compiler optimization from erasing the very disagreement the workbench is intended to expose.

8.5 The issue and action records make automatic work accountable

An issue names its root, parent if any, run, source units, affected candidates, semantic dimension and materiality. It records the observed discrepancy and the question that would resolve it. Processing state and resolution state are separate fields. A terminal processing state can coexist with unresolved meaning.

Resolution evidence is typed. A source-based resolution references retained spans and the reviewer or deterministic rule that accepted them. An adjudication references the authenticated decision. An equivalence resolution references the comparison report, domain and confirmation that no distinct source commitment remains. A bare model statement is not a valid resolution-evidence type.

An action names the run, issue root or initial role, round, permitted action type, input hashes, adapter, timeout, reservation and current state. It also stores the idempotency key, lease owner, fencing token and terminal result reference. The action's question and success criterion are immutable after dispatch. Changing them creates another counted action.

The action result separates transport success from substantive progress. A model request can return valid JSON yet fail to provide any supported new proposition. A retrieval can succeed technically while returning the wrong version. A solver can complete normally with an unknown result. The adapter reports its technical outcome, and the controller validates whether the result answers the action's question.

External text is untrusted data. It cannot name a new tool and cause execution, change a budget, alter role permissions or select an arbitrary filesystem path. Permitted actions come from a server-side registry. The model can propose an action from that registry, but the controller validates its applicability and reserves resources before dispatch. This is both a security boundary and a way to keep the method's stopping proof connected to actual behavior.

The issue record retains attempted actions even when they fail. It is often useful to know that an official dependency was unavailable at the assessment cut-off, that two classification prompts produced no new evidence, or that a formal check was outside the supported fragment. Those facts explain why the investigation stopped and what would justify reopening it.

8.6 Coverage records connect source units to consequences

A coverage record maps each inventoried unit to one or more dispositions: implemented rule, contextual explanation, definition dependency, separate obligation, reviewed out-of-scope provision, or unresolved question. The record includes the reason and responsible reviewer where a legal judgment is involved. A paragraph can have more than one role; it may contain both a definition and an operative condition.

Coverage validation first checks that the record refers to the same source hash as the inventory. It then checks that every inventoried unit appears, every referenced rule or issue exists, and any claim of implemented coverage has a validated bundle and verification evidence. It cannot decide the substantive adequacy of a legal disposition from structure alone, so that judgment remains an explicit review obligation.

Dependencies are edges with types and versions. An incorporated definition is stronger than a background analogy in the sense that missing it can directly prevent interpretation of the selected rule. The impact service uses these edges to identify affected candidates when a source changes. A generic document list would not tell the service which conclusions depend on which definitions.

The source inventory itself is versioned and reviewed. An improved extractor may find a previously missed footnote without any change to the official bytes. That creates a new derivative and inventory revision, not a new official source publication. Candidates whose coverage was assessed against the older inventory need reconsideration. Distinguishing source revision from extraction revision makes that event explainable.

For the second circular, the executable scope remains paragraph 10. The coverage

record must retain all eighteen paragraphs and four footnotes with their respective dispositions. A machine-readable count of one implemented paragraph and seventeen other paragraph dispositions is more honest than a banner saying “circular translated” without a denominator. The report can still present the selected slice as a successful engineering example while showing why whole- document release is not established.

8.7 Storage transactions and event history

The first implementation can extend the existing transactional store. Tables for runs, candidates, issues, actions, coverage revisions, reports and review decisions store canonical payloads with indexed identity and relationship fields. Foreign keys protect references, unique constraints protect idempotency, and transactions protect coupled state changes. The exact storage engine can remain the existing local choice for the prototype; bank deployment requires its own availability and administrative-control assessment.

The most important transaction reserves an action. It reads the current run and root counters under a lock or equivalent serializable mechanism, checks all limits, inserts the action and outbox row, increments counters and appends the audit event. If any step fails, none commits. A later worker cannot dispatch an action that lacks the committed reservation.

A resolution transaction similarly checks the issue revision, validates the resolution evidence, updates the issue projection and invalidates dependent results when necessary. It does not modify immutable candidate or source blobs. The event history records what changed and why, while the current projection supports efficient queries. Recovery can rebuild projections or verify them against the event sequence.

Client idempotency is scoped by authenticated caller and operation. Repeating the same key with the same canonical payload returns the existing result. Reusing the key with a different payload returns conflict. The service must not accept a client-supplied role in the payload as authority. Authentication determines the caller and its permitted operations.

The local audit log is useful for reproducibility but is not automatically tamper-resistant against a database administrator. A bank deployment may require external append-only storage, signed checkpoints or equivalent controls. Those choices are part of the operational security design, not properties conferred by calling a table an audit log. The prototype should preserve enough identity and ordering information to integrate such controls later.

8.8 An API with reviewable state transitions

The proposed HTTP extension uses asynchronous jobs for investigation and explicit resources for review. Creating a run requires the source packet, profile, configuration

and idempotency key. The server validates all references and returns a run identifier and job reference. It does not block the request until every model has completed.

Table 8.1: Proposed interpretation-service operations.
Existing v0.1 endpoints remain separate.

Operation	Purpose	Required behavior
POST/ interpretation-runs	Create a bounded investigation	Validate source/profile/policy; authenticate caller; idempotent creation
GET/ interpretation-runs/ {id}	Inspect current state	Return counters, phase, terminal reason and linked resources
POST/ interpretation-runs/ {id}/cancel	Stop new automatic work	Preserve outstanding actions and produce a cancelled report
GET/ interpretation-runs/ {id}/report	Retrieve a report version	Expose blocked and failed reports as well as review-ready ones
GET/ interpretation-issues/ {id}	Inspect a discrepancy	Return candidates, evidence need, attempts and status separately
POST/ interpretation-issues/ {id}/decisions	Record human adjudication	Require reviewer authority, expected revision and exact evidence links
POST/ interpretation-runs/ {id}/successors	Authorize further work	Require reason, new policy and lineage; never reset the old run

These paths are proposed additions, not descriptions of endpoints already present in the retained OpenAPI file. The implementation task must generate a new OpenAPI contract from actual request models and test it against the service. The book's path table states responsibilities; the machine-readable schema files state the record shapes.

Malformed requests return boundary errors without creating partial runs. Missing references and incompatible source versions return semantic validation errors. A stale expected revision returns conflict so the reviewer can reload the changed issue. Unauthorized decisions fail without writing a review or audit entry that suggests acceptance. The service records denied-operation security evidence through its appropriate logging channel without pretending that the mutation succeeded.

Pagination must preserve completeness. Collection responses include stable cursors and total or explicitly unknown counts; reports bind the complete underlying collection hashes. A user-interface page showing twenty issues cannot imply that only twenty exist. Exports include all issue records or a manifest identifying every page, with integrity checks that detect omission.

8.9 The review interface should expose the decisive difference

A reviewer should begin with the source passage and the decision under review, not a wall of model transcripts. The interface shows the activity and subject profile, the candidate's controlled-language rule, and the facts needed to apply it. A neighboring comparison shows the strongest surviving alternative and a scenario where the outcomes differ.

The discrepancy view explains the type of uncertainty. Missing factual evidence should lead to a data request; unresolved meaning should lead to a legal question; unsupported runtime operations should lead to engineering work. Mixing these under a single confidence badge makes review slower and encourages the wrong remedy. The interface can use concise status labels, with the full evidence one step away.

Source references open the retained version and highlight the exact span. A live link is useful for checking current authority, but it should not replace the retained version used in the run. If the live document differs, the interface shows the change and offers a successor investigation. This prevents a reviewer from unknowingly approving a candidate against a document that changed after its creation.

A decision form asks for the proposition being accepted, the evidence relied on, the treatment of the strongest objection, the scope of the decision and any conditions. It does not ask only for a confidence slider. The authenticated role and exact candidate/report hashes are supplied by the service. The reviewer cannot approve an unseen revision through a stale browser tab.

Unresolved reports remain useful work products. They can present a focused question for the next authorized reviewer, show which offers are affected and identify a temporary operational policy proposal. The interface should make clear whether that policy is a bank choice or a conclusion about the regulator's rule. This distinction is central to keeping cautious operations from becoming false statements of law.

8.10 Provider adapters are deliberately narrow

A provider adapter accepts a typed task and returns a bounded response plus usage and execution metadata. It has no direct database credentials for approvals, release records or budgets. It cannot invoke arbitrary tools selected in model text. The controller supplies the exact retained sources and permitted retrieval scope, and the adapter reports which inputs were actually sent.

The initial interface can support three adapter kinds: scripted fixtures, retrieval tools and model proposals. Scripted fixtures are essential because they make failures reproducible. Retrieval adapters return bytes and acquisition metadata for validation. Model adapters return structured proposals and source references that the controller checks. All adapters use the same action identity and timeout accounting.

The adapter must distinguish refusal, abstention, malformed output, transport failure, timeout and unknown remote result. Treating them all as an empty answer would lose the reason a mandatory role is incomplete. It must also report hidden provider behavior, such as automatic retries or server-side tool execution, when that behavior affects resource accounting or evidence provenance. An integration that cannot bound these behaviors is unsuitable for the unattended profile.

Prompt templates are versioned source files. They state the role, source perimeter, required structured fields and prohibition on treating source text as instructions to the system. The prompt should ask for assumptions and opposing reasons, but those requests do not make the response reliable by themselves. Validation and independent evidence remain necessary.

The service stores provider and model identifiers, relevant sampling settings, template version and source hashes. A provider alias that changes its underlying model must be recorded as such. Reproducibility may be limited even with these fields, so reports distinguish replay of a retained response from regeneration of a fresh model response. Deterministic execution of an accepted bundle does not depend on reproducing the model's original generation.

8.11 Security protects the meaning of the evidence

The source packet can contain malicious or misleading instructions, whether through a compromised document, an uploaded file or quoted text. The interpreter must treat them as source content, not commands. A paragraph saying to ignore prior rules cannot change the controller's budget or authentication policy. The model's task is to analyze that text within the source context; tool permissions come from the server.

Acquisition should retain the existing restrictions on HTTPS destinations, redirects, public addresses, response size and time. A user-supplied URL does not establish official authority. Uploaded documents remain unverified until a separate process establishes provenance. A source that is useful for comparison may still be ineligible to support release.

PDF parsing needs an isolation boundary in a bank deployment. The current local prototype's bounded parser is not an operating-system sandbox for arbitrary hostile files. A production intake worker should have limited filesystem and network access, resource limits and a narrow output contract. This protects the workbench without changing the legal interpretation method.

Bank factual data and public regulatory sources have different access needs. The initial ensemble prototype should use public sources and synthetic scenarios. Sending client or transaction data to a model provider requires the bank's approved data-handling arrangement. The architecture supports minimizing that exposure: legal interpretation can often use abstract fact patterns, while production evaluation remains inside the bank with the generated Java package.

Logs must avoid becoming an uncontrolled copy of sensitive evidence. Store references

and necessary metadata in operational logs, retain full evidence in access-controlled stores, and make redaction policies explicit. A redacted report must preserve enough structure to explain its limitations. Redaction cannot silently remove the only evidence supporting an approval while leaving the approval summary apparently complete.

Security tests should use concrete attacks: source text that requests a tool call, candidate output containing executable markup, a URL redirecting to an internal address, an oversized graph, a forged reviewer role and an action result that tries to change its run identifier. The expected behavior is rejection or safe treatment as data, with a recorded reason. These tests protect the authority boundaries on which the interpretation method relies.

8.12 The ordered implementation sequence

The implementation should proceed in small stages that each produce a reviewable capability. The first stage freezes contracts and builds deterministic examples. The next adds persistence and budgets. Only after the controller behaves correctly with scripted members should it call live model providers. This sequence avoids confusing stochastic model behavior with defects in state management.

Table 8.2: Ordered work packages for the proposed extension.

ID	Deliverable	Acceptance condition
E01	Contract models and reference policy	All valid examples pass; extra fields, incompatible limits and invalid transitions are rejected.
E02	Source inventory and dependency records	Every retained unit has a disposition; missing or mismatched units create issues even under unanimous proposals.
E03	Immutable candidate and assumption store	Revisions retain parents and source hashes; changing a definition invalidates dependent candidates.
E04	Issue identity and lifecycle	Child issues inherit root accounting; terminal unresolved states cannot masquerade as resolutions.
E05	Atomic budget and outbox service	Concurrent reservations never exceed caps; crash recovery preserves issued counts and fencing.
E06	Scripted provider and controller	Omission repair, persistent ambiguity, timeout, no-progress and cancellation traces match the specification.
E07	Structured comparison and witnesses	Different source contexts are detected; supported formal differences replay in Python and Java.

ID	Deliverable	Acceptance condition
E08	Complete report and review interface	Every issue and mandatory role is represented; dissent and unexplored families remain visible.
E09	Review and release integration	Exact report/candidate/build identity is required; all stale or unresolved release attempts are rejected.
E10	Restricted retrieval and model adapters	No uncounted retries or model-directed tools; malformed and unavailable responses retain distinct outcomes.
E11	Independent reference corpus	Source and scenario judgments are frozen before model outputs; disagreement and adjudication are retained.
E12	Equal-budget evaluation harness	Baselines share source access and cost accounting; uncertainty and abstention metrics are reported together.
E13	Optional argumentation/search extension	Small reference graphs and family-preservation tests pass; incomplete search remains explicit.
E14	Bank integration and shadow operation	Real identity, host mappings, operational dispositions and rollback procedures are approved and exercised.

Dependencies matter. E05 depends on the record and issue contracts, while E06 needs E05 and a scripted adapter. E09 depends on complete reports and existing release identity checks. E10 can be developed behind the adapter interface, but live evaluation should wait for E11 and E12. E13 is optional until the simpler scheduler's limitations are measured; it must not delay a clear implementation of bounded disagreement handling.

Each task should specify files to add or extend, existing interfaces to reuse, acceptance identifiers and evidence locations. The companion task JSON provides that structure without changing the existing master plan's completed T00–T22 history. These are new extension tasks, not a retroactive claim that the earlier MVP failed to implement its agreed scope.

An implementation agent should stop a task for a genuine contract inconsistency, missing authority or unavailable required infrastructure. It should not stop merely because a scripted candidate fails: detecting that failure is often the acceptance criterion. A failed interpretation candidate can trigger the next planned repair phase while still blocking release. The task record distinguishes those outcomes.

8.13 Acceptance scenarios that exercise the whole cycle

The first end-to-end fixture uses the explicit type-of-product omission. It freezes the source packet, emits two candidates, creates a discrepancy, reserves a repair action, validates the revised formula, reruns affected checks and produces a report. The report is ready for review only if all other required coverage and dependencies in that fixture are complete. The real second-circular packet remains blocked where its dependencies are unresolved.

The second fixture uses the cashback ambiguity with no resolving authority. It must preserve both supported candidates through the configured rounds and end in a blocked report. The test checks not only the terminal status but also the attempt history, retained objections, question for the reviewer and absence of release authority. This scenario prevents an implementation from treating exhaustion as permission to choose the highest score.

The third fixture tests unanimous omission. All candidate providers return the same incomplete rule, while an independent inventory identifies an uncovered source unit. The controller must open a material issue despite unanimous agreement. If it does not, the ensemble has implemented voting rather than the proposed method.

The fourth fixture tests failures under concurrency. Two workers attempt the last available reservation, one crashes after dispatch, and a stale worker later returns a result. Exactly one reservation may consume the remaining slot; the crashed action remains counted; the stale result cannot overwrite a newer state. The test uses a controlled clock and scripted transport so it is reproducible. Real distributed-system testing later extends this fixture under the deployment infrastructure.

The fifth fixture tests invalidation. A reviewer approves a candidate and report, then a source derivative reveals a previously omitted footnote. The affected coverage and interpretation records become stale. Release with the earlier approval must fail until the impact is reviewed and the required checks rerun. An unchanged Boolean body does not bypass the source-coverage question.

The sixth fixture tests report recovery. A run is cancelled or encounters a malformed provider response. The service must still produce a complete terminal report from durable records. If rich rendering fails, a minimal machine-readable report must remain available. The acceptance condition is not a pretty success screen; it is a reconstructible account of what happened.

8.14 The first vertical slice and its completion rule

A useful first vertical slice implements E01–E09 with scripted members and the public second-circular packet. It exercises source inventory, competing candidates, a repair, a persistent ambiguity, a report and the existing Java path. It does not need an expensive

model integration to test whether the controller's logic is coherent. This is the smallest product-shaped demonstration of the proposed method.

Completion requires more than green unit tests. The service must generate an inspectable report from a fresh run, preserve the exact input and output identities, show all unresolved issues in the interface, and reject a release attempt for the blocked example. A separate synthetic fully resolved fixture can demonstrate the positive review path using local test identities. Its approval status must remain clearly synthetic.

The implementation note should record the command, environment, source hashes, policy, result files and any departures from the design. If a departure changes meaning, budget accounting or approval behavior, it requires a contract revision and new acceptance cases. Minor internal choices can be made by the agent, but must not silently alter observable semantics.

Live models then replace scripted proposal adapters without changing the controller contract. That transition creates a new empirical question: whether the methods find useful alternatives and omissions on unseen circulars at acceptable cost. The next chapter supplies the reference process, baselines and uncertainty analysis needed to answer that question. Engineering completion of the vertical slice is a prerequisite for that study, not its result.

Chapter 9

Measuring error, uncertainty and the value of redundancy

9.1 The experiment must ask the right question

The proposed ensemble has not yet demonstrated improved interpretation of SFC circulars. Its design is motivated by inspected literature, the second-circular exercise and explicit failure analysis. Those are reasons to build and test it, not evidence that its combination of methods is superior. A credible evaluation must distinguish a working controller from a useful interpretation method and from a release process acceptable to the bank.

The main empirical question is whether the method discovers and reports material interpretation errors and omissions more reliably than simpler alternatives at comparable cost. A secondary question is whether its reports help reviewers make better, faster and more traceable decisions. A third concerns resource use: whether targeted resolution spends effort on questions that can actually be answered. These questions require different measurements.

The feared failure is a false clean result: a report appears complete and unambiguous while a material source requirement, alternative reading or factual assumption has been missed. A system that abstains on every case avoids some false clean results but may be operationally useless. Evaluation must therefore measure risk and useful coverage together. Neither raw agreement nor a low average model uncertainty score is an adequate promotion criterion.

Before a live study, write an evidence contract naming the corpus, baselines, reference process, primary outcome, uncertainty analysis, hard vetoes, resource limits and permitted conclusions. Freeze it before examining comparative results. The manuscript proposes such a protocol; it does not report a completed trial. Thresholds that depend on the bank's risk tolerance remain decisions for the responsible owners, not numbers to be invented by the implementation agent.

9.2 Define the unit of evaluation

A circular contains many questions, and a single question can affect many transactions. Counting every transaction as an independent interpretation example would exaggerate sample size when they all depend on the same disputed classification. The evalu-

ation should distinguish document, provision, interpretation question, candidate, scenario and transaction levels.

The primary unit for omission detection is a reviewed provision or interpretation question within a source packet. The source packet includes the circular and required dependencies as of a declared cut-off. Scenario-level outcomes measure how candidate rules behave under facts. Document-level summaries measure whether an apparently complete investigation misses any material issue. These views complement one another but have different denominators.

For example, the 729 ternary assignments are not 729 independent regulatory interpretations. They all arise from one selected formula and six abstract inputs. They are excellent finite-domain engineering coverage for that formula. Treating them as a sample of 729 circulars would produce meaningless uncertainty intervals for legal translation accuracy. The 22 named cases likewise share source, author and modeling assumptions.

The corpus should record these relationships explicitly. Each scenario names its source packet, provision, interpretation question and reference decision. Each candidate names its generation method and source version. Analysis can then aggregate at the appropriate level and use cluster-aware uncertainty estimates rather than treating related rows as independent observations.

An unresolved reference question is still an evaluation object. The desired output may be recognition of ambiguity, preservation of supported alternatives and escalation, rather than selection of a single label. If the dataset forces such questions into a binary answer, it rewards unjustified certainty and penalizes the behavior the bank actually needs.

9.3 Build the reference before seeing the model's answer

The reference process begins with independent source inventory. Reviewers enumerate provisions, footnotes, definitions, exceptions, dates and dependencies. A separate reader checks the inventory against the retained document. The process records which materials were unavailable and which judgments depend on them. The corpus should not silently exclude difficult documents merely because their references are inconvenient to acquire.

For each selected question, at least two appropriately qualified reviewers form initial judgments without seeing the candidate system's answer. They state the reading, assumptions, relevant passages, plausible alternatives and examples that would distinguish them. This is more work than assigning a label, but it creates a reference that can test interpretation rather than superficial answer matching. The exact number and qualifications of reviewers are part of the study plan.

When reviewers disagree, adjudication examines the reasons. A third reviewer or panel can resolve the issue, retain multiple acceptable readings, or declare the available au-

thority insufficient. The adjudication record preserves the original positions and the evidence that changed them. Majority voting among reviewers should not automatically settle a question of authority or source meaning.

Reviewer independence is documented, not assumed. Shared training, a common case summary, prior involvement in the implementation and access to model outputs can all influence judgments. Some collaboration is necessary for adjudication, but initial blinded judgments should remain distinguishable from the later consensus. The study reports both stages.

Reference revisions are inevitable when new evidence appears. They require a versioned correction with a reason. Comparative results are recomputed against the corrected reference, and the report explains the effect. A correction that favors one system is not automatically improper, but it must be justified without using that system's desired score as the reason.

The reference itself is not a universal truth oracle. It is an independently reviewed account of the declared source perimeter at a time. The study can measure agreement with that account, sensitivity to its material issues and handling of its unresolved questions. It cannot establish that no future authority or previously unknown source could change the interpretation.

9.4 Construct a corpus that exposes the expected failures

A useful corpus contains several kinds of difficulty: explicit Boolean conditions, exceptions, definitions supplied elsewhere, actor and activity scope, uncertain classification, amendments, transition periods, event histories, monetary units and interactions between requirements. The initial public examples can anchor the engineering harness, but a method comparison needs additional circular families that were not used to design the prompts or repair logic.

Selection should reflect the intended product perimeter. If the product focuses on private-bank distribution controls, an evaluation consisting entirely of simple product-authorization filing instructions would be a weak test of deployment readiness. Conversely, including arbitrary legal texts outside the supported language can be useful for measuring abstention, but should not be mixed with in-scope accuracy without reporting the distinction.

Temporal separation matters. A holdout containing a later amendment can reveal whether the system improperly reuses an earlier definition or priority. The source packet must state whether the task is historical interpretation as of a cut-off or current applicability review. A model with knowledge of a later rule can otherwise appear to correct a historical answer by importing information that was unavailable at the assessment date.

Synthetic cases are valuable for controlled defects. They can isolate a missing exception,

an ambiguous attachment or a contradictory fact. Their provenance must be visible, and their performance must be reported separately from naturally occurring circular questions. A system can learn the style of seeded mutations without becoming better at discovering unanticipated omissions.

Include negative controls in which no material ambiguity is supported, as well as positive controls with known alternative readings. A method that always invents a dispute should not receive full credit for finding ambiguity. The reference should distinguish a plausible source-supported alternative from an arbitrary logical variation. Review burden from unsupported alternatives is an operational cost worth measuring.

The corpus should also include deliberately incomplete packets. The correct behavior may be to identify the missing dependency and block a conclusion. These examples test whether the system recognizes the limits of its evidence. They must not be scored as ordinary translation errors simply because no final rule is produced.

9.5 Use a baseline ladder with comparable resources

The simplest baseline is a single structured reading with the same source packet and a fixed prompt. It establishes what the ensemble adds beyond ordinary translation. A stronger baseline uses that single reading plus deterministic schema, coverage and execution checks. This prevents the proposed method from claiming credit for checks that do not require an ensemble.

A third baseline uses repeated sampling and answer aggregation, inspired by self-consistency. It receives the same total resource allowance and reports its aggregation method. A fourth uses diverse initial proposals with a fixed, deterministic resolution scheduler. The enhanced method adds the proposed hypothesis-family coverage, targeted questions and optional argumentation or tree search. This ladder isolates the contribution of each mechanism.

Table 9.1: Comparison arms separate redundancy from search and verification.

Arm	Mechanism	Question it answers
B0	One structured proposal	What does ordinary structured translation achieve?
B1	B0 plus deterministic checks	How much comes from source coverage and validation alone?
B2	Repeated proposals with aggregation	Does repeated sampling add useful error detection at equal cost?
B3	Diverse proposals and fixed resolution	Does explicit disagreement handling improve useful coverage?
B4	B3 plus targeted search or argumentation	Does the enhanced scheduler find additional material issues?

Equal resources do not mean merely equal numbers of model calls. Calls differ in input length, output length, retrieval, latency and cost. The study should record both token

or monetary budget and elapsed time, and present comparisons at several predeclared budget levels if that is part of the question. Human reviewer access must also be comparable. Giving only the proposed method a legal reviewer would confound the method with extra expertise.

Prompt and configuration tuning use a development set. The strongest baseline should receive reasonable task-specific tuning rather than a deliberately weak prompt. The test set remains held out until the protocol is fixed. If a test failure triggers a repair, the repaired method is evaluated on a fresh holdout or reported as development evidence, not as an untouched final result.

Search methods require their own assumptions audit. A beam width, family quota, UCT exploration constant or no-progress threshold transferred from puzzles or another repository is a hypothesis. It can be a starting point, but must not become the default without target-specific evidence. The deterministic scheduler is the initial engineering baseline because its behavior is easier to inspect, not because it has been proven optimal.

9.6 Measure omissions, alternatives and useful abstention

Let Q be the set of independently reviewed material questions in the evaluation corpus. For each $q \in Q$, the reference records required conditions, accepted or plausible readings and whether available evidence resolves the question. A material omission occurs when the system's report fails to represent a required condition or unresolved question within its claimed scope. This definition includes missing uncertainty, not only incorrect final formulas.

A false clean result occurs when the system reports a question as resolved and eligible for review without disclosing a material unresolved defect identified by the reference. The denominator is the number of questions or documents declared clean, depending on the reported metric. A system that declares fewer clean cases may reduce this rate while increasing manual workload. The report must show both.

Define useful coverage as the fraction of eligible questions for which the system produces a reference-supported, sufficiently complete result without unresolved material defects. Define selective risk as the error rate among the questions it chooses to resolve. A risk-coverage curve varies the decision policy and shows the tradeoff. The policy must not be tuned on the final test labels after the fact.

Alternative-reading recall measures whether the report contains the reference's material supported alternatives. It should be assessed by meaning and consequence, not string matching. Alternative precision measures how many proposed alternatives are judged relevant and supported. These measures expose a method that increases recall by overwhelming reviewers with arbitrary possibilities.

Resolution quality has a different denominator: detected discrepancies. Measure whether the selected action obtains the needed evidence, produces a valid repair,

correctly escalates, or falsely closes the issue. A successful HTTP response is not a successful resolution. The action's declared question and expected evidence type make this classification possible.

Report completeness is largely an engineering metric. It checks that every issue, role, candidate disposition and resource outcome appears. It can be tested exhaustively over bounded controller fixtures. It does not establish that the issues themselves cover every material legal question. Keeping these metrics separate prevents a perfect report-rendering score from being presented as perfect interpretation accuracy.

9.7 Severity and direction of error need explicit treatment

Not every error has the same consequence. Missing a prohibition may permit an impermissible offer; inventing a prohibition may block a legitimate activity or mislead the bank about its obligations. Both are errors relative to the accepted interpretation, even if one appears operationally conservative. The evaluation should report their directions rather than collapsing them into a single accuracy number.

Severity categories must be defined before scoring. They can consider whether the error changes an action, affects many clients, is hard to detect downstream or undermines the source and approval chain. Assigning weights is a policy choice requiring the bank's input. The study can report unweighted counts and separate severity strata until those weights are approved.

A weighted average can hide a rare severe error among many trivial successes. Therefore severe false clean outcomes should also be reported as individual cases with their mechanisms. A predeclared severe-error veto can block promotion even when average performance is favorable. That veto should not automatically stop all research: if the failure reveals a repairable omission detector, the next planned repair phase may remain justified.

The report should distinguish an invalid experiment from a failed candidate. Leakage of test labels into prompts invalidates the comparison and may require rebuilding the evaluation. A candidate missing an exception on a valid heldout case is evidence against that candidate's readiness. An unavailable provider is an operational failure that may prevent estimating interpretation performance. These outcomes should not all be summarized as "the experiment failed."

Reviewer workload also has severity. A report containing hundreds of unsupported alternatives may conceal the important issue through overload. Measure review time, number of evidence inspections, unresolved-question quality and corrections made by reviewers. A method that catches more errors but makes the review process unmanageable needs a different presentation or search policy, not an unqualified claim of superiority.

9.8 Zero observed errors does not mean zero risk

Suppose a study observes no errors in n independent, identically distributed trials with unknown error probability p . Under this simplified binomial model, the probability of seeing zero errors is $(1 - p)^n$. A one-sided upper confidence bound at level $1 - \alpha$ solves $(1 - p)^n = \alpha$, giving

$$p_{\text{upper}} = 1 - \alpha^{1/n}. \quad (9.1)$$

For a 95 percent bound and $n = 100$, this is approximately 2.95 percent. With $n = 1000$, it is approximately 0.30 percent. These calculations show why a small perfect test set cannot establish a very small error rate.

The independence assumption is substantial. One hundred scenarios derived from one circular do not generally behave like one hundred independent circulars. Shared wording, definitions and prompts create clusters. The calculation is therefore illustrative unless the trial definition and sampling design justify the model. It should not be applied to the 729 truth-table assignments as a legal error bound.

For method comparisons, paired evaluation on the same source questions reduces irrelevant variation. A question-level difference records whether one method finds a material issue that the other misses. Confidence intervals or permutation procedures should preserve the document or question clustering appropriate to the sampling design. Resampling individual rows while ignoring their shared circular can produce overly narrow intervals.

Multiple stochastic generations require multiple seeds or repeated provider runs, with the seed and configuration recorded where supported. Differences from one run are descriptive. If a model API does not guarantee reproducibility from a seed, the study records independent request repetitions and their limitations. The retained responses permit audit even when exact regeneration is impossible.

Tail measures such as maximum review time or the 99th percentile of action cost need particularly cautious interpretation. A few difficult circulars can dominate them, and small samples give unstable estimates. They are useful diagnostics and capacity-planning signals, but a superiority claim requires a predeclared analysis with sufficient observations and uncertainty evidence.

The evaluation report should state which candidates failed hard safety or integrity screens, which remain viable, whether any ranking is statistically supported, and which differences are descriptive only. Passing a hard screen is not evidence that one viable method is better than another. This discipline is especially important when the apparent gain comes from a small number of rare severe errors.

9.9 Calibration asks a narrower question than truth

A calibrated score has a defined relationship to an observed event in a target population. For example, among cases assigned a particular probability range, the fraction agreeing

with the independently reviewed reference should be consistent with that range, within sampling uncertainty. Calibration is not a property conferred by using the word confidence in a JSON field.

The target event must be precise. “The interpretation is correct” is too broad unless the reference process, source perimeter and acceptable alternatives are specified. A more measurable event might be “the proposed paragraph-level rule has no material discrepancy against the adjudicated reference for this corpus.” Even that event depends on the quality and scope of the reference.

Calibration data must be separate from the data used to tune the score and from the final evaluation set. A score trained on one circular family may be miscalibrated on another, especially when definitions, authority structure or source quality change. Model and prompt updates can also invalidate an earlier calibration. The calibration record therefore binds the method and population versions.

Reliability diagrams and proper scoring rules can diagnose probability estimates, but a visually plausible diagram with few cases is weak evidence. Calibration also does not ensure discrimination: a model that assigns the base rate to every case can be calibrated yet unhelpful for selecting review priorities. The study should assess both risk ranking and calibration, with uncertainty intervals.

Semantic entropy can nominate uncertain answers, as discussed in Chapter 5. Its published evaluations use particular datasets and correctness proxies (Kuhn et al., 2023). A high area under a ranking curve in those tasks does not establish calibrated legal error probabilities. In the workbench, entropy may help prioritize review after a target-specific study, while mandatory source and release checks remain categorical requirements.

Low entropy is particularly dangerous under common-mode omission. Every sampled answer can concentrate on the same incomplete reading. The independent inventory and source review remain necessary even if an uncertainty score appears excellent on average. A useful evaluation explicitly includes these unanimous-error cases rather than assuming that disagreement accompanies every mistake.

9.10 Conformal prediction can support sets under stated assumptions

Conformal prediction offers a principled way to construct prediction sets from calibration examples (Angelopoulos and Bates, 2022, section 1.1 and appendix D). It is relevant because the proposed product should sometimes return several candidates rather than a forced answer. Its guarantee, however, concerns a specified statistical prediction problem under exchangeability. It is not a theorem about all possible interpretations of English.

To make the mechanism concrete, suppose each task has a finite reviewed label space $\mathcal{Y}$, such as a small set of predetermined interpretation classes. A fitted scoring function

$s(x, y)$ assigns larger values when label y looks less compatible with task x . The scoring function is fixed before calibration. For n calibration tasks with reviewed labels (X_i, Y_i) , compute $S_i = s(X_i, Y_i)$.

Choose a target miscoverage level α and define

$$k = \lceil (n + 1)(1 - \alpha) \rceil, \quad \hat{q} = \begin{cases} S_{(k)}, & k \leq n, \\ +\infty, & k = n + 1, \end{cases} \quad (9.2)$$

where $S_{(k)}$ is the k th smallest calibration score. For a new task x , return

$$\Gamma(x) = \{y \in \mathcal{Y} : s(x, y) \leq \hat{q}\}. \quad (9.3)$$

The finite-sample correction uses $n + 1$, not simply the empirical $1 - \alpha$ quantile with an unspecified interpolation rule. The inclusive inequality also matters when scores tie.

Why does this work? Conditional on the separately fitted scoring rule, if the n calibration examples and the new example are exchangeable, the position of the new true-label score among the $n + 1$ scores is symmetric. In the no-tie case, its rank is uniform. Excluding it requires a rank beyond k , whose probability is at most α by the definition of k . With ties and inclusive sets, the basic coverage statement is conservative. This gives marginal coverage over a new exchangeable example and the calibration sample.

For $n = 19$ and $\alpha = 0.1$, $k = 18$, so the threshold is the eighteenth ordered calibration score. For $n = 4$ and $\alpha = 0.1$, $k = 5$, requiring the infinite threshold and potentially the entire label space. The latter example makes a practical point: a very small calibration set cannot support a narrow set with a strong finite-sample coverage target through this construction.

The guarantee does not say that each particular circular has a 90 percent chance of containing the right reading in its returned set. Nor does it guarantee coverage conditional on every rare ambiguity type. Marginal coverage can coexist with poor performance on a small important subgroup. The distinction between marginal and conditional coverage is discussed explicitly in the tutorial ([Angelopoulos and Bates, 2022](#), section 3) and is central to a high-stakes application.

The label space is another limitation. If the generator has never proposed a material reading and the label space contains only generated candidates, a set selection method cannot include that missing reading. One can add an “unrepresented interpretation” or escalation outcome to a reviewed task formulation, but that changes the prediction problem and requires reference data. Conformal calibration does not establish semantic completeness of open-ended candidate generation.

The appropriate early experiment is therefore narrow: use a fixed, reviewed classification task or a defined error-detection score, calibrate on separate source families where exchangeability is defensible, and measure set size, coverage and subgroup behavior. Source amendments, provider changes and shifts in document type should trigger renewed assessment. The inspected official tutorial code is useful for checking quantile and set construction, but its image examples are not a ready-made legal calibration protocol.

9.11 Search coverage and semantic completeness are different

A controller can report that it explored every node in a finite generated graph. That is a meaningful computational fact. It does not establish that the graph contains every supported legal interpretation. The generator, inventory and construction operators determine what enters the graph. Evaluation must examine both the search procedure and the generation boundary.

For each reference alternative, record whether it was generated, retained, investigated, correctly dispositioned and represented in the final report. This stage-by-stage accounting locates the failure. If an alternative was never generated, changing the scheduler may not help. If it was generated but pruned by a global beam before investigation, family preservation may be useful. If it was investigated but falsely defeated by a weak argument, the evaluation criterion needs repair.

The same accounting applies to omissions. A source unit can be missed at acquisition, extraction, inventory, normative classification or rule mapping. Treating all such failures as model reasoning errors hides the actionable cause. The ensemble's modular design makes it possible to test which redundant activity actually contributed detection.

Ablations remove one component at a time under comparable budgets: the independent inventory, the alternative reader, the argument critic, the witness generator or the no-progress scheduler. If removing a component changes both resources and information access, the interpretation of the ablation is limited. The study should state whether it measures the component's total contribution or its value at fixed cost.

Search completeness claims must name the finite object. "All twenty-four retained candidates were compared on the declared Boolean domain" is testable. "All legal readings were considered" is unsupported. The report should also expose the unexplored frontier, discarded unsupported proposals and resource-limited comparisons so that reviewers can judge whether more investigation is justified.

9.12 Evaluate the reviewer, not only the model

The final product is a workbench used by people. A study should therefore compare review with and without the ensemble dossier, using a design that controls for case difficulty and learning effects. Reviewers should not see the same case in both conditions in an order that makes the second condition trivially easier. A counterbalanced or appropriately randomized assignment can reduce this problem.

Outcomes include detection of material defects, quality of the final rationale, time to a defensible decision, number of unnecessary escalations and ability to identify remaining uncertainty. Faster review is valuable only if the substantive quality is preserved. A dashboard that encourages reviewers to accept a misleading clean badge can reduce

time while increasing risk.

The study should inspect automation bias directly. Include cases where the leading model answer is wrong, where a minority candidate is right relative to the reference, and where no single answer is justified. Observe whether reviewers inspect the opposing argument and source. The purpose is not to trick staff, but to learn whether the interface supports the intended skeptical review.

Reviewer feedback also tests self-containment. Can a compliance reader understand the formal rule through its controlled-language rendering and examples? Can an engineer identify the required Java behavior without consulting an external paper? Can both explain why a solver result does not resolve an uncertain definition? These comprehension tasks are more informative than asking whether the report looks professional.

Any study involving actual bank staff, confidential materials or production workflows requires the bank's normal authorization and data controls. The public prototype can first use synthetic tasks and qualified independent readers. Its findings should be reported at that level. A successful public-source usability exercise is not evidence that every private-bank process has been validated.

9.13 Promotion decisions and the next evidence needed

The study should predeclare a small set of promotion criteria and hard vetoes. A severe false clean result, broken source identity, uncounted external action, unauthorized release or incomplete mandatory report can veto promotion. Average runtime, model score and cost are explanatory until the correctness and integrity conditions are met. A method cannot trade away a required release invariant for better throughput.

Among methods that pass the hard screens, ranking requires the predeclared uncertainty analysis. If the data do not distinguish them, report them as viable under current evidence. A descriptively favorable method may justify a longer study or an optional feature, but not a new default merely because its mean score is highest in a small sample.

A failed candidate should trigger the repair that its failure supports. Missing source units suggest improving inventory and extraction diversity. Losing a minority interpretation suggests changing family retention. Repeated ineffective actions suggest a better question-selection policy. A contaminated reference requires repairing the evaluation before comparing methods again. These are specific next steps, not reasons to declare the entire research direction either successful or impossible.

Every serious run should retain a manifest with source and code identities, commands, environment, model/provider versions, configurations, seeds or request repetitions, wall time, resource use, plan and result paths. The result note should include the strongest alternative explanation and the evidence that would overturn its conclusion. This makes the evaluation reproducible enough to guide product decisions rather than

Table 9.2: Decision record required after a substantive method comparison.

Decision field	Required interpretation
Primary criterion	State the observed result and uncertainty against the frozen criterion.
Veto status	Identify severe errors, integrity failures and invalid comparisons before discussing speed.
Viable candidates	List methods that remain eligible for further evaluation; do not imply an unsupported ranking.
Main uncertainty	Distinguish reference ambiguity, sampling error, distribution shift and untested implementation.
Next justified action	Promote a bounded use, extend the study, repair a mechanism, or rebuild an invalid harness.
Limit of conclusion	State the source perimeter, task population and deployment claims not established.

merely produce an attractive benchmark table.

The immediate conclusion remains limited: the proposed method is ready to be implemented as a bounded, testable extension, and its empirical legal-interpretation performance remains to be established. The next chapter describes how the bank can preserve that distinction through shadow operation, source changes and ongoing review.

Chapter 10

Operating the product without losing its evidence

10.1 The bank remains responsible for the control

The workbench can organize evidence, expose alternatives, verify consequences and prevent certain unauthorized transitions. It cannot assume the bank's legal or managerial responsibility. The operational design must identify who owns source applicability, interpretation, factual mappings, software correctness and release. A system that assigns every difficult question to a generic "human review" queue has not completed that design.

Meaning review belongs to people authorized to assess the relevant regulatory requirement. Data owners establish what evidence supports facts, how long it remains valid and how corrections are represented. Engineering owners establish that the software preserves the approved specification. Release owners verify the package, approvals and operational readiness. Business owners determine the workflow response to unknown or blocked results within the bank's approved policy.

These responsibilities can overlap institutionally, but conflicts and separation requirements must be explicit. An author should not be able to approve its own interpretation through a second display name. A model-generated reviewer identity has no authority. The local prototype's test roles exercise the mechanics; the bank deployment must bind them to enterprise identity and actual delegated responsibilities.

The product's value is not that it eliminates judgment. It makes the judgment specific and attributable. Instead of asking a reviewer to bless a long generated summary, it asks which definition applies, why an exception attaches to a given condition, or whether a source is current for the activity. The final control then carries those decisions into its executable form and future amendment review.

10.2 Start with a narrow operational perimeter

A first deployment should name the activity, legal entity, products, client or counterparty categories, source families and decision points it supports. This perimeter is not a marketing description. It determines which circulars enter the workbench, which factual schemas are required and which host workflows can consume the resulting package.

For example, a paragraph-10 incentive-control pilot could focus on reviewing a single component of a proposed offer before it is launched. It would need a reliable definition of the component, evidence of its product relationship and a reviewed classification process for gift and fee discount. It would not by itself cover every marketing, disclosure or distribution requirement. The host must continue to route the offer through other applicable controls.

Narrowing the perimeter is useful when it removes an unsupported ambiguity by an explicit operational restriction. If the bank excludes mixed offers from the automated pilot and routes them to manual review, the system can assess simpler cases under a clearer subject definition. That restriction must be enforced by input validation and workflow routing, not merely stated in a project memo. Otherwise mixed offers will eventually arrive and receive an unintended answer.

The perimeter also defines negative tests. Inputs from another actor, product type or jurisdiction should produce the specified out-of-scope or unsupported-profile result. They should not be coerced into the closest known category. A controlled failure at the boundary is preferable to a plausible answer about the wrong activity.

Expansion requires evidence. Adding a new circular family may introduce temporal obligations, different authority relationships or vocabulary that the original schema does not support. The team should create a new source and method review, update the factual mappings, and evaluate the expanded task. Success on a narrow pilot is a reason to consider expansion, not proof that transfer is automatic.

10.3 Keep regulatory meaning distinct from bank policy

A bank can choose a more restrictive operational policy than the minimum condition identified in a circular. It may decline an uncertain promotion, require an additional approval or retain a manual check. These decisions should be represented as bank policy with their own owner, rationale and effective interval. They should not be folded into the regulatory rule and later cited as if the SFC required them.

This separation improves both accountability and maintenance. If a regulatory interpretation changes, the team can see whether the bank policy still makes sense. If a business owner changes an internal approval threshold, it does not appear that the regulator amended the circular. Reports can explain a blocked offer as blocked by bank policy pending classification, rather than falsely declaring it prohibited by the source.

The runtime can evaluate both kinds of controls, provided the bundles identify their authority and purpose. The host's orchestration specifies how their results combine. A regulatory condition returning false may still be blocked by a bank policy; a bank policy returning permissive cannot override a regulatory prohibition. The combination rules need their own reviewed semantics and tests.

An unresolved interpretation should not be hidden in a conservative default. Suppose the team treats every cashback as a gift until clarification. That can be a cautious temporary policy, but the uncertainty report should still state that the legal classification re-

mains unresolved. Otherwise future reviewers may mistake a risk-management choice for settled authority and propagate it into other products.

Temporary policies need expiry or review triggers. An indefinite “pending clarification” rule can become permanent through neglect. The workbench should record the question, owner, next review condition and affected activities. A reminder is an operational aid; it does not automatically extend the policy or resolve the underlying source issue.

10.4 Shadow operation is a study, not a hidden release

Shadow operation runs the proposed control alongside the existing process without letting its result directly determine the transaction. It allows the team to observe real input quality, classification workload, latency and disagreement. The scope and access to real data require the bank’s normal authorization. Shadow results must be clearly distinguished from approved production decisions.

A useful shadow study records the existing process’s decision, the workbench’s result, the facts available to each and the timing of those facts. When they disagree, an independent review examines the reasons. The existing process is not automatically the truth oracle; manual procedures can be wrong. Equally, the workbench is not correct merely because it produces a more detailed explanation. The study needs an adjudication process consistent with Chapter 9.

The team should predeclare what would justify promotion, what would veto it and what would trigger a repair. A severe false clean result might block promotion while motivating a targeted fix. A data-mapping defect might invalidate a subset of observations until corrected. A provider outage may reveal operational fragility without answering the interpretation question. These distinctions keep the study from becoming a succession of improvised demonstrations.

Shadow operation also reveals whether the selected unit of assessment matches business practice. If staff enter a whole campaign while the rule expects one benefit component, the system may receive apparently valid but semantically wrong inputs. The remedy may be a workflow redesign, not a more capable model. The interface should help staff identify and evidence components consistently.

Reports from shadow operation remain subject to access, retention and correction rules. They can contain sensitive facts and preliminary judgments. A later adjudication should append a correction rather than rewrite the original observation. This preserves the study’s evidence and prevents an apparently perfect retrospective record from being created by editing away disagreements.

10.5 Source monitoring begins a new interpretation cycle

The workbench should monitor the source families in its approved perimeter for new publications, revisions and incorporated-material changes. Discovery creates a candidate source record; it does not automatically establish current applicability or overwrite the active rule. The acquisition process retains bytes, metadata, retrieval time and the relationship to prior versions.

A textual difference is only the first signal. A changed definition may affect many controls even when their operative paragraphs are unchanged. A formatting change may have no substantive consequence but still change the file hash. The impact service therefore combines byte identity, extracted text differences, source-unit mapping and the explicit dependency graph.

The first impact report identifies which candidates, facts, rules, duties, reports and releases depend on the changed material. It should distinguish direct citations from indirect definition dependencies. A control can be affected through an incorporated code provision that is not quoted in its main explanation. This is why dependency closure was required before initial interpretation.

An unchanged structural dependency report is not a proof of semantic equivalence. Conversely, a changed paragraph number does not prove a changed obligation. Semantic comparison and legal review are separate actions. The ensemble can propose interpretations of the change and construct scenarios on which old and new rules differ, but a reviewer must establish the intended effective regime.

Every changed rule version requires the relevant review, build and verification steps again. Reusing a previously approved JAR because its human-readable title looks the same would break the evidence chain. Reusing a verified runtime library can be legitimate if its identity and supported semantics remain unchanged, while the new policy bundle and generated binding receive fresh evidence.

10.6 A worked amendment scenario

Consider a synthetic amendment used only to explain the process. The original selected rule contains a fee-discount exception. A later source adds a condition requiring a particular relationship between the discount and the transaction. This is not a claim that the SFC has made such an amendment; it is a controlled example of how a change would propagate.

The source custodian retains the new document and identifies the added condition. The inventory maps it to the earlier exception and opens an interpretation run. Candidate A treats the condition as applying to all offers assessed after the new commencement date. Candidate B distinguishes offers already committed before that date under a possible transition provision. The source packet must include that transition provision if one

exists; the system cannot infer grandfathering from business preference.

The scenario designer creates offers before and after commencement, an offer agreed before but performed after, and an offer with incomplete timing evidence. These examples reveal whether the competing readings differ. The formal checker can compare their executable conditions once the timing concepts are defined. It cannot determine which transition interpretation the source supports without the relevant authority.

When reviewers accept the new interpretation, the data owner confirms that the required relationship and timing facts can be supplied. Engineering builds and verifies the new Java package. The release manifest binds its effective interval and applicability profile. The host's release selector is tested at the exact boundary, including the excluded endpoint of the earlier interval.

Existing event obligations require separate treatment. If a duty was opened under the old bundle, the current MVP preserves that binding. A legal decision may require migration, continuation or another treatment. The workbench records the question and any approved migration specification. No background update silently changes the source version attached to a historical duty.

The old source, interpretation, JAR and decisions remain available for replay. A later investigation can ask what the system would have decided under the old rule with the evidence known then, or what a corrected assessment under the new rule would conclude. Those are different questions and their results carry different assessment contexts.

10.7 Bitemporal replay explains rather than rewrites history

The two-time model is particularly useful in an incident. Valid time asks when a fact or rule applies to the underlying event. Knowledge time asks what information was available to the system at a specified point. A replay must state both. Without this distinction, later corrections can make earlier decisions appear inexplicably wrong or retrospectively perfect.

Suppose a transaction was assessed on Monday using a classification record available that morning. On Wednesday, a reviewer finds evidence that changes the classification effective from Monday. A historical replay with Monday knowledge reproduces the original evidence state. A corrected assessment with Wednesday knowledge can produce a different result for the same Monday transaction. Both results are legitimate records of different questions.

The system should label the corrected assessment and link it to the original. It should not replace the original execution hash or pretend that Wednesday's evidence was available on Monday. This distinction supports fair investigation of whether the failure concerned the rule, data, process or delayed information. It also identifies which other decisions relied on the same evidence.

Replay requires the exact historical runtime or a verified equivalent path, the bundle, fact snapshot, event history where relevant, calendars and assessment context. Retaining only a final status and a current source URL is insufficient. The retention policy must preserve these dependencies for the bank's required period; this book does not invent a statutory retention duration.

Reproducing an old decision does not endorse it. A replay can faithfully show that an earlier rule omitted an exception. That is useful evidence about the incident. The corrected rule and reassessment should be linked, with the reason for change and any operational remediation. Traceability supports accountability precisely because it can preserve an error honestly.

10.8 Operational responses to unknown and conflict

Unknown and conflict are ordinary outcomes in a system built around evidence. They should have designed workflows rather than being treated as exceptional software crashes. The host policy identifies the responsible queue, information request, permitted temporary action and escalation deadline for each reason. These choices depend on the bank's process and must be approved there.

A missing classification might route to a product specialist. Conflicting fee records might route to the data owner. An unavailable incorporated source might require legal review before any automated decision. An unsupported RuleIR operator requires engineering and meaning review together. The reason code helps select the remedy, but the workflow should expose the substantive question in ordinary language.

A conflict should preserve both records and their provenance. Automatically taking the newest record can hide a disagreement between systems or an unauthorized correction. If the bank adopts a precedence rule, that rule must be explicit and reviewed for the evidence type. It should not be embedded in a generic database query that happens to order by timestamp.

Manual overrides need their own authority and scope. An override may authorize a specific operational action under bank policy while leaving the regulatory interpretation unresolved. The record should say exactly what was overridden, by whom, for which case and period, and with what rationale. It must not change the underlying source or make future cases automatically inherit the exception.

Queues themselves can fail. Unreviewed items can accumulate, deadlines can pass, and staff can misunderstand a blocked status. Monitoring should therefore track age, owner, materiality and next action for unresolved issues. A low model error rate is little comfort if the operational process routinely ignores the issues that the model correctly escalates.

10.9 Incident analysis should locate the failed boundary

When an incorrect control or decision is discovered, the first question is where the evidence chain failed. Was the wrong source acquired? Was a qualification missed? Was the accepted interpretation encoded incorrectly? Did the host supply misclassified facts? Did a transaction race invalidate the snapshot? Did an unauthorized or stale package reach production? These mechanisms require different repairs and different assessments of affected decisions.

The incident record should preserve the exact failing example, source versions, candidate, approvals, JAR, input snapshot, result and operational action. It should also preserve the strongest competing interpretation that was available at the time. If the objection was present but suppressed, the remedy concerns reporting and review as well as generation quality.

Containment is a bank decision. It may involve suspending a particular offer category, returning to an approved manual process, disabling a faulty adapter or replacing an implementation while retaining the same accepted rule. The incident team must distinguish these options from reverting to a legally superseded rule. The workbench supplies impact information; it does not decide the bank's external communications or regulatory response.

Root-cause analysis should avoid a generic conclusion that "the AI hallucinated." That phrase can conceal a missing source, a contaminated oracle, an undefined classification or a budget policy that forced premature closure. A specific mechanism supports a specific preventive control. The discovered failure becomes a retained regression scenario, with its source and interpretation status clear.

The team should also ask what similar decisions may be affected. Dependency and lineage queries can identify every package using the disputed definition and every assessment using the relevant fact mapping. A database search by final status alone will miss decisions that happened to be correct for other reasons. The scope of remediation follows the causal dependency, not merely the visible outcome.

10.10 Availability failures must not change the meaning of a result

The production Java evaluator should not depend on a live model or source website. That separation lets an approved control continue to execute during an interpretation-provider outage, subject to the bank's release and source-monitoring policy. An outage in the workbench still matters for new controls and amendment review, but it need not make every existing decision nondeterministic.

The host must distinguish an unavailable interpretation service from an unknown fact and from an expired release. These conditions can require different actions. An ap-

proved, still-applicable local package may remain usable when a research provider is unavailable. A release whose effective status cannot be established presents a different problem. The fallback policy must state the conditions under which each path is permitted.

No fallback may convert a technical error into a substantive false result. If a Java evaluation fails because the snapshot is malformed, returning false would imply that the selected condition was evaluated and not satisfied. The correct result is an error with an operational disposition. Likewise, a missing interpretation report cannot be replaced by a default approval flag.

The ensemble controller has its own recovery obligations. A restart recovers reserved and dispatched actions, preserves their charges and determines which results are known, late or unknown. It does not restart the investigation from zero unless an authorized successor run is created. The supervisor monitors absolute deadlines independently of the worker's progress messages.

Capacity planning should consider the difficult tail: large dependency graphs, many genuine alternatives and long-running external checks. Graph and action limits protect the service, but they create explicit incomplete outcomes. The operational owner needs to know how often those outcomes occur and whether the manual review process can absorb them. Increasing limits without measuring cost and review quality is not a complete capacity strategy.

10.11 Model updates are method changes

A new model version can change which alternatives are generated, how sources are summarized and which questions are prioritized. Even if the JSON schema remains unchanged, the interpretation method has changed. The workbench should bind provider, model, prompt and configuration versions to each run and evaluate material updates before making them the default.

The update review asks whether known omission cases remain detected, minority readings remain visible, unsupported alternatives increase, and uncertainty scores retain their measured relationship to the reference. It also checks provider behavior relevant to budgets, such as retries, tool use and token accounting. A faster model that silently changes these behaviors can invalidate the controller's operational assumptions.

Existing approved Java packages do not need to be regenerated merely because the proposal model changes. Their authority derives from the retained interpretation, review and verification evidence. A new model may identify a concern that triggers review, but its existence alone does not invalidate every prior decision. This separation limits unnecessary churn while allowing genuine discoveries to enter an accountable amendment or incident process.

Prompt changes deserve similar treatment. Adding a leading example can improve performance on one family while anchoring all members to the same interpretation. Removing a request for counterarguments can reduce cost while weakening diversity.

The change record should explain the intended mechanism, evaluation evidence and remaining uncertainty. A prompt is part of the method, not incidental prose.

Search-policy changes also require evidence. Switching from a deterministic scheduler to MCTS, changing family quotas or increasing a no-progress threshold alters which questions receive attention. The study should compare these choices under equal resources and independent judgments. Passing controller invariants is necessary for either scheduler, but does not establish which investigates law more effectively.

10.12 A release readiness review with concrete evidence

A readiness review should inspect a concrete package, not approve a future promise to generate one. The package contains the accepted interpretation dossier, source coverage, factual mapping, verification report, build and release manifests, operational dispositions and unresolved-risk record. All identities must agree. An approval of one candidate cannot be reused for a different generated bundle.

The meaning reviewer checks that the selected source perimeter and assumptions support the rule, that plausible alternatives have a reasoned disposition, and that any remaining uncertainty is compatible with the proposed scope. The data owner checks input definitions, evidence quality, freshness, corrections and calendar or valuation policies. Engineering checks exact behavior, traces, resource limits and integration. The release owner checks authority, deployment mechanics, monitoring and recovery.

A useful readiness table separates evidence from a decision:

Table 10.1: Evidence required for a bounded deployment decision.

Area	Evidence to inspect	Question that remains a human responsibility
Meaning	Source inventory, candidates, objections, adjudication	Is this reading justified for the selected activity and time?
Data	Fact dictionary, provenance, validity, conflict policy	Do the bank's records establish the facts the rule needs?
Execution	Conformance, witnesses, mutations, Java comparison	Is the supported implementation scope adequate for the process?
Authority	Exact manifests, reviewer roles, signatures	Are the required people authorized and accountable for this release?

Area	Evidence to inspect	Question that remains a human responsibility
Operation	Host dispositions, race tests, fallback and incident rehearsal	Can staff act correctly when the result is unknown or blocked?
Evidence	Retained sources, histories, reproducible reports	Can a later reviewer reconstruct the decision without relying on memory?

A missing item should produce a specific block and next action. It should not trigger a vague demand for more confidence. If a classification is unresolved, obtain the authority or narrow the scope. If Java traces differ, repair the implementation. If enterprise identity is not integrated, complete that integration before relying on local synthetic roles. The package makes these distinctions visible enough to assign work.

A review may approve a limited deployment with explicit exclusions and monitoring conditions. Those conditions become enforceable configuration and expiry or review triggers. They must not remain only in meeting minutes. The system should be able to show which operational path enforces each exclusion and which test would fail if it were bypassed.

10.13 Three operational cases for the implementation team

The first case is a missing definition. The source packet is intact, the candidate formula compiles, all named scenarios pass, and the ensemble agrees on the expression. An incorporated definition is unavailable. The correct report is blocked for the affected classification, even though the engineering checks pass. The next action is to obtain or adjudicate the definition, not to add more random truth-table tests.

The second case is a wrong host mapping. Reviewers accepted a component-level rule, but the host sends one Boolean classification for an entire mixed offer. The Java evaluator faithfully applies its inputs and returns a reproducible result. The error lies at the subject and fact boundary. The remedy includes a component-aware adapter, affected-case analysis and a regression scenario that would expose a whole-offer mapping. Improving the Boolean evaluator alone would not address the failure.

The third case is an unresolved exception priority. Two exception guards are true. The current RuleIR default returns conflict, while a developer proposes to choose the first exception in the array. That change would make the program run, but it would introduce a priority that the specification did not authorize. The workbench must preserve conflict and ask whether the source establishes priority, whether explicit nesting is justified, or whether the rule lies outside the supported fragment.

These cases demonstrate why completion criteria must follow the claimed target. In the

first, source evidence is incomplete. In the second, the application mapping is wrong relative to the accepted interpretation. In the third, a proposed implementation change would alter semantics. The same dashboard label “needs review” is not enough; the system must name the responsible boundary and the evidence needed to repair it.

A team rehearsal should walk through each case using the actual interface and reports. Staff should be able to locate the source, identify the affected candidate, understand the block and record a decision without editing hidden configuration. Rehearsal findings become product changes and acceptance tests. A successful technical build does not substitute for this operational exercise.

10.14 Maintaining a useful literature and method record

The paper library is a working research asset. Each retained paper has a stable file-name, source URL, version or publication information, digest and reading status. The title-and-first-author naming convention requested for this project helps people locate a paper, while the manifest disambiguates revised preprints and related editions. A citation should point to the version actually inspected.

The bibliography and reading notes distinguish published results, local derivations and proposed adaptations. Catala’s treatment of defaults, Stipula’s state and resource semantics, argumentation frameworks, tree search and uncertainty methods answer different questions. Combining them in one architecture does not transfer every theorem or empirical result to the combination. The method record states which mechanism is borrowed and what must be re-established locally.

Official code is valuable evidence about practical assumptions. The semantic- clustering example in Chapter 5 shows why code and paper can require separate inspection. A pinned repository revision may implement a broader condition than the prose suggests. The correct response is to document and test the difference, not silently choose the version that makes the proposal look stronger.

The literature record should also preserve inaccessible or uninspected leads. They can guide future work, but cannot support categorical implementation claims. The present review is a substantial bounded survey of the retained sources, not a declaration that no relevant paper exists outside the library. New work can change the best engineering choice, especially in a rapidly developing area.

ResearchAssistant is useful for acquisition, parsing and source structure. MathDevMCP is useful for named formal obligations. DynareMCP contributes patterns for explicit hypotheses, falsifiers and bounded investigations. Their reuse should remain concrete and versioned. The bank should be able to identify which function, data structure or proof obligation is being borrowed, rather than relying on a broad claim that another project already solved the difficult part.

10.15 A practical sequence from manuscript to product

The first step is contract implementation with public sources and scripted members. This establishes that the ensemble can preserve alternatives, detect specified discrepancies, respect limits and produce complete reports. The second-circular example provides a running integration case, while synthetic fully resolved cases exercise the positive review route. The original blocked example should remain blocked unless actual new evidence resolves its issues.

The second step is independent reference construction and restricted live-model evaluation. The team freezes source inventories and judgments before seeing model outputs, runs the baseline ladder and records uncertainty. Failures are classified by boundary and used for planned repairs. No model is promoted because it merely reproduces the author's original 22-case oracle.

The third step is enterprise integration: identity, approved data access, isolated source parsing, provider controls, signed or equivalently authenticated releases, monitoring and host transaction behavior. This work can proceed alongside method evaluation where dependencies permit. It should not be postponed until a final demo, because operational constraints may change which designs are feasible.

The fourth step is authorized shadow operation in a narrow activity perimeter. The study compares the workbench with the existing process and independently adjudicates disagreements. It measures both substantive quality and reviewer workload. The team rehearses source amendments, unknown facts, provider outages, worker crashes and rollback decisions.

The fifth step is a bounded deployment decision based on the concrete evidence package. The decision may approve only a subset of controls or retain manual review for disputed categories. Monitoring then continues the evidence process: new sources, unexpected disagreements and incidents create successor investigations. Deployment is not the end of interpretation assurance.

This sequence does not assign an arbitrary calendar duration or staffing number. The bank's source perimeter, access constraints and review capacity determine those estimates. The task dependencies and acceptance evidence are concrete enough to build a schedule once those inputs are known. Inventing a confident delivery date without them would add apparent precision without resolving the underlying work.

10.16 What the completed prototype will and will not establish

A completed deterministic prototype will establish that the specified records, controller, budgets, reports and release checks behave as tested. It will produce actual Java packages from accepted RuleIR and preserve the existing deterministic execution contract. It will demonstrate that known omissions and persistent ambiguities lead to the intended

different outcomes in scripted scenarios.

A successful independent interpretation study can establish empirical performance on its declared task population, with uncertainty and limitations. It may support a bounded use or a choice among methods. It cannot prove that every future circular will be interpreted correctly, especially under new vocabulary, authority or distribution shift. A formal proof of the controller's release invariant similarly depends on the recorded evidence and does not prove that no source obligation was ever missed.

These limits do not make the product pointless. The present manual process also contains interpretation, transcription, coding and review errors, often without a complete record of competing readings. The proposed workbench makes those stages explicit, introduces complementary checks and preserves the questions that remain. Its value should be demonstrated against that real process with a fair reference and comparable resources.

The high stakes justify both ambition and precision. The system should actively search for ways its preferred reading could be wrong, retain those challenges, and stop automatic work honestly when the evidence or budget is insufficient. It should deliver code whose relationship to an approved specification can be checked, while keeping the approval of meaning visible as a separate act. That combination is a concrete engineering objective and a defensible basis for building the next version of LegalMath.

Bibliography

- Daneshvar Amrollahi, Jerry Lopez, and Clark Barrett. Faithful Autoformalization via Roundtrip Verification and Repair, 2026. URL <https://arxiv.org/pdf/2604.25031v2>. Reviewed preprint v2, 9 May 2026.
- Chenyang An et al. A Neurosymbolic Approach to Natural Language Formalization and Verification. arXiv:2511.09008, 2025. URL <https://arxiv.org/pdf/2511.09008v2>. First posted 2025; reviewed v2, 13 July 2026.
- Anastasios N. Angelopoulos and Stephen Bates. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. arXiv:2107.07511, 2022. URL <https://arxiv.org/pdf/2107.07511v6>. Reviewed v6, 7 December 2022; first posted 2021. Split-conformal method, marginal/conditional coverage, shift discussion and appendix D proof inspected.
- Grigoris Antoniou, David Billington, Guido Governatori, and Michael J. Maher. Representation Results for Defeasible Logic. arXiv:cs/0003082, 2000. URL <https://arxiv.org/pdf/cs/0003082>. Reviewed 2000 preprint; journal version published in 2001.
- Joaquín Arias, Mar Moreno-Rebato, José A. Rodríguez-García, and Sascha Ossowski. Automated legal reasoning with discretion to act using s(LAW). Artificial Intelligence and Law, 2024. URL <https://arxiv.org/pdf/2401.14511>. 2024 arXiv copy; journal first published online in 2023.
- Tara Athan, Guido Governatori, Monica Palmirani, Adrian Paschke, and Adam Wyner. Legal Interpretations in LegalRuleML. CEUR Workshop Proceedings, 1296, 2014. URL <https://ceur-ws.org/Vol-1296/paper2.pdf>. Workshop paper.
- T.J.M. Bench-Capon. HYPO’s Legacy: Introduction to the Virtual Special Issue. Artificial Intelligence and Law, 25, 205–250, 2017. URL <https://www.csc.liv.ac.uk/~tbc/publications/hypoLegacy.pdf>. Author manuscript; virtual-special-issue introduction.
- Trevor Bench-Capon and Giovanni Sartor. A Model of Legal Reasoning with Cases Incorporating Theories and Values. Artificial Intelligence, 150, 97–143, 2003. URL <https://www.csc.liv.ac.uk/~tbc/publications/aijsartor.pdf>. Author-hosted full text; publication year and DOI follow the journal record.
- Alison Chorley and Trevor Bench-Capon. AGATHA: Using heuristic search to automate the construction of case law theories. Artificial Intelligence and Law, 13, 9–51, 2005. URL <https://www.csc.liv.ac.uk/~tbc/publications/agatha.pdf>. Author PDF title page is dated 2006; Crossref records print publication 2005 and online publication 2006; cited as volume 13 (2005).

- Silvia Crafa, Cosimo Laneve, and Giovanni Sartor. Pacta sunt servanda: legal contracts in Stipula. arXiv:2110.11069, 2021. URL <https://arxiv.org/pdf/2110.11069>. Reviewed three-author 2021 preprint; later journal version has a different author list.
- Giorgio Delzanno, Cosimo Laneve, Arnaud Sangnier, and Gianluigi Zavattaro. Clause-reachability is undecidable in legal contracts. *International Journal on Software Tools for Technology Transfer*, 28, 255–275, 2026. URL <https://link.springer.com/content/pdf/10.1007/s10009-026-00841-5.pdf>. Published 19 March 2026; June issue.
- Christopher A. Esterhuyse, Tim Müller, and L. Thomas van Binsbergen. A Stable Model Semantics for eFLINT Norm Specifications and Model Checking Scenarios. *GPCE 2025*, 80–93, 2025. URL <https://pure.uva.nl/ws/files/270385564/3742876.3742882.pdf>. University repository copy with cover; distinct semantics from the earlier interpreter.
- Thomas F. Gordon, Henry Prakken, and Douglas Walton. The Carneades model of argument and burden of proof. *Artificial Intelligence*, 171, 875–896, 2007. URL <https://webpace.science.uu.nl/~prakk101/pubs/GordonPrakkenWalton2007a.pdf>. Author-hosted full text; inspected formal definitions and burden-of-proof sections.
- Pierre Goutagny, Aymeric Fromherz, and Raphaël Monat. CUTEcat: Concolic Execution for Computational Law. *Programming Languages and Systems (ESOP 2025)*, 2025. URL https://link.springer.com/content/pdf/10.1007/978-3-031-91121-7_2.pdf. Publisher open-access chapter.
- Guido Governatori and Antonino Rotolo. Logic of Violations: A Gentzen System for Reasoning with Contrary-To-Duty Obligations. *Australasian Journal of Logic*, 4, 193–215, 2006. URL <https://ojs.victoria.ac.nz/ajl/article/download/1780/6684>. Title-page publication 2006; migrated index year 2018 is not used.
- Guido Governatori, Francesco Olivieri, Antonino Rotolo, and Simone Scannapieco. Computing Strong and Weak Permissions in Defeasible Logic. arXiv:1212.0079, 2012. URL <https://arxiv.org/pdf/1212.0079>. Reviewed December 2012 preprint; later journal DOI.
- Mustafa Hashmi, Guido Governatori, and Moe Thandar Wynn. Normative requirements for regulatory compliance: An abstract formal framework. *Information Systems Frontiers*, 18, 429–455, 2016. URL <https://eprints.qut.edu.au/83732/23/Main.pdf>. Author manuscript; online publication 2015, issue 2016.
- Liane Huttner and Denis Merigoux. Catala: Moving Towards the Future of Legal Expert Systems. *Artificial Intelligence and Law*, 2022. URL

- <https://inria.hal.science/hal-02936606/document>. Accepted manuscript, 10 August 2022.
- Reiner Hähnle, Cosimo Laneve, and Adele Veschetti. Formal Verification of Legal Contracts: A Translation-based Approach (Extended Version). arXiv:2509.20421, 2025. URL <https://arxiv.org/pdf/2509.20421v2>. Extended version, v2, 26 September 2025.
- William Jurayj, Nils Holzenberger, and Benjamin Van Durme. Language Models and Logic Programs for Trustworthy Tax Reasoning. Proceedings of AAAI, 40(45), 38688–38698, 2026. URL <https://ojs.aaai.org/index.php/AAAI/article/download/41212/45173>. Published proceedings copy.
- Alan Khoja, Martin Kölbl, Stefan Leue, and Rüdiger Wilhelmi. Automated consistency analysis for legal contracts. Artificial Intelligence and Law, 2025. URL <https://link.springer.com/content/pdf/10.1007/s10506-025-09456-8.pdf>. Publisher version.
- Yuta Koreeda and Christopher D. Manning. ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts. Findings of EMNLP 2021, 2021. URL <https://aclanthology.org/2021.findings-emnlp.164.pdf>. ACL Anthology proceedings copy.
- Robert Kowalski, Jacinto Dávila, Galileo Sator, and Miguel Calejo. Logical English for Law and Education. Prolog: The Next 50 Years, 2023. URL <https://www.doc.ic.ac.uk/~rak/papers/Logical%20English%20for%20Law%20and%20Education%20.pdf>. Author-hosted version; third author is Galileo Sator as on title page.
- Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. ICLR 2023, 2023. URL <https://arxiv.org/pdf/2302.09664v3>. Reviewed v3, 15 April 2023; sections 3–6 and appendix A.1–A.2. Official clustering source inspected separately; no experiment run.
- Cosimo Laneve. Liquidity analysis in resource-aware programming. Journal of Logical and Algebraic Methods in Programming, 135, 100889, 2023. URL https://www.cs.unibo.it/~laneve/papers/Stipula_LiquidityFULL.pdf. Reviewed author preprint dated 5 October 2022; journal edition 2023.
- Cosimo Laneve, Alessandro Parenti, and Giovanni Sartor. Legal Contracts Amending with Stipula. Author-hosted manuscript, 2023. URL https://www.cs.unibo.it/~laneve/papers/H0_Stipula_llncs.pdf. 2023 author-hosted edition; do not identify this PDF with the different 2024 chapter Programming Contract Amending.
- David Makinson and Leendert van der Torre. Input/Output Logics. Journal of Philosophical Logic, 29, 383–408, 2000. URL <https://icr.uni.lu/leonvandertorre/papers/jpl00.pdf>. Author-hosted manuscript.

- Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A Programming Language for the Law. *Proceedings of the ACM on Programming Languages*, 5 (ICFP), 2021. URL <https://arxiv.org/pdf/2103.03198v2>. arXiv v2, 2 July 2021.
- Raphaël Monat, Aymeric Fromherz, and Denis Merigoux. Formalizing Date Arithmetic and Statically Detecting Ambiguities for the Law. *Programming Languages and Systems (ESOP 2024)*, 2024. URL https://link.springer.com/content/pdf/10.1007/978-3-031-57267-8_16.pdf. Publisher open-access chapter.
- Robert D. Nowak. The Geometry of Generalized Binary Search. *IEEE Transactions on Information Theory*, 57(12), 7893–7906, 2011. URL <https://arxiv.org/abs/0910.4397v5>. Reviewed arXiv v5 (25 June 2013); journal publication 2011.
- OASIS LegalRuleML Technical Committee. LegalRuleML Core Specification Version 1.0, 2021. URL <https://docs.oasis-open.org/legalruleml/legalruleml-core-spec/v1.0/os/legalruleml-core-spec-v1.0-os.html>. OASIS Standard, 30 August 2021.
- Object Management Group. Decision Model and Notation, Version 1.5, 2024. URL <https://www.omg.org/spec/DMN/1.5/PDF>. Formal specification; hit-policy discussion in section 8.2.11.
- Theo X. Olausson, Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. *Proceedings of EMNLP 2023*, 2023. URL <https://aclanthology.org/2023.emnlp-main.313.pdf>. ACL Anthology proceedings copy.
- Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. *Findings of EMNLP 2023*, 2023. URL <https://aclanthology.org/2023.findings-emnlp.248.pdf>. ACL Anthology proceedings copy.
- Henry Prakken. An abstract framework for argumentation with structured arguments. *Argument and Computation*, 1, 93–124, 2010. URL <https://webpace.science.uu.nl/~prakk101/pubs/aspicAF.pdf>. Author-hosted full text; correction page retained in the staging directory.
- M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, and H. T. Cory. The British Nationality Act as a Logic Program. *Communications of the ACM*, 29(5), 370–386, 1986. URL <https://www.doc.ic.ac.uk/~rak/papers/British%20Nationality%20Act.pdf>. Author-hosted scan.

- SFC. Circular on distributors providing additional returns and other services or arrangements when marketing SFC-authorised funds. 24 October, reference 23EC46, 2023. URL <https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=23EC46>.
- Sepehr Sharifi, Alireza Parvizimosaed, Daniel Amyot, Luigi Logrippo, and John Mylopoulos. Symboleo: Towards a Specification Language for Legal Contracts. IEEE International Requirements Engineering Conference 2020, 2020. URL https://drive.google.com/uc?export=download&id=1WXwXeLrZdaJjhSJcCrt_wBXxDvhFkq2k. Author preprint linked by official Symboleo repository; some ancillary semantics references remain anonymized.
- L. Thomas van Binsbergen, Lu-Chi Liu, Robert van Doesburg, and Tom van Engers. eFLINT: a Domain-Specific Language for Executable Norm Specifications. GPCE 2020, 2020. URL <https://ir.cwi.nl/pub/29922/29922.pdf>. CWI author copy.
- L. Thomas van Binsbergen, Christopher A. Esterhuyse, and Tim Müller. Reflections on the design, applications and implementations of the normative specification language eFLINT. *Journal of Computer Languages*, 2026. URL <https://arxiv.org/pdf/2511.12276v3>. Reviewed arXiv v3, 4 August 2026; journal title differs slightly; first preprint 2025.
- Olivia Peiyu Wang, Sanna Wong-Toropainen, Daneshvar Amrollahi, Ryan Bai, Tashvi Bansal, Arush Garg, and Leilani H. Gilpin. Know Your Limits: On the Faithfulness of LLMs as Solvers and Autoformalizers in Legal Reasoning. arXiv:2606.16118, 2026. URL <https://arxiv.org/pdf/2606.16118v2>. v2, 19 June 2026; under review; methodological caveats retained.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023, 2023. URL <https://arxiv.org/pdf/2203.11171v4>. Reviewed v4, 7 March 2023; first posted 2022. Method, sampling, evaluation and aggregation inspected; no benchmark replay.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601, 2023. URL <https://arxiv.org/abs/2305.10601v2>. Reviewed arXiv v2 (3 December 2023); the method explores partial thoughts with breadth-first and depth-first search.
- Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. *Proceedings of ICML 2024*, PMLR 235, 2024. URL <https://arxiv.org/abs/2310.04406v3>. Reviewed arXiv v3 (6 June 2024); first posted 2023.